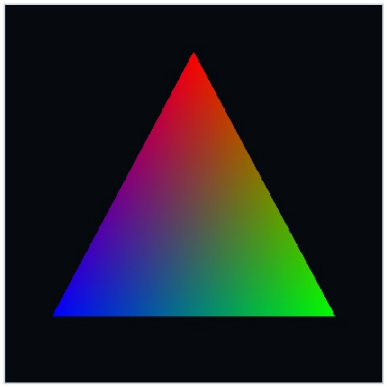
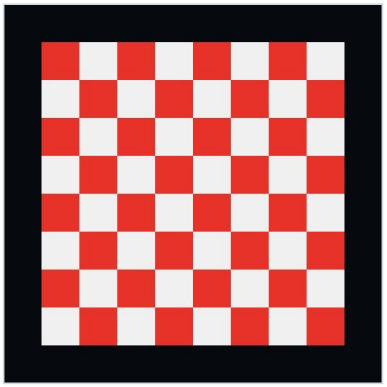
Vulkan 自学教材

从第一张图片到纹理和连续帧

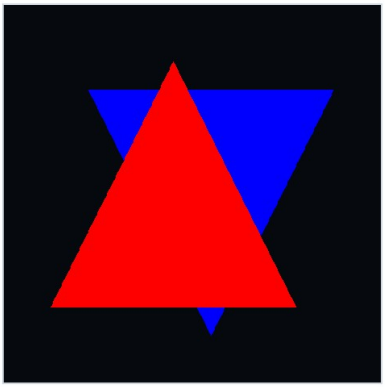
C++17 Vulkan 1.3 可运行实验与逐题解析



顶点与颜色



纹理采样



深度测试

从 CPU 像素数组出发，逐步理解资源的创建与绑定、Shader 的数据输入、矩阵变换、纹理采样、深度测试和同步。主线功能均附有可运行源码与可检查的输出结果。

适合具备基本 C++ 阅读能力、但还没有系统学习过图形 API 的读者。

2026 年 10 月 31 日 · 语言审校修订版

目录

阅读这本教材.....3
第 1 章 从一张图片开始.....4
第 2 章 构建工程并找到设备.....8
第 3 章 为图像准备资源和内存.....12
第 4 章 记录清屏和复制命令.....17
第 5 章 提交工作并取回图片.....22
第 6 章 三角形从顶点到像素.....27
第 7 章 顶点 Buffer 与字节布局.....31
第 8 章 相机矩阵与 Shader 资源接口.....35
第 9 章 纹理上传与采样.....42
第 10 章 mipLevels 与 arrayLayers.....46
第 11 章 多物体与深度.....52
第 12 章 从前后读写推导同步.....57
第 13 章 CPU 与 GPU 并行及连续帧.....61
第 14 章 窗口接入设计（拓展）.....66
第 15 章 从报错到定位问题.....70
第 16 章 综合练习与后续学习.....74
实验复现与验证记录.....78
一手资料索引.....80

阅读这本教材

本书从一段生成图片的 C++ 程序开始，逐步将同一个任务迁移到 GPU，最终实现连续多帧绘制。学完后，你应当能够修改配套工程，并解释每个新增资源解决了什么问题、Shader 怎样取得数据，以及何时可以修改或释放资源。

阅读前只需掌握 C++ 的函数、结构体、指针和数组；需要用到的图形学知识会在相应章节讲解。实验基于 C++17 和 Vulkan 1.3 的 C 接口，使用一条图形队列、动态渲染（Dynamic Rendering）和 Synchronization2。主线生成可保存、可比较的离屏图片；窗口呈现留到第 14 章作为扩展内容。

怎样进行一次有效的自学

建议先运行原始工程，再阅读本章涉及的函数。每段需要动手修改的代码都标出了文件和函数位置；请在对应位置改动，而不是把所有片段拼进新的 main 函数。为便于排版，书中较长代码可能调整换行；配套工程保留完整的创建、提交、错误处理与资源释放流程。

每次练习遵循同一顺序：先预测结果，再修改和运行，最后对照图片与日志。请尽量先独立作答，再查看附带的推理过程。遇到问题可以查第 15 章。页面中的长命令可能因排版折行，实际输入时仍应作为一条命令执行。

学习路线与运行入口

下表按照学习顺序列出了各章实验入口。第 2 章介绍构建与命令行用法；除另有说明外，所有相对路径都以解压后的 lab 目录为起点。

章节	完成的事情	对应入口
1 至 2	写出 CPU 图片，构建工程，查询 Vulkan 设备	cpu_reference 与 probe
3 至 5	创建图像，记录清屏，提交并读回	clear
6	理解顶点到像素的过程	triangle
7	从 Buffer 读取顶点	vertex
8	通过 Uniform Buffer 传递矩阵	camera
9	上传图片并在 Shader 中采样	texture
10	选择不同 mip 级别和数组层	mips 与 layers
11	让重叠的表面按深度显示	depth
12 至 13	推导数据依赖，安全更新连续帧	frames
14 至 16	理解窗口接入，定位错误，完成综合练习	拓展阅读与已有实验的修改

完成一章后，合上教材，你应当仍能找到对应函数，完成一个小修改，并说清输出变化的原因。

第 1 章 从一张图片开始

本章先不使用 Vulkan，而是生成一张 512×512 的深蓝色图片。先弄清图片的像素在内存中怎样排列，再学习 Image、Buffer 和 GPU 读回，后续概念就有了可以对照的实例。

图片首先是一组数值

把数字图片放大，可以看到一格一格、按行排列的像素。本章使用三个 8 位无符号整数，分别表示红、绿、蓝三个分量；每个分量的取值范围是 0～255。三个分量都是 0，表示黑色；都是 255，表示白色；只有红色分量是 255，则表示纯红色。这里先采用这种简单格式，后面还会遇到浮点数格式与深度格式。

图像宽、高均为 512，因此共有 512×512＝262144 个像素。每个像素占 3 字节，所以不计文件头的 RGB 数据共占 786432 字节。这个大小只适用于本章约定的格式。GPU 图像可能使用其他格式、包含多个 Mip Level 或 Array Layer，也可能有额外的内存对齐要求，不能直接沿用这个结果。

为了直接输出图片，我们采用 PPM 的 P6 格式：文件开头用文本写明宽度、高度和分量最大值，随后紧跟二进制 RGB 像素数据。像素按行保存：先从上到下排列各行，再从左到右排列每行的像素。本书用它检查实验的数值结果，写文件时不另外执行颜色空间转换；完整的颜色管理不在本次实验范围内。[1]

第一段完整程序

打开 extras/cpu_reference.cpp。下面给出这个文件的完整内容。第 2 章会说明如何通过工程构建它；如果你已经有 C++17 编译器，也可以先独立编译这个文件。

如果已经安装 GNU 或 Clang C++ 编译器，可以在终端中，进入 lab 目录可先运行：

```
c++ -std=c++17 extras/cpu_reference.cpp -o cpu_reference
./cpu_reference out/cpu.ppm
```

如果使用 Windows 的 Visual Studio 工具链，可以先阅读第 2 章，按统一的 CMake 流程构建 cpu_reference。

```
#include <array>
#include <cstdint>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <vector>

// A CPU-only starting point for Chapter 1. No Vulkan is used here.
int main(int argc, char** argv) {
    try {
        const std::filesystem::path output =
            argc > 1 ? argv[1] : "out/cpu.ppm";
        constexpr std::uint32_t width = 512;
        constexpr std::uint32_t height = 512;
        const std::array<std::uint8_t, 3> color{6, 9, 14};
```

```
        std::vector<std::uint8_t> pixels(width * height * 3);

        for (std::uint32_t y = 0; y < height; ++y) {
            for (std::uint32_t x = 0; x < width; ++x) {
                const std::size_t offset = (y * width + x) * 3;
                pixels[offset + 0] = color[0];
                pixels[offset + 1] = color[1];
                pixels[offset + 2] = color[2];
            }
        }

        if (!output.parent_path().empty()) {
            std::filesystem::create_directories(output.parent_path());
        }
        std::ofstream file(output, std::ios::binary);
        if (!file) throw std::runtime_error("Cannot open output file");
        file << "P6\n" << width << ' ' << height << "\n255\n";
        file.write(reinterpret_cast<const char*>(pixels.data()),
                   static_cast<std::streamsize>(pixels.size()));
        if (!file) throw std::runtime_error("Cannot write output file");
        std::cout << "CPU image: " << output.string() << '\n';
        return 0;
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << '\n';
        return 1;
    }
}
```

先看中间的两层循环。外层遍历各行，内层遍历当前行的像素。表达式 `(y * width + x) * 3` 的含义可以分成两步：`y * width + x` 得到像素在整个一维数组中的序号，再乘 3 得到这个像素第一个分量的字节偏移。随后三个赋值分别写入红、绿、蓝。

例如宽度为 512 时，坐标 `(1, 0)` 是第一行的第二个像素，它的 RGB 从偏移 3 开始；坐标 `(0, 1)` 是第二行的第一个像素，从字节偏移 1536 开始。这里没有任何 GPU 特有的知识，只是在说明二维位置怎样对应到一段线性存储。

`std::ios::binary` 指定以二进制方式写出。文件头是可读的文本，像素数据却不是字符串“6 9 14”，而是三个值分别为 6、9、14 的原始字节。如果把它们转换为十进制字符串再输出，就不符合本程序声明的 P6 数据格式。

动手修改颜色并检查结果

按上一节独立编译后，运行 `./cpu_reference out/cpu.ppm`。你会得到一张深蓝色的纯色图片。然后把 `color{6, 9, 14}` 改成 `color{255, 0, 0}`，重新编译，并将结果写入另一个文件，例如 out/cpu_red.ppm。整张图都应该变成红色。文件尺寸不变，因为你改变了像素值，没有改变像素数量或存储格式。如果按照第 2 章使用 CMake 构建，则应运行该章指定的 build 目录中的可执行程序。

如果图片查看器不支持 PPM，第 2 章提供的脚本可以把它转换成 PNG。若文件不存在，先看程序是否输出了 `Error:`；若找不到预期位置的文件，检查运行命令所在的工作目录。相对路径 out/cpu.ppm 总是相对于运行程序时的目录，不是源文件所在目录。

同一任务交给 GPU 后，会增加哪些步骤？

CPU 版本的执行顺序比较直观：循环将数组写完后，后续代码就可以读取其中的数据。Vulkan 则经常先让 CPU 录制命令，再把这些命令交给 GPU。录制命令的函数返回，并不等于 GPU 已经执行了它；即使提交函数返回，也不意味着图像已经写好。[2]

因此，要把同一个任务交给 GPU，我们需要先创建可供 GPU 写入的图像，再录制清屏和复制命令，将命令提交给队列，等待 GPU 完成，最后让 CPU 读取结果。第 2～5 章将依次解释这些步骤。最后得到的仍是深蓝色图片，便于逐项确认每个新增对象的作用。

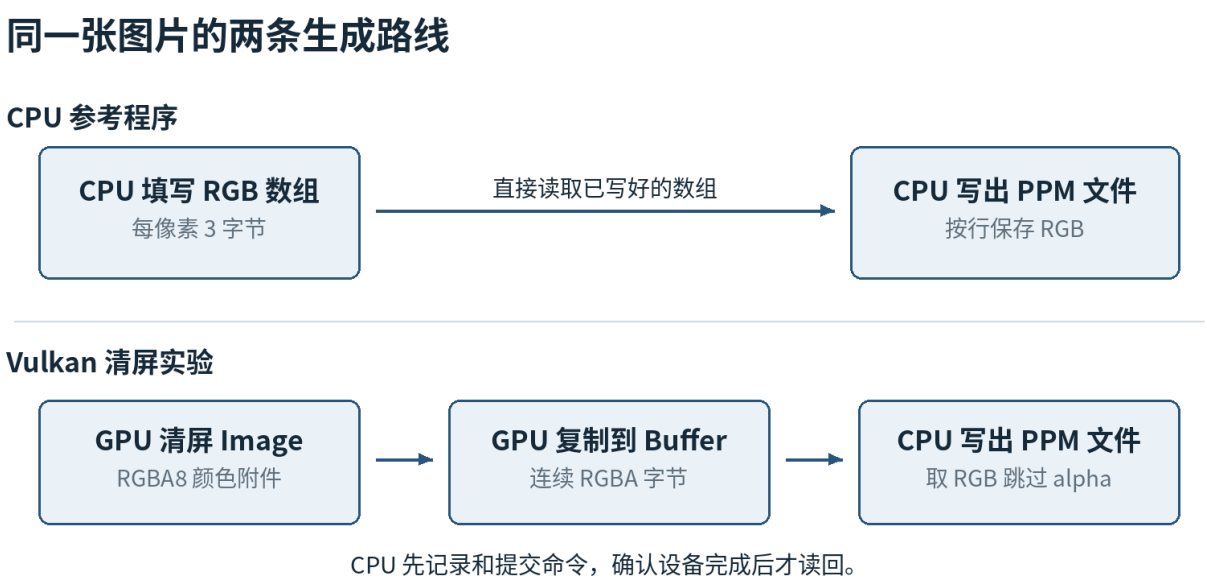


图 1 CPU 与 GPU 的图片生成路线

图 1 中的箭头表示本实验的数据与工作路线。GPU 图像的存储由实现安排；用于读回的 Buffer 则提供本程序需要的连续 RGBA 字节。CPU 读回 RGBA 数据后，再去掉 Alpha 分量，将 RGB 像素写入和本章相同格式的 PPM 文件。

从第 3 章起，你会同时看到 pixel（像素）和 texel（纹素）。Pixel 一般指输出图片或屏幕上的像素，texel 指纹理图像中的元素。最初清屏时，两者可以一一对应；到了第 9 章，一个输出像素可能由多个 texel 采样、过滤而成，这时就必须区分两个概念。

本章练习

练习 1：把循环中的 `pixels[offset + 0]` 改为 `x < width / 2 ? 255 : 0`，同时令另外两个分量都为 0。先画出你预计的结果，再运行。为什么条件里用 x，而不是数组偏移 offset？

解析：图片左半边应为红色，右半边为黑色。x 表示当前像素所在的列，直接与半个宽度比较即可。offset 表示从整个文件像素数据开头计算的字节位置，会随着行数增加而增大；它不是当前行内的横坐标。

练习 2：把宽度从 512 改成 256，高度不变。像素数据总共需要多少字节？为什么不需要重新推导数组下标公式？

解析：数据为 `256 * 512 * 3 = 393216` 字节。公式使用 width 而不是把 512 写死，因此每行的字节偏移会随新宽度自动调整。这个习惯以后也很有用：当图像尺寸变化时，创建、复制范围和文件头应使用同一个尺寸来源。

进入第 2 章前，请确认自己能够回答三个问题：一张图片怎样表示为数据？二维坐标怎样换算成数组偏移？为什么“CPU 录制命令”和“GPU 执行完成”不是同一个时刻？

第 2 章 构建工程并找到设备

本章先让两个程序顺利运行：CPU 参考程序负责生成图片，Vulkan 程序负责发现和检查设备。先排除环境问题，再阅读设备查询代码。这样，后面修改某一项功能后出现错误，才能更容易判断问题是否来自该项改动。

编译工具和运行环境各做什么

本工程使用 CMake 组织源文件并调用 C++ 编译器与链接器。代码中的 Vulkan 类型和函数声明来自开发头文件；运行时，调用通过 Vulkan Loader 分发到相应设备实现，由显卡驱动或软件实现执行。只有头文件还不够：运行 Vulkan 程序还需要正确安装并能够被 Loader 发现的实现。[3][4]

Shader 是在 GPU 上执行的程序。本工程先编写 GLSL 源码，再用 glslc 或 glslangValidator 编译为 SPIR-V，运行时交给 Vulkan。C++ 编译器不会顺便编译 GLSL。CMake 优先寻找 glslc，找不到时再尝试 glslangValidator；至少需要其中一个工具。[5]

Windows 和 Linux 可以按照 LunarG Vulkan SDK 的对应安装说明准备开发环境。还要确保当前使用的 GPU 驱动提供本书需要的 Vulkan 功能。在 Windows 上，可以安装包含 C++ 工具链的 Visual Studio；Linux 需要可用的 C++17 编译器和 CMake 3.21 或更高版本。SDK 安装方式、驱动包名称和路径会随系统变化，遇到安装问题时优先对照官方的当前说明。[3][4]

本书要求运行环境提供 Vulkan 1.3，并支持且启用 dynamicRendering 和 synchronization2。工程会查询这些条件，条件不满足时退出并说明原因。不要通过删掉检查来“继续运行”：后面确实会调用依赖这些能力的命令。macOS 的 Vulkan 通常经由移植实现提供，涉及不同的支持范围和初始化要求，本工程没有把它列为已验证平台。

第一次构建

解压实验包，进入其中的 lab 目录。下面命令都在这个目录运行。先不要修改源码，完成第一次构建。`-S .` 表示源码目录是当前目录，`-B build` 表示把生成的构建文件放入 build 子目录。

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --config Debug --parallel
```

本工程在 CMake 中统一了各配置的可执行文件输出位置。在 Linux 上，可执行文件为 build/vulkan_lessons 和 build/cpu_reference；Windows 使用 build/vulkan_lessons.exe 和 build/cpu_reference.exe，即使采用 Visual Studio 的 Debug 配置，也直接放在 build 目录。下面使用 Linux 写法；Windows 加上 .exe 并使用相应终端的路径写法即可，后面的参数相同。

```
build/cpu_reference out/cpu.ppm
build/vulkan_lessons --lesson probe
build/vulkan_lessons --lesson clear --output out/clear.ppm
```

第一条命令生成 CPU 图片；第二条查询当前可见的物理设备及其属性、队列族和功能支持，不创建逻辑设备，也不写图像。第三条才选择满足本书条件的图形设备、创建逻辑设备并产生清屏图片。这里允许你暂时把 clear 当作一次预览：第 3 至 5 章会逐步解释它的完整执行过程。

成功构建并运行这三个入口后，C++ 编译、Vulkan 加载和图片输出就有了可检查的基础。但 probe 即使没有发现设备，也可能正常退出，因此不能只看进程返回码；还应检查它输出的设备数量、API 版本和功能支持情况。如果这些条件都满足而 clear 仍失败，再按错误所在步骤逐项排查。

怎样检查程序输出

实验包中的 scripts/ppm_tools.py 用 Python 3 读取本工程产生的 P6 文件，能够转换图片并检查像素。各项子命令以 README 和脚本的 --help 输出为准；后续各章也会给出本工程对应的命令。先完成一次转换，再打开 PNG：

```
python3 scripts/ppm_tools.py out/clear.ppm --png out/clear.png --expect clear
```

Windows 若使用 Python Launcher，可把 python3 换成 py -3。脚本只使用 Python 标准库，不需要安装图像处理包。

希望在终端看到 Vulkan API 使用和同步问题时，请加 `--validation`：

```
build/vulkan_lessons --lesson clear --output out/clear.ppm --validation
```

这个选项要求存在 VK_LAYER_KHRONOS_validation，并开启同步验证。若验证层缺失，工程会报错，而不会在验证层缺失时静默退回未验证运行。你可以先不带此选项确认驱动可用，再按 SDK 说明准备验证层。本书后续的修改实验建议使用验证模式。[6]

验证层能发现大量 API 使用和同步错误，但“没有验证消息”并不证明画面符合设计。比如把顶点坐标写成远离可见区域的合法数值，往往不会违反 API 规则，却会得到空白图片。因此每次实验仍需要同时检查消息和输出。

按程序所需的顺序认识设备对象

打开 src/context.cpp，找到 create_context。第一次阅读时，先看它最终提供了哪些对象，不必立即钻进设备枚举和所有错误分支：后面的章节至少需要一个逻辑设备、一条支持图形工作的队列，以及可查询的物理设备信息。

`VkInstance` 是程序使用 Vulkan 的入口之一，通过它枚举系统暴露的物理设备。`VkPhysicalDevice` 是枚举得到的设备对象，用来查询属性、功能、队列族和内存类型。选定它之后，程序创建 `VkDevice`，声明本次实际要启用哪些功能、取得哪些队列。随后创建 Image、Buffer 等对象时，会使用这个逻辑设备。[7][8]

队列是提交设备工作的接口。队列族把具有兼容能力的队列分组；本工程选择带有 VK_QUEUE_GRAPHICS_BIT 的队列族，并请求其中一条队列。请求队列时，queueFamilyIndex 指定队列族编号，queueCount 指定从该队列族取得几条队列，两者不能互换。我们现在只用一条队列，这已经足够完成整条主线。[9]

把设备初始化理解为三个问题即可：系统有哪些候选设备？哪一个能够完成本书的任务？应用需要启用它的哪些功能和队列？设备句柄不是图片数据或图片内存的地址，而是后续创建资源、提交工作的基础。

查询功能与启用功能是两个动作

下面展示 create_context 中需要理解的结构关系。查询时，程序把 Vulkan 1.3 功能结构挂到 features2 的 pNext 上。调用完成后，字段里保存设备支持情况。创建逻辑设备时，再提供要求启用的功能结构。完整工程还会检查版本、队列、返回值和验证选项。

```
VkPhysicalDeviceVulkan13Features supported13{};
supported13.sType =
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_3_FEATURES;

VkPhysicalDeviceFeatures2 supported{};
supported.sType = VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2;
supported.pNext = &supported13;
vkGetPhysicalDeviceFeatures2(physicalDevice, &supported);

// 先检查 supported13 中的值，再准备启用列表。
VkPhysicalDeviceVulkan13Features enabled13{};
enabled13.sType =
    VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_VULKAN_1_3_FEATURES;
enabled13.dynamicRendering = VK_TRUE;
enabled13.synchronization2 = VK_TRUE;
// 创建逻辑设备时，让 VkDeviceCreateInfo::pNext 指向 enabled13。
```

查询阶段的 TRUE 表示设备支持该功能；创建逻辑设备时传入的 TRUE 表示应用要求启用它。虽然字段相同，信息的方向却不同。查询成功不会自动替你启用所有功能。工程为这两步使用不同的变量，便于看出哪个结构是设备填写的结果、哪个结构是程序提出的要求。[10][11]

为什么 Vulkan 的 CreateInfo 要这样填写？

用 `VkImageCreateInfo info{};` 创建 C++ 结构体时，花括号会先完成值初始化，使成员不会保留未初始化的随机值。随后必须把 `sType` 设为对应结构体类型。`Vulkan` 通过 `sType` 判断结构的身份，通过 `pNext` 读取附加的扩展参数；这不是 C++ 类继承，而是显式的结构体链。[2]

字段名以 p 开头通常表示指针，相应的 Count 字段常用于说明所指数组包含多少个元素。被指向的 C++ 数组至少应在相应 API 调用期间有效。不要用“所有指针都是异步保存”或“所有参数都立即复制”这样的总规则推断对象寿命；创建参数与设备以后使用的资源句柄有不同要求。第 5 章会用清屏流程区分这两类寿命。

第一次失败时怎样缩小范围

遇到故障时，先找最早出现、能够定位问题的错误。如果 CMake 找不到 Vulkan，检查 SDK 与开发文件路径；如果链接失败，确认缺失的库或符号；如果实例创建或设备枚举失败，检查 Loader 和驱动。如果只在启用 --validation 后失败，优先检查验证层是否正确安装。后续错误可能只是前一个失败引发的连锁反应。

环境修复之后，重新运行本章的三个入口，再继续改代码。先保留可运行的基线，后续才更容易判断错误是否由新改动引起。

本章练习

练习 1：你的头文件已经包含 Vulkan 1.3 的函数声明，但 probe 显示设备不满足本书条件。重新编译能不能让设备自动支持这些功能？

解析：不能。头文件解决 C++ 编译时对类型和函数的识别，设备查询反映当前运行实现提供的能力。应检查驱动、实际选用的设备和运行实现。仅提高编译时版本数字不会增加实现能力。

练习 2：把 enabled13.dynamicRendering 设为 FALSE，同时保留后面使用 Dynamic Rendering 的代码，会发生什么？先作判断，不必特意在未验证环境运行。

解析：程序不再启用后续命令所要求的功能。即使查询结果显示支持，仍不满足合法使用条件。合适的排查位置是逻辑设备创建时的功能链，而不是为 Image 增加 usage。两个字段描述的是不同层面的事情。

本章完成时，请保留一份成功的 probe 输出和 clear.ppm。第 3 章将从 clear 所需要的资源开始，逐步把这张图的来源讲清楚。

第 3 章 为图像准备资源和内存

现在开始解释 clear 实验。要把结果保存为文件，GPU 先要有一个写入颜色的目标，CPU 还要有一块能够读回像素的数据存储。打开 src/lessons.cpp 的 run_clear，再顺着它使用的渲染目标找到 src/render.cpp 和 src/resources.cpp。本章先学习资源如何创建，命令录制放到下一章。

先确定两种资源的用途

本实验的颜色图像采用 VK_FORMAT_R8G8B8A8_UNORM：R、G、B、A 是四个分量，每个分量占 8 位；UNORM 表示这些无符号整数在使用时按 0～1 的范围解释。清屏命令输入浮点颜色，图像则按这种格式保存结果。[12]

第 1 章的文件只有 RGB，这里多出来的 A 是 alpha 分量。实验始终把它清为 1，写 PPM 时跳过这个分量。不能因为最终文件只有 RGB，就把读回 Buffer 的像素容量按 3 字节计算：GPU 从 RGBA8 图像复制出来的每个 texel 仍然占 4 字节。512×512 个 texel 的紧密 RGBA 数据需要 1048576 字节。

图像有两个用途。第一，用作颜色附件，供渲染操作写入；第二，用作复制源，把结果复制到读回 Buffer。因此创建它时把 COLOR_ATTACHMENT 和 TRANSFER_SRC 两个 usage 位合在一起。读回 Buffer 作为复制目标被写入，所以它的 usage 包含 TRANSFER_DST。[13][14]

这里的“附件”可以先理解成一次渲染所使用的图像资源。颜色附件负责接收颜色输出，类似绘画时的画布。第 11 章还会引入深度附件，用来保存深度测试需要的数据。

对象	本实验保存什么	预先声明的用途	谁最后读取
颜色 Image	RGBA8 图像	COLOR_ATTACHMENT 与 TRANSFER_SRC	GPU 复制命令
读回 Buffer	按行紧密排列的 RGBA 字节	TRANSFER_DST	CPU

Usage 只声明资源允许进行哪些操作，不会自动触发这些操作。Image 允许作为复制源，不意味着创建后就会自动复制；Buffer 允许作为复制目标，也不意味着 CPU 已经收到结果。实际工作必须在命令缓冲区中明确录制。

逐项理解 Image 的创建参数

在 src/resources.cpp 的 create_image 中，你会看到下面这些赋值。这里把清屏实验传入的实际值展开，以便和字段逐个对应。完整函数还包含格式与能力检查、内存分配以及 View 创建。

```
VkImageCreateInfo info{};
info.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
info.imageType = VK_IMAGE_TYPE_2D;
info.format = VK_FORMAT_R8G8B8A8_UNORM;
info.extent = {512, 512, 1};
info.mipLevels = 1;
info.arrayLayers = 1;
info.samples = VK_SAMPLE_COUNT_1_BIT;
info.tiling = VK_IMAGE_TILING_OPTIMAL;
info.usage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT |
             VK_IMAGE_USAGE_TRANSFER_SRC_BIT;
info.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
info.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
```

这里创建的是二维图像，所以 extent.depth 为 1。mipLevels 和 arrayLayers 都表示数量，本章均为 1，即只有 Mip 0 和 Array Layer 0；它们的编号仍然从 0 开始。第 10 章会单独解释这两个维度，并提供访问不同子资源的实验。[13]

samples 为 1，表示本实验不使用多重采样。tiling 设为 OPTIMAL，使实现可以采用适合 GPU 访问的存储方式。但这不意味着图像内存一定按第 1 章的逐行数组排列。本书稍后通过复制命令，把图像内容写入布局明确的 Buffer，再由 CPU 读取。[13]

sharingMode 取 EXCLUSIVE，因为本书的主线始终只使用一个队列族。initialLayout 取 UNDEFINED，表示我们不需要保留图像原有内容。首次清屏会覆盖整个渲染区域，所以不依赖图像创建后原先的像素值；UNDEFINED 绝不等于“初始全黑”。

试着假设删去图像的 TRANSFER_SRC 用途标志，先不要运行，预测哪一步会失去合法性。图像创建未必因此失败，清屏用途也仍然存在；真正不再被允许的是后面的图像到 Buffer 复制。这个判断方法比把所有 usage 位都加上更有用：从实际操作反推资源必须预先声明哪些用途。

为什么创建 Image 后还需要绑定内存？

vkCreateImage 负责创建图像资源对象，但本书使用的普通、非稀疏 Image 还必须绑定满足要求的 VkDeviceMemory，才能用于后续操作。Image 定义尺寸、格式和用途，VkDeviceMemory 则表示一次内存分配；两者通过绑定建立关系。[15]

不要直接把 512×512×4 当作这张 OPTIMAL Image 的分配大小。程序首先调用 vkGetImageMemoryRequirements，取得 size、alignment、memoryTypeBits：分别表示所需分配范围、绑定偏移的对齐要求，以及允许使用的内存类型。实际需求取决于当前设备与资源，不能只凭像素数量推算。[16]

本工程为每个 Image 或 Buffer 单独分配一块内存，并在偏移 0 绑定。偏移 0 自然满足对齐要求；allocationSize 使用查询得到的 requirements.size。VkMemoryDedicatedAllocateInfo 在分配时说明这块内存专用于哪个资源。这种一对一分配便于初学时追踪资源的归属与范围；资源很多时可以再学习子分配，把多个资源放进更大的内存块。[17][18]

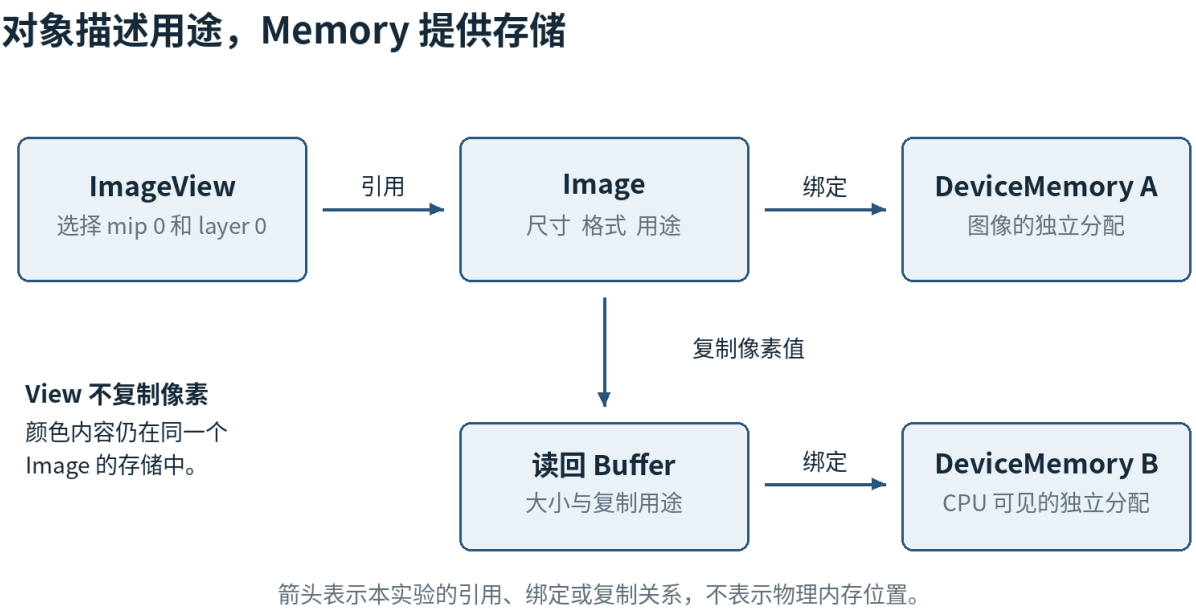


图 2 Image Memory 和 View 的关系

图 2 中，内存绑定让 Image 获得存储空间；ImageView 只是引用已有 Image 的某个范围，并不复制像素数据。读回 Buffer 有自己的内存，后续复制命令负责把图像结果写进去。

内存类型必须同时满足哪些条件？

设备会报告多种内存类型。memoryTypeBits 的第 i 位为 1，表示这个资源允许使用第 i 种类型。每种类型又有属性：HOST_VISIBLE 允许 CPU 映射访问，HOST_COHERENT 决定能否免去显式 Flush / Invalidate 等缓存维护操作，DEVICE_LOCAL 表示设备本地内存属性。这些标志可以同时出现；不能把 HOST_VISIBLE 和 DEVICE_LOCAL 想象成永远互斥的两类。[19]

对读回 Buffer，本工程必须选择 HOST_VISIBLE 内存，优先选择 HOST_COHERENT；对颜色 Image，则倾向 DEVICE_LOCAL。打开 choose_memory_type，可以看到它先检查某种内存类型是否在 memoryTypeBits 允许的范围内，再检查是否满足必需属性。如果没有符合偏好属性的类型，函数会退回其他满足硬性要求的类型。

下面给出完整的选择函数。allowed 来自资源的 memoryTypeBits，required 是不可缺少的属性，preferred 是有则优先的属性。逐项检查是为了同时遵守资源要求和访问需求。

```
static uint32_t choose_memory_type(Context& context, uint32_t allowed,
    VkMemoryPropertyFlags required,
    VkMemoryPropertyFlags preferred) {
    for (int pass = 0; pass < 2; ++pass) {
        for (uint32_t i = 0; i < context.memory_properties.memoryTypeCount; ++i) {
            const auto flags = context.memory_properties.memoryTypes[i].propertyFlags;
            if ((allowed & (1u << i)) && (flags & required) == required &&
                (pass == 1 || (flags & preferred) == preferred)) return i;
        }
    }
    throw std::runtime_error("No compatible memory type.");
}
```

表达式 `(flags & required) == required` 表示 required 中的每一位都存在。如果你只写 `flags & required`，当 required 包含多个属性时，只满足其中一项也会被误判为成功。位标志的检查方式会在很多 Vulkan 代码里重复出现。

创建 Buffer 也遵循同一流程

src/resources.cpp 的 create_buffer 与 Image 创建遵循相同的内存分配原则：先根据 size 和 usage 创建 Buffer，再查询内存要求、选类型、分配并绑定。Buffer 的逻辑大小与实际内存分配大小可能不同；即使底层分配得更大，也不能把额外的字节视为 Buffer 的合法访问范围。

读回 Buffer 使用 HOST_VISIBLE 类型，并不表示 CPU 已经可以读到正确像素。它只说明 CPU 可以获得映射地址。GPU 是否已经把数据写进来，还取决于复制命令有没有执行、同步是否完成；这些步骤在第 4 和第 5 章实现。

ImageView 负责选择图像的哪一部分

渲染时通常把 ImageView 交给附件描述。View 引用某个 Image，并描述从它看到的格式、维度和子资源范围。本实验的 View 使用相同的 RGBA8 格式、二维类型、COLOR aspect，范围为 mip 0 的一层。[20]

aspect 在这里表示图像的哪类组成部分。颜色图像使用 COLOR；深度格式会涉及 DEPTH，某些格式还包含 STENCIL。现在先记住 View 描述的是已有数据的使用视角。创建 View 并不会上传、清除或复制像素；那些都需要相应的实际操作。

请在 create_image 末尾找到 `view.subresourceRange`。其中 `{aspect, 0, mip_levels, 0, layers}` 依次填写 aspect、起始 mip、mip 数量、起始数组层、数组层数量。本章传入的数量都是 1，所以整个范围只有一个颜色子资源。第 10 章将用同一个结构选择更复杂的范围。

运行和定位本章相关问题

重新运行 `build/vulkan_lessons --lesson clear --output out/clear.ppm --validation`。本章不要求输出画面发生变化，而是要求你在源码中找出三个确切位置：图像的 usage 从哪里传入，设备内存大小从哪里取得，View 选择了哪个范围。

如果创建图像失败，先检查报错发生在格式支持查询还是 vkCreateImage，并查看尺寸、格式和 usage 的组合。如果错误发生在分配或绑定，查看 memoryTypeBits、选中的类型以及分配大小。若复制时报“资源用途不允许”，回到创建处检查 usage。错误可能在复制时暴露，真正需要修改的却是资源创建参数，这种情况后面会经常遇到。

本章练习

练习 1：给读回 Buffer 选择了不带 HOST_VISIBLE 的内存，然后想在 CPU 上调用 vkMapMemory。为了修复它，应该给 Buffer 增加 usage，还是重新选择内存类型？

解析：需要重新选择满足 HOST_VISIBLE 的兼容内存类型，并重新创建或配置相应资源。usage 声明 Buffer 怎样被 Vulkan 命令使用，不会改变已绑定内存能否被 CPU 映射。类型还必须被该资源的 memoryTypeBits 允许。[19][21]

练习 2：假设 Image 有 4 个 mip 级别，并另建一个仅覆盖 mip 0、levelCount 为 1 的颜色附件 View，再只清屏 mip 0。mip 1 至 3 是否自动得到缩小后的清屏结果？

解析：不会。数量决定图像包含哪些子资源，没有生成内容的含义。除非另有命令写入那些级别，否则它们没有产生本实验所要求的有效内容。本工程 create_image 的默认 View 覆盖全部级别，不能只改 mipLevels 后就把这个多级 View 直接当颜色附件使用；颜色附件 View 需要符合单个 mip 级别等要求。第 10 章会区分采样 View 与附件 View 的范围。[22]

练习 3：为什么第 1 章只需 786432 字节 RGB 数据，第 3 章的读回 Buffer 却要 1048576 字节？

解析：文件使用每像素 3 字节的 RGB；GPU 图像使用每 texel 4 字节的 RGBA8，复制到 Buffer 时仍是 RGBA8。最后写文件时逐像素取前三个分量，所以两种大小都有明确位置。不能在复制之前就按最终文件大小缩小 Buffer。

开始第 4 章之前，你应当能从“GPU 要清屏并把结果给 CPU”这个需求，独立写出颜色图像与读回 Buffer 的用途，并解释为什么创建对象、分配内存、创建 View 是几个不同的动作。

第 4 章 记录清屏和复制命令

资源已经创建，但仅有资源并不会自动产生清屏结果。本章的任务是将 GPU 要做的工作明确录制下来：准备颜色附件、清除颜色、将图像复制到读回 Buffer，并为 CPU 的后续读取建立正确的数据依赖。至于提交命令和读取结果，留到第 5 章完成。

请打开 src/render.cpp，定位 begin_rendering 和 end_rendering_and_copy。它们承担了清屏实验的主要命令录制工作。两者之间以后会放入三角形绘制命令；当前 clear 模式只使用附件的清屏操作。

命令缓冲区负责记录将来要执行的工作

`VkCommandBuffer` 用来保存已经录制的 GPU 命令。CPU 调用 `vkCmd...` 函数时，通常只是将命令写进缓冲区；随后把命令缓冲提交到队列，GPU 才会按执行顺序与同步规则处理它们。本书只使用可以直接提交的主命令缓冲区。[23]

命令缓冲从 CommandPool 分配。创建池时需要指定队列族，因为从该池分配的命令缓冲区只能提交给对应队列族中的队列。本工程让它与第 2 章选中的 graphics_family 一致。池还启用了 RESET_COMMAND_BUFFER_BIT，让程序可以单独重置其中的命令缓冲。[24]

状态	在本工程中怎样进入	此时可以做什么
Initial	分配后或合法重置后	开始录制
Recording	vkBeginCommandBuffer 成功	记录允许的命令
Executable	vkEndCommandBuffer 成功	提交到兼容队列
Pending	提交的工作尚未完成	等待完成，保持被使用资源有效
Invalid	ONE_TIME_SUBMIT 的一次执行结束后	重置，再开始新的录制

这张表描述本工程使用的单次提交路径。命令缓冲还有其他合法使用方式，但现在不需要全部展开。最关键的是 Pending：这时设备仍可能使用命令缓冲以及命令引用的对象，CPU 不能通过重置把它“提前收回来”。本书的 begin_commands 要求调用者先确认上一轮已经完成；如果资源仍在使用时就重置命令缓冲区，事后再等待也不能使先前的操作合法。[25]

首次使用 Image 前，先准备布局

第 3 章把图像创建时的 initialLayout 设为 UNDEFINED。现在我们要把它用作颜色附件，所以在第一次开始渲染之前，记录一次转换到 COLOR_ATTACHMENT_OPTIMAL 的 Image barrier。

Layout 规定 Image 子资源在某种用途下应处于怎样的布局。只修改 C++ 中保存布局的变量，并不会改变 GPU 资源的实际状态。本工程使用 vkCmdPipelineBarrier2 和 VkImageMemoryBarrier2 录制布局转换，并同时描述转换前后的读写依赖。[26]

首次使用这张新图像时，前面没有需要保留的内容，也没有需要同步的先前访问，所以源阶段和源访问均设为 NONE。接下来颜色附件要写入图像，因此目标阶段选 COLOR_ATTACHMENT_OUTPUT，目标访问选 COLOR_ATTACHMENT_WRITE。现在先把阶段理解为 GPU 工作的类别；第 12 章再完整分析同步关系。

在源码中，transition_image 接收这些明确参数，然后填写 barrier 并记录命令。这个辅助函数不会推断 Image 的历史访问，也不会自动补足你没有提供的同步条件。以下为函数的完整实现，读它时先找 oldLayout、newLayout、前后两组 stage/access，以及 subresourceRange。

```
void transition_image(VkCommandBuffer command, VkImage image,
    VkImageLayout old_layout, VkImageLayout new_layout,
    VkPipelineStageFlags2 source_stage, VkAccessFlags2 source_access,
    VkPipelineStageFlags2 destination_stage,
    VkAccessFlags2 destination_access,
    VkImageAspectFlags aspect, uint32_t base_mip, uint32_t mip_count,
    uint32_t base_layer, uint32_t layer_count) {
    VkImageMemoryBarrier2 barrier{VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2};
    barrier.srcStageMask = source_stage;
    barrier.srcAccessMask = source_access;
    barrier.dstStageMask = destination_stage;
    barrier.dstAccessMask = destination_access;
    barrier.oldLayout = old_layout;
    barrier.newLayout = new_layout;
    barrier.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
    barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
    barrier.image = image;
    barrier.subresourceRange = {
        aspect, base_mip, mip_count, base_layer, layer_count};
    VkDependencyInfo dependency{VK_STRUCTURE_TYPE_DEPENDENCY_INFO};
    dependency.imageMemoryBarrierCount = 1;
    dependency.pImageMemoryBarriers = &barrier;
    vkCmdPipelineBarrier2(command, &dependency);
}
```

两个 queue family 字段填 IGNORED，表示本次不处理队列族所有权转移。subresourceRange 指定这次转换针对哪些子资源。清屏图像只有 COLOR aspect、mip 0、layer 0，所以默认范围就是我们创建的整张图。第 10 章扩大子资源数量时，这个范围也必须明确选择。

用一次渲染操作完成清屏

接下来填写颜色附件：使用第 3 章创建的 View，指定 Image 当前应处于 COLOR_ATTACHMENT_OPTIMAL，把 loadOp 设为 CLEAR，并给出要清成的颜色。下面是 begin_rendering 中颜色附件的核心内容；变量 target 保存已创建的资源。

```
VkRenderingAttachmentInfo color{};
color.sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO;
color.imageView = target.color.view;
color.imageLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
color.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR;
color.storeOp = VK_ATTACHMENT_STORE_OP_STORE;
color.clearValue.color.float32[0] = 6.0f / 255.0f;
color.clearValue.color.float32[1] = 9.0f / 255.0f;
color.clearValue.color.float32[2] = 14.0f / 255.0f;
color.clearValue.color.float32[3] = 1.0f;
```

loadOp 决定开始渲染时怎样处理附件内容：CLEAR 清成指定颜色，LOAD 使用原有内容，因此必须保证原有内容有效。storeOp 则决定渲染结束后是否保留结果。因为后面还要复制这张图，本实验选择 STORE。[22]

`color.imageLayout` 只是在附件描述中声明图像应处于什么布局，并不会自动执行转换。必须先完成相应的布局准备，才能按照这个声明使用附件。

下面将两个函数中的关键调用放在一起，展示开始与结束渲染的对应关系。实际源码由 begin_rendering 调用 Begin，由 end_rendering_and_copy 调用 End；不要在 begin_rendering 内再添加一次 End。VkRenderingInfo 把附件和渲染区域组合起来。本实验从 `(0, 0)` 开始覆盖完整 512×512 范围，layerCount 为 1，只有一个颜色附件。clear 模式不需要深度附件。

```
VkRenderingInfo info{};
info.sType = VK_STRUCTURE_TYPE_RENDERING_INFO;
info.renderArea = {{0, 0}, {512, 512}};
info.layerCount = 1;
info.colorAttachmentCount = 1;
info.pColorAttachments = &color;
vkCmdBeginRendering(command, &info);
// clear 模式在这个范围内不记录 draw。
vkCmdEndRendering(command);
```

开始与结束渲染之间即使没有 Draw，附件的 loadOp=CLEAR 仍然可以完成清屏。因此，本章不必创建图形 Pipeline 或 Shader。第 6 章会在同一渲染范围内加入 Draw，让三角形覆盖区域的颜色由 Shader 计算。[22][27]

注意共享的 begin_rendering 还设置了 viewport 和 scissor，供后续绘制实验使用。这里的附件 loadOp 清屏范围来自 renderArea；不要以为修改 Scissor 就能改变 loadOp=CLEAR 的清除范围。

渲染结束后，将颜色结果交给复制命令

清屏先向 Image 写入颜色，后面的复制命令再从同一 Image 读取。为了使复制命令能够获得正确的绘制结果，需要建立从颜色附件写入到复制读取的内存依赖，同时将 Image 从 COLOR_ATTACHMENT_OPTIMAL 转换为 TRANSFER_SRC_OPTIMAL。[26]

因此，在 end_rendering_and_copy 中，Image Barrier 的源阶段和源访问分别是 COLOR_ATTACHMENT_OUTPUT、COLOR_ATTACHMENT_WRITE；目标阶段和目标访问则是 COPY、TRANSFER_READ。注意，这里的目标访问是读取 Image。复制命令还会写入另一个对象——读回 Buffer，那部分需要单独处理。

目前不必记下所有 Stage 标志。先学会从实际操作推导：谁刚刚写了这个资源？谁接下来要读它？Barrier 参数由这两次访问确定，而不是根据函数名称猜测。

复制范围也需要明确

复制操作用 VkBufferImageCopy 指定 Buffer 与 Image 之间的数据对应范围。下面把本工程这次复制的范围完整展开。bufferRowLength 和 bufferImageHeight 为 0 表示按 imageExtent 指定的宽、高计算紧密排列的行、层步长；imageSubresource 指定颜色 aspect 的 mip 0，从 layer 0 开始复制 1 层。[28]

```
VkBufferImageCopy copy{};
copy.bufferOffset = 0;
copy.bufferRowLength = 0;
copy.bufferImageHeight = 0;
copy.imageSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
copy.imageSubresource.mipLevel = 0;
copy.imageSubresource.baseArrayLayer = 0;
copy.imageSubresource.layerCount = 1;
copy.imageOffset = {0, 0, 0};
copy.imageExtent = {512, 512, 1};

vkCmdCopyImageToBuffer(
    command, target.color.image,
    VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
    target.readback.buffer, 1, &copy);
```

复制完成后，读回 Buffer 中会有 512×512×4 字节连续排列的 RGBA8 数据。但复制命令不会写磁盘文件，也不会把 RGBA 转成 RGB；这些工作仍由 CPU 完成。复制还要求 Image 具有 TRANSFER_SRC 用途、Buffer 具有 TRANSFER_DST 用途，并且读写范围都在资源的合法大小之内。[29]

为 CPU 读取准备最后一段数据依赖

复制命令写入读回 Buffer，接下来 CPU 要读取它。因此，我们还要录制一个 Buffer Barrier：源阶段为 COPY、源访问为 TRANSFER_WRITE，目标阶段为 HOST、目标访问为 HOST_READ，范围覆盖此次读回的数据。这是 Buffer 的内存依赖，不涉及 Image Layout 转换。[30]

录制完 Buffer Barrier 后，CPU 仍然不能马上读取结果：这些命令尚未提交，GPU 也未必执行。第 5 章将完成提交和等待，并说明非一致性内存的缓存维护。

亲手改变清屏颜色

现在回到 begin_rendering，将 clearValue 的 RGBA 四个分量设为 (0.0f, 1.0f, 0.0f, 1.0f)，即红 0、绿 1、蓝 0、alpha 1。重新构建并运行 clear，把结果保存为 out/clear_green.ppm。得到的图片应当是均匀绿色。

如果修改后仍是旧颜色，先确认构建成功、运行的是本次构建的可执行文件、打开的是新的输出文件。确认程序和文件无误后，再排查 Vulkan API 的错误。把这些普通程序问题与图形问题分开，能避免不必要地改同步参数。

完成观察后恢复原清屏颜色，继续后续章节。其他实验会把这组深蓝色作为背景，验证脚本也会用它识别绘制区域。

本章练习

练习 1：你想保留图像里原有的内容，于是把 loadOp 改为 LOAD，但第一次使用时仍从 UNDEFINED 开始。现在是否可以期望原有内容被保留？

解析：不可以。你已经通过 UNDEFINED 表明不保留原内容，而新图像也没有被本程序写入预期内容。要保留上一轮结果，需要确实存在有效旧内容，使用正确的旧布局和同步关系，再选择 LOAD。只修改 loadOp，不能凭空得到需要保留的旧内容。

练习 2：图像到 Buffer 复制为什么需要先给 Image 建立“写到读”的交接，之后又给 Buffer 建立“写到读”的交接？

解析：这是两个资源上的两段关系。图像先被清屏写入，再被复制命令读取；Buffer 先被复制命令写入，再被 CPU 读取。复制命令同时扮演前一段的读取者和后一段的写入者。把这两段关系分别画出来，就能决定各自 barrier 的资源、范围和访问位。

练习 3：把 vkCmdCopyImageToBuffer 放在 vkCmdBeginRendering 和 vkCmdEndRendering 之间，能否因为“顺序看起来更紧凑”而正常使用？

解析：不能这样移动。本命令的合法使用要求它在渲染范围之外执行。工程先结束渲染，再进行图像交接和复制。这说明录制代码还需要遵守命令允许出现的范围，不能仅靠资源已经创建来判断。[29]

第 5 章 提交工作并取回图片

现在已经准备好资源，也录制好了清屏、复制与同步命令。下一步要解决“时间”问题：由谁提交这些工作？CPU 何时能读取结果？设备结束使用后，资源又何时能释放？本章将补齐一次 clear 实验的最后几步。

提交和完成有不同含义

打开 src/context.cpp 的 submit_and_wait。函数先结束录制，将命令缓冲区加入提交描述，再调用 vkQueueSubmit2。提交成功只说明工作已进入队列，并不意味着 GPU 已经完成。CPU 可以继续做其他事，也可以像本实验这样等待结果。[31]

为了判断提交何时完成，本工程使用 Fence。Fence 有 signaled（已触发）和 unsignaled（未触发）两种状态。提交关联一个未触发的 Fence；当对应工作完成时，设备将其设为已触发，CPU 便可通过 vkWaitForFences 等待。公共 Fence 初次创建时设为已触发，在真正提交前再重置。[32]

下面是 submit_and_wait 的完整实现。函数名已经说明它会等待这次工作完成，因此主线前面的单次实验返回后可以进行读回。第 13 章会调整等待位置，允许两组资源交替使用。

```
void submit_and_wait(Context& context, VkCommandBuffer command) {
    check(vkEndCommandBuffer(command), "vkEndCommandBuffer");
    check(vkWaitForFences(context.device, 1, &context.fence,
                          VK_TRUE, UINT64_MAX),
          "wait previous submit fence");
    VkCommandBufferSubmitInfo command_info{
        VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO};
    command_info.commandBuffer = command;
    VkSubmitInfo2 submit{VK_STRUCTURE_TYPE_SUBMIT_INFO_2};
    submit.commandBufferInfoCount = 1;
    submit.pCommandBufferInfos = &command_info;
    // 录制成功后、确定会提交时，才把 signaled fence 变为 unsignaled。
    check(vkResetFences(context.device, 1, &context.fence), "vkResetFences");
    check(vkQueueSubmit2(context.graphics_queue, 1, &submit, context.fence),
          "vkQueueSubmit2");
    check(vkWaitForFences(context.device, 1, &context.fence,
                          VK_TRUE, UINT64_MAX),
          "wait submitted work");
}
```

先关注最后的两个调用：vkQueueSubmit2 提交命令，vkWaitForFences 等待对应提交完成。前面的 VkCommandBufferSubmitInfo 和 VkSubmitInfo2 负责描述这次要提交的命令缓冲区。本实验没有跨队列的 Semaphore 等待或通知，因此相关数量设为 0。

为什么要在录制成功后才重置 Fence？因为 Reset 会把它改成未触发状态，只有后续提交完成才可能再次触发。如果先 Reset，随后又因为错误或其他分支跳过提交，那么下一次等待就可能永远没有对应的完成信号。第 14 章的窗口重建流程会再次用到这个判断。

Barrier 与 Fence 分别解决什么问题?

第 4 章已经为“GPU 复制写入 Buffer → CPU 读取 Buffer”录制了 Barrier。既然如此，为什么还需要等待 Fence？因为两者解决的问题不同：Barrier 建立前后操作的执行与内存依赖；Fence 则让 CPU 判断某次提交是否完成。数据是否可以被正确访问，与 CPU 是否已等到这次工作完成，必须分别处理。[30][32]

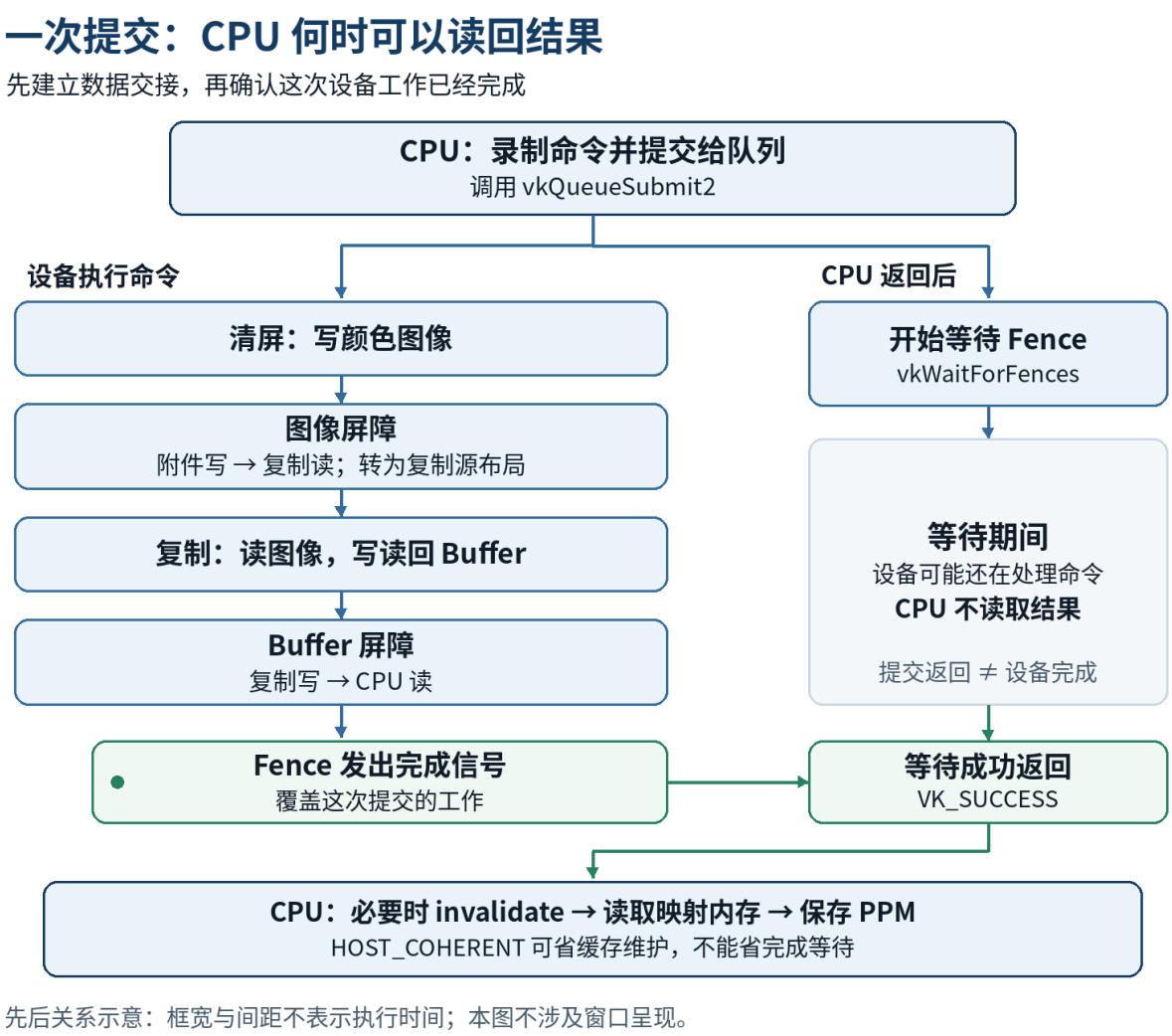


图 3 单次提交和 CPU 读回的时间关系

图 3 只示意执行先后，图中条块的长度不是性能数据。提交函数返回后，CPU 仍可继续运行；若随后调用 Fence 等待，就会在对应提交完成前停留在等待处。本程序只有等到该调用成功返回，才读取映射内存。不能凭估计认为 GPU 应该已经画完。

映射内存，让 CPU 获得访问地址

src/render.cpp 的 readback_ppm 在调用者确认 Fence 完成之后执行。vkMapMemory 给 CPU 一个访问指定设备内存分配的映射地址。本工程从分配偏移 0 映射 VK_WHOLE_SIZE，拿到的地址可以按字节读取。[21]

vkMapMemory 的职责是建立 CPU 可以访问的内存映射，不负责执行图像到 Buffer 的复制，也不会暗中等待 GPU 完成。复制由先前录制的命令完成，完成时刻由 Fence 等待确认。因此，“取得一个有效指针”与“指针中的新数据已经可读”必须分开判断。

非一致性内存还需要显式缓存维护

如果读回内存不具备 HOST_COHERENT 属性，那么在 CPU 读取前，还应调用 vkInvalidateMappedMemoryRanges，处理非一致性内存的缓存可见性。正确顺序是：先等待设备写入完成，再 Invalidate，最后读取。Invalidate 本身不会等待 GPU。[33]

HOST_COHERENT 表示不需要这一显式 flush 或 invalidate 步骤。它没有承诺 GPU 比 CPU 更早完成，也不允许 CPU 抢先读一份正在被写的数据。第 8 章和第 13 章中 CPU 改写 Uniform Buffer 时，同样要把缓存维护与资源是否仍在被使用分开判断。[19]

VkMappedMemoryRange 还有对齐要求，涉及 nonCoherentAtomSize。本工程的分配、绑定和映射都采用整块独立内存、偏移 0，维护范围也填 offset 0 和 VK_WHOLE_SIZE，从而避开局部范围的边界计算。将来改为大块内存中的子分配时，不能只照抄“按实际写入字节数填 size”，必须同时检查映射范围和 atom 对齐。[34]

下面列出读回操作的关键顺序。完整的输出和错误处理在 readback_ppm 内。

```
// 调用此处之前，本次提交的 Fence 已经等待成功。
void* mapped = nullptr;
check(vkMapMemory(readback.device, readback.memory,
                  0, VK_WHOLE_SIZE, 0, &mapped), "map readback");

if (!(readback.properties & VK_MEMORY_PROPERTY_HOST_COHERENT_BIT)) {
    VkMappedMemoryRange range{};
    range.sType = VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE;
    range.memory = readback.memory;
    range.offset = 0;
    range.size = VK_WHOLE_SIZE;
    check(vkInvalidateMappedMemoryRanges(
        readback.device, 1, &range), "invalidate readback");
}
// 随后读取 mapped，复制到 CPU 自己持有的 RGB 数组。
// 实际工程在错误路径也会解除映射。
```

将 RGBA 数据转换为第 1 章的 RGB 文件

读回 Buffer 中每个 RGBA8 像素占 4 字节，而输出文件只保存 3 字节 RGB。因此，第 i 个源像素从 rgba + i*4 开始，目标像素写入 rgb + i*3。复制前三个分量即可跳过 Alpha。完成 CPU 侧数组后，可以解除内存映射，再写入 PPM 文件。

这一步把第 1 章和 GPU 实验连接起来了。文件头和 RGB 布局仍然相同，区别在于像素值的来源：第 1 章由 CPU 循环直接写入；这里来自设备上的清屏和复制。两个文件都应当是 512×512、每个像素 (6， 9， 14) 的纯色图。

```
python3 scripts/ppm_tools.py out/cpu.ppm --expect clear
python3 scripts/ppm_tools.py out/clear.ppm --expect clear
```

两次输出都应包含 changed_pixels 为 0、unique_colors 为 1；center_rgb 和 corner_rgb 都应为 [6, 9, 14]。

请重新生成 cpu.ppm 和 clear.ppm，然后用验证脚本比较像素值。这样的检查不依赖图片查看器如何显示深蓝色；它能区分“屏幕看起来不一样”和“文件中的实际像素值不同”。

沿源码完成一次从头到尾的阅读

现在回到 src/lessons.cpp 的 run_clear。此前各章已经解释过它调用的辅助函数：create_render_target 创建 Image、View 和读回 Buffer；begin_commands 开始新一轮录制；begin_rendering 准备附件并记录清屏；end_rendering_and_copy 结束渲染并安排两段数据交接；submit_and_wait 送出并等待；readback_ppm 读取结果并写出文件。

```
void run_clear(Context& context, const Options& options) {
    // 第 5 章：没有 shader、pipeline 或 draw；附件 loadOp 完成清屏。
    RenderTarget target = create_render_target(context);
    CompletionGuard completion{context};
    begin_commands(context, context.command);
    begin_rendering(context, context.command, target);
    end_rendering_and_copy(context.command, target);
    submit_and_wait(context, context.command);
    readback_ppm(target, options.output);
}
```

不要只记住封装函数的名称。请逐个打开定义，对照前几章解释每一步的具体作用。封装只是把操作整理进函数，并没有让格式、同步范围或复制区域变得不重要。以后修改它们时，仍要找到实际设置这些参数的位置。

资源在什么时候才能释放?

提交后，命令缓冲区引用的 Image、ImageView、Buffer 及其内存在设备仍可能使用时必须继续有效。C++ 局部变量离开作用域，不会自动终止 GPU 正在进行的访问。前面讲过的 CommandBuffer Pending 状态，就是检查这一生命周期约束的重要线索。[2][25]

配套工程用带资源所有权的 C++ 封装管理对象。Image 的析构按 View、Image、Memory 的顺序释放；Buffer 的析构释放 Buffer 和 Memory。这些类型禁止复制，只允许移动，避免两个拥有者重复释放同一对象。它们本身不替代同步：调用者仍需让设备先完成，再触发释放。

实验还使用 CompletionGuard 在整次实验结束或异常退出时等待设备。它在本轮资源创建之后声明，利用 C++ 局部对象逆序析构，让这次等待先于资源释放发生。你可以在 src/context.hpp 查看它的短定义。这个 guard 只用于实验收尾，不能放在每帧循环里，否则会把原本可能并行的工作反复等空。

创建时使用的普通 C++ 参数结构体，与 GPU 后续引用的资源对象，生命周期并不相同。例如 vkCreateImage 返回后，局部 VkImageCreateInfo 可以结束生命周期；创建出来的 Image 却要保持有效，直到所有相关 GPU 操作完成。理解这个区别，才能知道哪些数据适合放在局部变量中，哪些资源必须由更长寿命的对象管理。

本章练习

练习 1：GPU 提交后你立刻调用 vkMapMemory，并且返回了一个非空指针。这是否能证明图片已经复制完成？

解析：不能。得到地址只证明建立了符合条件的映射。程序仍需要正确记录复制与数据依赖，并等待提交完成。如果内存非 coherent，还要在读取前 invalidate。成功映射不是“设备已完成”的通知。[21][33]

练习 2：读回 Buffer 的内存恰好带 HOST_COHERENT，于是你删除了 Fence 等待。为什么这个修改仍然不成立？

解析：coherent 免去的是显式缓存维护。CPU 仍可能读到设备写入之前的内容，或与尚未结束的写入并发。完成时刻要由同步条件证明，不能由内存属性推断。

练习 3：clear 能运行并产生文件，但颜色每隔几个字节就错乱。程序使用 RGBA8 读回，却按 `rgba + i * 3` 取源像素。为什么看同步代码之前，应先修正这个下标？

解析：读回 Buffer 是每像素 4 字节的 RGBA 数据。步长写成 3，会把前一个像素的 alpha 当成下一个像素的颜色，导致所有后续分量错位。这是一个确定的数据解释错误；即使同步完全正确，它也会发生。

走完本章后，请尝试在一张纸上写出自己的 clear 流程，并为每一步标出“CPU 此时做什么、GPU 以后做什么、操作哪个对象”。下一章会保留这些基础，只在渲染范围中增加三角形绘制。

第 6 章 三角形从顶点到像素

前面我们已经让 GPU 把整张颜色附件清成深蓝色，并把结果读回 CPU。现在要在图像中央画一个三角形。仅告诉 GPU 三个顶点的位置，还不足以确定每个像素的颜色：它还需要把顶点组装成图元、裁剪和映射到画面、判断覆盖范围，再计算片元颜色。图形流水线就是将这些工作分成不同阶段。

在配套工程根目录执行下面的命令。前面的构建步骤已经生成 `build/vulkan_lessons`；后面每次修改 C++ 或 Shader 后，都要先重新构建，再运行程序。

```
cmake --build build --config Debug --parallel
./build/vulkan_lessons --lesson triangle --output out/triangle.ppm --validation
```

预期得到一张 512×512 图片：深色背景上有一个三角形，上角偏红、右下角偏绿、左下角偏蓝，内部颜色连续过渡。打开 `src/lessons.cpp` 中的 `run_triangle`，并对照 `shaders/triangle.vert`、`shaders/color.frag`。本章先让坐标写在 Shader 里，因此无需顶点 Buffer。



清屏　　加入三个顶点

本次软件 Vulkan 实测输出

图 4 清屏与三角形的实际输出

每个顶点先回答：“我在哪里？”

Shader 是运行在 GPU 上的程序。本书用 GLSL 编写，再编译成 Vulkan 使用的 SPIR-V。GLSL 中的 vec2、vec3、vec4 分别表示具有 2、3、4 个浮点分量的向量。顶点着色器为每个被处理的顶点计算位置，并可向后续阶段传递颜色等属性；它不会自行遍历画面中的全部像素。[35]

下面是三角形实验的核心写法。`gl_VertexIndex` 是非索引绘制时的顶点编号；调用 `vkCmdDraw(cmd, 3, 1, 0, 0)` 时，本例需要的编号为 0、1、2。`gl_Position` 是输出位置的内建变量，`vertexColor` 则是我们自己定义的输出。

```
#version 450
layout(location = 0) out vec3 vertexColor;

const vec2 positions[3] = vec2[](
    vec2( 0.00, -0.75),
    vec2( 0.75,  0.65),
    vec2(-0.75,  0.65)
);
const vec3 colors[3] = vec3[](
    vec3(1.0, 0.0, 0.0),
    vec3(0.0, 1.0, 0.0),
    vec3(0.0, 0.0, 1.0)
);

void main() {
    gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);
    vertexColor = colors[gl_VertexIndex];
}
```

本例选择三角形列表拓扑 VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST，所以每三个顶点组成一个三角形。vkCmdDraw 中的 3 表示三个顶点，1 表示绘制一个实例，后面的两个 0 分别是第一个顶点、实例的起始编号。这里没有索引表，读取顺序就是 0、1、2。换拓扑会改变顶点怎样组装成图元，不会改变顶点自身的坐标意义。[36]

四个裁剪坐标怎样映射到二维图片？

gl_Position 的四个数是裁剪坐标 (x_c, y_c, z_c, w_c)。名称里的“裁剪”意味着：在进入图片坐标系以前，流水线先判断几何体的哪一部分位于允许观察的范围内。本书使用默认深度裁剪、不开启深度钳制；其范围满足 −w_c ≤ x_c ≤ w_c、−w_c ≤ y_c ≤ w_c、0 ≤ z_c ≤ w_c。一个角越界时，可能只裁掉三角形的一部分；不能把裁剪理解成“只要一个顶点越界就丢掉整面”。[37]

顶点着色器输出的 gl_Position 仍是四维裁剪坐标。随后 GPU 执行透视除法：分别用 x、y、z 除以 w，得到归一化设备坐标（NDC）。本章的 w 都是 1，因此前三个数值不变。保留 w 是为了后面实现透视效果；不要在 Shader 中先手动除以 w，却又把原来的 w 写回 gl_Position，否则流水线会再次进行透视除法。[37]

NDC 是与具体图片尺寸无关的相对坐标。Viewport（视口）负责把 NDC 映射到实际图片尺寸。对本实验的 (x,y)=(0,0)、宽高都是 512 的正高度视口，有 x_fb=(x_ndc+1)×256、y_fb=(y_ndc+1)×256。因此红色顶点 (0,−0.75) 对应图片中的 (256,64)，负 Y 朝向图片上方；正 Y 朝下。图片中央是 (256,256)。这里的坐标是连续的几何坐标，像素覆盖还要由光栅化决定。[37]

Viewport（视口）决定 NDC 怎样映射到帧缓冲坐标；Scissor（裁剪矩形）则限制允许写入的区域。视口宽度减半，会把图形横向映射到较窄区域；只将 Scissor 宽度减半，并不会压缩图形，而是裁掉矩形之外的部分。两种结果看起来都可能“少了一半”，原理却不同。

三角形内部的颜色从哪里来？

经过裁剪与视口变换后，光栅化阶段根据三角形覆盖情况生成片元。本书使用单采样，可以暂时把片元理解成“某个输出位置的候选颜色结果”。但片元不等于最终像素：深度测试可能拒绝它，同一个像素位置也可能先后处理多个三角形产生的片元。[38]

片元着色器收到的颜色来自顶点输出的插值。可以先想象三角形内任意一点：可以用三个非负权重 a、b、c 表示该点，且 a+b+c=1。在本章所有 w 都等于 1 时，颜色就是 a×红+b×绿+c×蓝。靠近红角，红色权重大；三者权重相等时，三个颜色分量相等。流水线计算这些权重，不需要在片元 Shader 中自己判断它属于三角形哪一部分。

到了透视投影，屏幕上的等距离不再对应物体表面的等距离。默认的平滑浮点插值会利用顶点 w 修正：对某个属性 f，结果由下面的透视校正插值公式得到。现阶段不用记忆公式，只需知道颜色和 UV 的中间值由插值规则得到；未来倾斜平面上的贴图能够保持合理形状，正依赖这一步。[38]

$$f = \frac{a\frac{f_0}{w_0} + b\frac{f_1}{w_1} + c\frac{f_2}{w_2}}{\frac{a}{w_0} + \frac{b}{w_1} + \frac{c}{w_2}}$$

片元 Shader 本身可以很短：

```
#version 450
layout(location = 0) in vec3 vertexColor;
layout(location = 0) out vec4 outColor;

void main() {
    outColor = vec4(vertexColor, 1.0);
}
```

顶点输出的 `location=0` 与片元输入的 `location=0` 连接颜色，变量名不必相同，但接口位置和类型要匹配。片元输出的另一个 `location=0` 表示颜色附件的输出位置 0；这是片元输出接口的编号，不是顶点输入属性的延续。此处 alpha 写 1，颜色混合关闭，因此通过测试的片元把颜色写入附件。[35]

用图形 Pipeline 组织这些绘制规则

`src/render.cpp` 中的 `create_graphics_pipeline` 把顶点与片元 Shader、三角形拓扑、填充方式、关闭剔除、单采样、颜色写入规则组合为图形 Pipeline。本实验还通过动态渲染的 Pipeline 创建信息声明颜色附件格式。附件的实际 ImageView 在 `vkCmdBeginRendering` 时选择：可以把它概括为：Pipeline 定义绘制规则，Rendering 指定本次写入的附件。

本书把视口与 Scissor 设为动态状态。这里的“动态”有一个具体含义：Pipeline 创建时声明这两项将在命令缓冲区里提供，`begin_rendering` 随后用 `vkCmdSetViewport`、`vkCmdSetScissor` 写入本次值。只声明动态却不提供值，会留下缺失状态；如果只在 CPU 上修改 VkViewport，却没有录制相应设置命令，绘制结果也不会改变。以后改变输出尺寸时，你可以先沿着这两个真正生效的调用检查几何映射。

现在可以从最终结果反推流水线：颜色附件接收片元输出；片元的位置与插值属性由光栅化产生；光栅化处理经过裁剪和视口变换的图元；图元的顶点位置与属性由顶点 Shader 提供。一次 Draw 让 GPU 按管线规定的规则处理指定的顶点范围。

在已经开始的 Rendering 范围内，最核心的录制代码如下。`cmd` 是正在录制的命令缓冲区，`pipeline` 是前述函数创建的句柄；完整函数还负责视口、Scissor 及资源交接。

```
vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS, pipeline);
vkCmdDraw(cmd, 3, 1, 0, 0);
```

请连续做两次小改动，每次先预测再重建运行。第一次在顶点 Shader 的末尾加入 `gl_Position.xy *= 0.5;`：三个角向原点靠近，三角形缩小而颜色顺序保持。恢复后，把片元输出改为 `vec4(1.0, 1.0, 0.0, 1.0)`：形状保持，整面变黄。这两次实验分别定位了“位置在哪里决定”和“可见颜色在哪里决定”。最后恢复原代码，给后续章节保留一致起点。

如果图片只有背景，先确认 `run_triangle` 真的录制了 Draw，再检查 Pipeline、视口与 Scissor、Shader 编译产物是否更新，以及三个位置是否仍在可见范围。背景正确至少说明清屏和读回路径产生了预期结果，不能证明 Draw 的输入正确。若只有颜色不对，优先检查阶段之间的 `location` 和类型；若图形上下颠倒，先按当前正高度视口的公式计算一个顶点，避免在 Shader 和视口处同时翻转。

先计算，再对照答案

练习一：一个顶点输出 `(-0.5, 0.5, 0.2, 1.0)`，在本章视口里落到哪一个连续图片坐标？把 w 单独改成 2 后又在哪里？

练习二：把三角形最右顶点的 x 从 0.75 改成 1.5，其他值不变。应该看到整面消失，还是看到越界部分被裁掉？片元 Shader 是否负责执行这个裁剪？

答案与推理：第一问先除 w，再做视口变换。w=1 时得到 `(128,384)`；w=2 时 NDC 变为 `(-0.25,0.25,0.1)`，图片坐标为 `(192,320)`。这说明修改 w 会同时影响位置与深度。第二问中三角形有一部分仍位于裁剪范围内，因此应看到保留下来的区域；裁剪发生在顶点处理之后、光栅化之前，片元 Shader 接到的工作已经来自裁剪后的覆盖范围。[37]

第 7 章 顶点 Buffer 与字节布局

上一章想改变三角形的位置或颜色，需要修改 Shader 中的硬编码数组。如果要绘制上百个不同模型，这种做法显然难以维护，因为计算方法往往相同，变化的是顶点数据。我们因此把位置、颜色等属性放进 Buffer。但 GPU 看不到 C++ 的字段名称，只能按照字节读取；所以还要明确 Buffer 来源、顶点之间的步长，以及顶点内每个属性的偏移。

运行这一阶段：

```
cmake --build build --config Debug --parallel
./build/vulkan_lessons --lesson vertex --output out/vertex.ppm --validation
```

图片应与上一章的彩色三角形一致。Shader 不再保存硬编码的顶点位置，但画面仍与上一章一致，说明顶点数据确实来自 Buffer。完整流程在 `src/lessons.cpp` 的 `run_vertex`；创建和填充 Buffer 的通用部分在 `src/resources.cpp` 的 `create_buffer`，输入解释规则在 `src/render.cpp` 的 `create_graphics_pipeline`。

从一个熟悉的 C++ 数组出发

实验统一使用下列顶点结构。UV 是以后定位贴图的二维坐标，本章尚不读取它，但保留这个字段，便于后续实验复用结构。

```
#include <cstddef>
#include <type_traits>

struct Vertex {
    float position[3];
    float color[3];
    float uv[2];
};
static_assert(std::is_standard_layout<Vertex>::value);
static_assert(sizeof(float) == 4);
static_assert(sizeof(Vertex) == 32);
static_assert(offsetof(Vertex, color) == 12);
static_assert(offsetof(Vertex, uv) == 24);

const Vertex vertices[] = {
    {{ 0.00f, -0.75f, 0.0f}, {1.0f, 0.0f, 0.0f}, {0.5f, 0.0f}},
    {{ 0.75f,  0.65f, 0.0f}, {0.0f, 1.0f, 0.0f}, {1.0f, 1.0f}},
    {{-0.75f,  0.65f, 0.0f}, {0.0f, 0.0f, 1.0f}, {0.0f, 1.0f}}
};
```

一个顶点占 32 字节。前 12 字节是位置，接下来的 12 字节是颜色，最后 8 字节是 UV。三个顶点的有效数据共 96 字节。这只是有效顶点数据的字节数；给资源分配 DeviceMemory 时，仍然要遵守 `vkGetBufferMemoryRequirements` 返回的大小、对齐和可用内存类型。

一个顶点内的字节范围	含义	Shader 中的输入
0–11	三个 float 的位置	location 0，vec3
12–23	三个 float 的颜色	location 1，vec3

一个顶点内的字节范围	含义	Shader 中的输入
24–31	两个 float 的 UV	本章不读取；贴图阶段用 location 2

这张表描述的是本工程的 Vertex 布局，不是所有 C++ 结构体都必然如此。成员类型、声明顺序以及对齐规则都可能改变实际字节布局。请用 offsetof 获取字段偏移，用 sizeof 获取整体步长，并通过编译期断言检查本章的布局假设。

例如，将位置从三个 float 扩展为四个后，颜色字段可能从偏移 12 移到 16，顶点记录的总大小也可能改变。如果仍按原先的 32 字节步长读取，下一个顶点就会从错误位置开始。修复时必须同步修改上传的数据布局和 Vertex Input 描述，而不是先去调整片元 Shader 的颜色计算。

Binding 和 Location 分别负责什么?

设绑定时的 Buffer 起始偏移是 B，顶点序号是 i，步长是 S，字段偏移是 O，则当前字段的读取位置是 B+i×S+0。例如读取第二个顶点的颜色，B=0、i=1、S=32、O=12，所以从 Buffer 的字节偏移 44 开始读取三个 float。这个简单算式解释了大部分顶点输入错误。[36]

Vulkan 将“数据来自哪里”和“送入哪个 Shader 输入”分开描述。Binding 定义一条顶点数据流的步长，以及按顶点还是按实例推进；Attribute 则定义某个 Shader Location 从哪条 Binding 读取、起始偏移是多少、按什么格式解释。本章的位置与颜色共用 Binding 0，但分别交给 Location 0 和 1。[36]

下面是 `create_graphics_pipeline` 为这个阶段设置的核心内容。`binding` 和 `attributes` 是传入创建函数的 C++ 描述数据，创建 Pipeline 时由 `VkPipelineVertexInputStateCreateInfo` 引用。

```
VkVertexInputBindingDescription binding{};
binding.binding = 0;
binding.stride = sizeof(Vertex);
binding.inputRate = VK_VERTEX_INPUT_RATE_VERTEX;

VkVertexInputAttributeDescription attributes[2]{};
attributes[0].location = 0;
attributes[0].binding = 0;
attributes[0].format = VK_FORMAT_R32G32B32_SFLOAT;
attributes[0].offset = offsetof(Vertex, position);

attributes[1].location = 1;
attributes[1].binding = 0;
attributes[1].format = VK_FORMAT_R32G32B32_SFLOAT;
attributes[1].offset = offsetof(Vertex, color);

VkPipelineVertexInputStateCreateInfo input{};
input.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
input.vertexBindingDescriptionCount = 1;
input.pVertexBindingDescriptions = &binding;
input.vertexAttributeDescriptionCount = 2;
input.pVertexAttributeDescriptions = attributes;
```

格式名称里的 R32G32B32 在这里表示三个 32 位分量，SFLOAT 表示有符号浮点格式。格式名中的 R、G、B 并不意味着它必须承载颜色；位置同样能用这个格式。若实际存的是 float，却声明成 32 位整数，GPU 不会读取 C++ 的类型信息来修正错误，它会按照声明的格式解释字节。后续 texture 阶段再加入 location 2、R32G32_SFLOAT、偏移 24 的 UV 属性。[36]

Shader 相应变成从接口读取：

```
layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inColor;
layout(location = 0) out vec3 vertexColor;

void main() {
    gl_Position = vec4(inPosition, 1.0);
    vertexColor = inColor;
}
```

顶点输入的 Location 1 与顶点 Shader 输出的 Location 0 属于不同接口。前者是读入顶点属性，后者是向下一阶段传递数据。因此，最好同时说明“哪一阶段、输入还是输出”，而不是孤立记住一个 Location 编号。

写入顶点数据后，还要在绘制时绑定 Buffer

实验创建带有 VK_BUFFER_USAGE_VERTEX_BUFFER_BIT 用途的 Buffer，并为它绑定满足要求的内存。为了让第一次接触数据上传时少一条复制路径，这个小实验使用 CPU 可见的内存：映射后把 `vertices` 的字节复制进去；如果所选内存不是 HOST_COHERENT，按前章介绍的规则 flush。写入在提交绘制以前完成，并保持资源存活到 GPU 使用结束。

`src/resources.cpp` 的 `write_buffer` 封装了映射、复制、必要的 flush 与取消映射。可以在 `run_vertex` 中顺着下面两行代码追踪：`context` 是前面建立的设备与队列上下文，返回的 `Buffer` 是工程中同时管理句柄和分配内存的资源包装，而 `vertex.buffer` 才是传给 Vulkan 绑定命令的 `VkBuffer`。

```
Buffer vertex = create_buffer(context, sizeof(vertices),
                              VK_BUFFER_USAGE_VERTEX_BUFFER_BIT, true);
write_buffer(vertex, vertices, sizeof(vertices));
```

还有两种“偏移”也要分开：`vkBindBufferMemory` 的偏移说明整个 Buffer 放在 DeviceMemory 分配中的哪里；`vkCmdBindVertexBuffers` 的偏移说明此次顶点读取从 Buffer 内的哪里开始。本章字段偏移还要再加在后者之上。单个独立分配让前两种偏移在本实验中恰好都为 0，不能因此把它们理解为同一个参数。

CPU 可以映射的内存不一定适合存放大量、长期由 GPU 读取的模型数据。以后可以先把模型写入 Staging Buffer，再复制到更适合 GPU 访问的顶点 Buffer。这样，目标 Buffer 还需声明 TRANSFER_DST 用途，并建立复制写入到顶点属性读取的依赖。上传方法可以改变，但本章的 Stride 和 Attribute 读取规则仍然适用。[39]

顶点数据写好以后，还需要在命令录制阶段选择这个 Buffer。下面 `vertexBuffer` 是已经填充的 Buffer 句柄，`cmd` 正在 Rendering 范围内录制，Pipeline 已经绑定。

```
const VkDeviceSize vertexOffset = 0;
vkCmdBindVertexBuffers(cmd, 0, 1, &vertexBuffer, &vertexOffset);
vkCmdDraw(cmd, 3, 1, 0, 0);
```

vkCmdBindVertexBuffers 只把某个 Buffer 连接到顶点输入的 Binding，并不会帮你把顶点数据写进去；反过来，即使 CPU 已经上传了顶点，也必须在绘制时正确绑定。数据写入、接口解释和绘制绑定，是三个独立的步骤。[36]

现在将 src/lessons.cpp 中 triangle_vertices 的第一个顶点 x 坐标临时改为 0.35。重新构建后，上角向右移动，下方两个角不变；Shader 算法保持不变。再恢复位置，把三个颜色都改为 `(0.2,0.8,0.9)`，三角形应成为统一的青色。你已经可以通过修改 C++ 数据控制形状与颜色。

从画面异常推断哪段字节可能被读错

如果第一个顶点正常，后面的顶点逐渐偏离预期，先检查 Stride：步长错误会随顶点序号逐步累积。如果位置正确但颜色混乱，检查颜色的 Offset 与 Format；如果整个三角形消失，检查 Buffer 是否已写入、绑定偏移和 Draw 范围。验证层可以发现部分非法配置，但无法判断你原本希望这些字节代表位置还是颜色。

需要增加顶点时，先扩展数组和 Buffer 的有效数据，再调整 Draw 的顶点数。把 3 改成 6 而实际 Buffer 中没有另外三个可供读取的顶点，不是在复制原来的三角形，而是在请求读取后续数据。以后 Index Buffer 可以让多个三角形共享顶点；它解决复用索引的问题，不会替代本章对顶点记录的字节解释。

练习一：仍使用本章结构，绑定偏移设成 32、`firstVertex` 设成 0，且 Buffer 中确实存了足够顶点。此次绘制的第一个颜色字段位于原 Buffer 第几个字节？

练习二：把 C++ 的 `color` 放到 `position` 前面，但 Shader 仍然要求 location 0 是位置、location 1 是颜色。必须改 Shader 的算法吗？怎样用最少改动保持正确画面？

答案与推理：第一题把 B=32、i=0、O=12 代入，得到 44。绑定偏移改变的是整条流的起点，字段偏移仍然相对一个顶点。第二题不必改变 Shader 算法，只要上传的是新的结构布局，并让 attribute 的 offset 通过 `offsetof(Vertex, position)`、`offsetof(Vertex, color)` 跟随字段顺序。若结构大小也改变，stride 同样要跟随。输入接口保持了“location 0 是位置”的承诺，内部存储顺序可以调整。

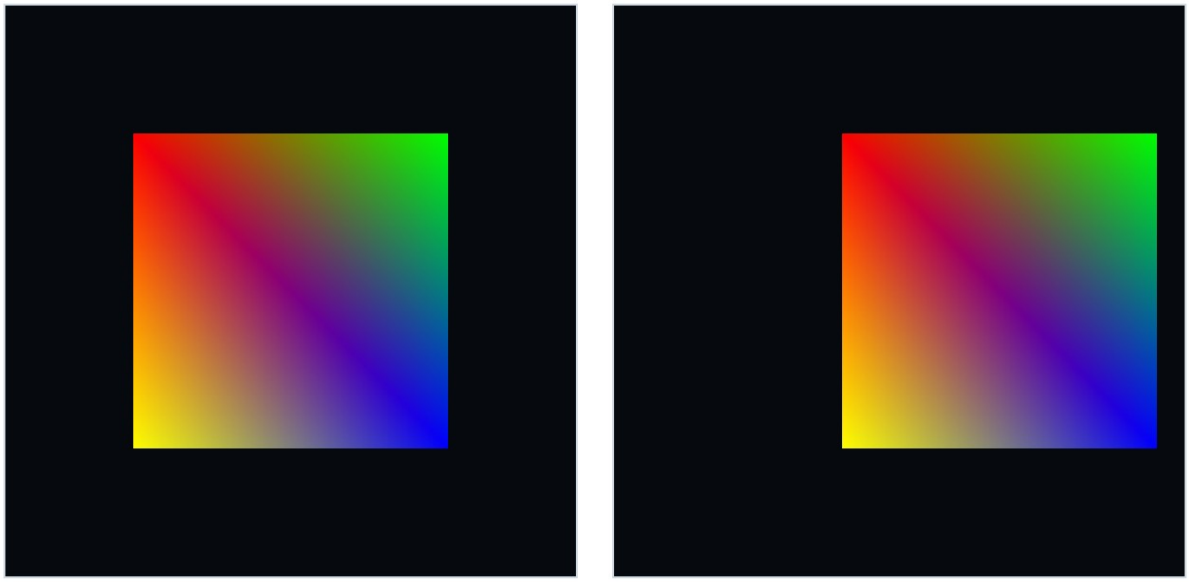
第 8 章 相机矩阵与 Shader 资源接口

顶点数据现在可以独立修改，但如果要让整个场景平移，每次改动所有顶点就很麻烦。许多顶点其实共享同一套变换：相机位置、投影方式或物体自身的旋转。我们可以让模型顶点保持原样，把变换参数交给 Shader。这样就出现两个新问题：数学上怎样表示变换？Vulkan 又怎样让 Shader 读取这些参数？

本章的第一幅实验只使用平移矩阵，以便独立验证资源连接；随后再把它放回真正相机变换的框架。这个最小实验的 `--shift` 表示对场景施加的 X 平移量，单位是本例 w=1 时的 NDC 坐标，正数使场景向画面右方移动。它没有把命令行参数当成相机的世界坐标。

```
cmake --build build --config Debug --parallel
./build/vulkan_lessons --lesson camera --shift 0 --output out/camera_center.ppm --
  validation
./build/vulkan_lessons --lesson camera --shift 0.35 --output out/camera_right.ppm --
  validation
```

预期第一张是居中的彩色矩形，第二张保持形状与颜色、整体右移。读 `src/lessons.cpp` 的 `run_camera`、`src/math.hpp` 的 `make_camera_matrix`，再对照相机顶点 Shader。矩形由两个三角形组成，其模型坐标范围是 ±0.55。



几何平移约 89.6 像素；本次前景质心移动 89.5 像素

图 5 用矩阵参数改变画面位置

从一次平移理解四阶矩阵

假设一个点位于 (x, y, z)，现在要将它沿 X 轴移动 t，得到 (x+t, y, z)。直接相加很简单，但我们还希望使用统一的形式表达缩放、旋转和投影。于是将点写成齐次列向量 (x, y, z, 1)，再乘以一个 4×4 矩阵。

“矩阵乘列向量”可以按四个输出分量理解。一个输出分量等于矩阵对应行与输入四个分量逐项相乘再相加。例如第一行是 (1,0,0,t) 时，新的 x 就是 1×x+0×y+0×z+t×1=x+t。最后一行取 (0,0,0,1)，输出 w 仍为 1。第四个分量使平移也能用相乘表达，不是无关紧要的填充值。

$$\begin{bmatrix} 1 & 0 & 0 & t \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x+t \\ y \\ z \\ 1 \end{bmatrix}$$

在本书使用的 GLSL 默认矩阵布局中，mat4 按列存储；“列向量参与运算”和“矩阵字节按列排列”是两个要分别约定的事情。为了让 CPU 与 Shader 一致，本书都使用列主序：连续四个 float 是第一列，再四个是第二列。平移量 t 位于数学矩阵的第一行第四列，在线性数组里是 m[12]。不要因为它显示在第一行，就误写成 m[3]。[40]

下面给出工程中这份数据的明确 C++ 表示。std::array<float,16> 提供连续的十六个 float；CPU 字段名 view_projection 与 GLSL 字段名 viewProjection 不同，只要实际字节顺序与偏移一致，就不影响读取。完整实验由 make_camera_matrix 生成等价矩阵并上传。

```
#include <array>
struct alignas(16) CameraUniform {
    std::array<float, 16> view_projection{};
};
static_assert(sizeof(CameraUniform) == 64);

CameraUniform camera{};
camera.view_projection[0]  = 1.0f;
camera.view_projection[5]  = 1.0f;
camera.view_projection[10] = 1.0f;
camera.view_projection[15] = 1.0f;
camera.view_projection[12] = 0.35f;
```

按列阅读这个矩阵，可以得到 (1,0,0,0)、(0,1,0,0)、(0,0,1,0)、(0.35,0,0,1)。将第一个顶点的 x=−0.55 代入，可得 x=−0.20。所有顶点的 x 都增加 0.35，矩形大小保持不变。对于宽 512 像素的视口，0.35 个 NDC 单位对应约 89.6 像素。

真正的相机为什么通常包含三个变换

模型顶点通常先用自身的局部坐标表示。例如，一个立方体以自身中心为原点，模型矩阵 M 将它变换到世界坐标；视图矩阵 V 将世界坐标变换到相机坐标；投影矩阵 P 再把相机坐标变换成裁剪坐标。之后才进行裁剪、透视除法和视口变换。采用列向量时，完整关系为 clip = P×V×M×local，最右侧的 M 最先作用。

$$p_{clip} = P\,V\,M\,p_{local}$$

相机向右移动 1 个单位，与保持相机不动、让场景整体向左移动 1 个单位，产生相同的相对位移。因此不旋转的相机位于世界坐标 (c_x,c_y,c_z) 时，V 的平移量为 (-c_x,-c_y,-c_z)。相机旋转以后，V 还要撤销相机的旋转；它是相机世界变换的逆。这个反向关系解释了为什么本实验 --shift 0.35 对应场景右移，而不能直接描述为“相机向右移动 0.35”。

正交投影不会因距离变化而改变物体在画面上的大小；透视投影则使远处的物体看起来更小。直观地说，同样宽的物体离相机越远，占据的视角就越小。在透视投影中，w_clip 与位置的深度有关，随后除以 w，便产生了远小近大的效果。

先明确下面这组坐标约定：相机朝自己的负 Z 方向看，近裁剪距离 n、远裁剪距离 f 都为正，且 0<n<f；竖直视角为 θ，宽高比为 a，令 s=1/tan(θ/2)。投影可写成 x_clip=(s/a)×x_view、y_clip=−s×y_view、z_clip=[f/(n−f)]×z_view+[fn/(n−f)]、w_clip=−z_view。Y 的负号是为了配合本书正高度视口，让相机坐标的正 Y 在图片里向上。把 z_view=−n 代入会得到 z_ndc=0，把 −f 代入会得到 1；这与本书 Vulkan 深度范围一致。[37]

$$x_{clip} = \frac{s}{a} x_{view}$$
$$y_{clip} = - s\, y_{view}$$
$$z_{clip} = \frac{f}{n-f} z_{view} + \frac{fn}{n-f}$$
$$w_{clip} = - z_{view}$$

使用这份透视矩阵时，要先把当前位于 z=0 的模型移到相机前方，例如在视图变换中让 z 减少 2。否则会得到 w_clip=0，无法完成正常的透视除法。注意，矩阵的数值虽然改变，但仍可以通过同一条 Uniform Buffer 和 Descriptor 通路传给 Shader。

Shader 怎样理解这 64 个字节?

顶点 Shader 的资源声明如下。`Camera` 是 `Uniform` 块类型名，`camera` 是 `Shader` 访问该块的实例名；二者不是 Vulkan 句柄。`set=0,binding=0` 指明资源接口位置，`std140` 规定块中成员的存储布局。

```
layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inColor;
layout(location = 0) out vec3 vertexColor;

layout(set = 0, binding = 0, std140) uniform Camera {
    mat4 viewProjection;
} camera;

void main() {
    gl_Position = camera.viewProjection * vec4(inPosition, 1.0);
    vertexColor = inColor;
}
```

`std140` 规定 `uniform` 块成员的对齐与布局规则，不能直接假定它与 C++ 结构体布局相同。本例中的列主序 `mat4` 相当于四列 `vec4`，每列占 16 字节，共 64 字节，起始位置按 16 字节对齐。因为块内只有一个矩阵，所以没有复杂的成员间填充。[41][40]

加入其他成员时就要重新推导。比如 GLSL 写 `float exposure; vec3 light; mat4 viewProjection;`，在本书采用的 std140 布局中，exposure 从偏移 0 开始，light 要从满足其 16 字节对齐的偏移 16 开始，矩阵再从偏移 32 开始。如果直接用普通 float 数组按 1+3+16 的顺序紧密排列，后两部分会从偏移 4 和 16 开始，不能直接上传使用。一个稳妥做法是按已核对的偏移补 padding，并用 `offsetof` 和 `sizeof` 断言。[41]

本章的成员对齐与另一项限制也要分清：当一个大 Buffer 中连续存放多个 Uniform 块时，descriptor 的 Buffer offset 还必须满足设备报告的 `minUniformBufferOffsetAlignment`。它可能比 16 大。我们当前让这份独立 Buffer 从 offset 0 开始，所以不需要通过猜测对齐值来放置多个块。[42]

从数据内容到资源接口：追踪一次 Shader 读取

第一步，确认数据内容：一份带 VK_BUFFER_USAGE_UNIFORM_BUFFER_BIT 用途的 Buffer 存着刚才的 64 字节，CPU 负责把矩阵写进去，并完成必要的缓存维护。Shader 运行时读到的数来自这里。

第二步，确认资源连接：DescriptorSet 的 binding 0 保存“从哪一个 Buffer、哪个 offset、多少字节范围读取”的描述。`vkUpdateDescriptorSets` 更新的是这个描述，不是把矩阵内容复制到 Shader 内部。下面 `device` 是逻辑设备，`uniformBuffer` 是已填充的句柄，`descriptorSet` 是分配好的集合。[42]

```
VkDescriptorBufferInfo bufferInfo{};
bufferInfo.buffer = uniformBuffer;
bufferInfo.offset = 0;
bufferInfo.range = sizeof(CameraUniform);

VkWriteDescriptorSet write{};
write.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
write.dstSet = descriptorSet;
write.dstBinding = 0;
write.descriptorCount = 1;
write.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
write.pBufferInfo = &bufferInfo;
vkUpdateDescriptorSets(device, 1, &write, 0, nullptr);
```

最后看接口：DescriptorSetLayout 声明每个 Binding 所需的资源类型、数量以及可访问的 Shader 阶段；PipelineLayout 则将不同 Set 的 Layout 按 Set 编号组织起来，并参与 Pipeline 创建与绑定。这些 Layout 只规定接口结构，不保存实际的相机矩阵数据。[42]

```
VkDescriptorSetLayoutBinding cameraBinding{};
cameraBinding.binding = 0;
cameraBinding.descriptorType = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
cameraBinding.descriptorCount = 1;
cameraBinding.stageFlags = VK_SHADER_STAGE_VERTEX_BIT;
```

完整 `run_camera` 调用 `src/render.cpp` 的 `create_bindings` 创建这个集合布局，再从 DescriptorPool 分配集合；池既要有足够的 Uniform Buffer Descriptor 容量，也要有足够的 Descriptor Set 名额。创建 PipelineLayout 时，把集合布局放在 `pSetLayouts[0]`，对应 Shader 的 set 0。执行 Draw 前再调用下列绑定。这里的 `pipelineLayout` 不是 Image 的布局状态，两个名字里的 layout 解决的是不同问题。

```
vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_GRAPHICS,
                        pipelineLayout, 0, 1, &descriptorSet,
                        0, nullptr);
```

如果只将平移量从 0 改为 0.35，且 Buffer 句柄、范围与 Shader 接口都没有改变，那么只要在安全时机修改 Buffer 内容即可。通常不必重建 Pipeline，也不必重新写入 Descriptor。如果改为读取另一个 Buffer，则需要更新资源连接，或者绑定指向新资源的另一份 Set。连续帧时还要确保 GPU 已经结束对旧数据的读取，第 13 章会专门处理。[43]

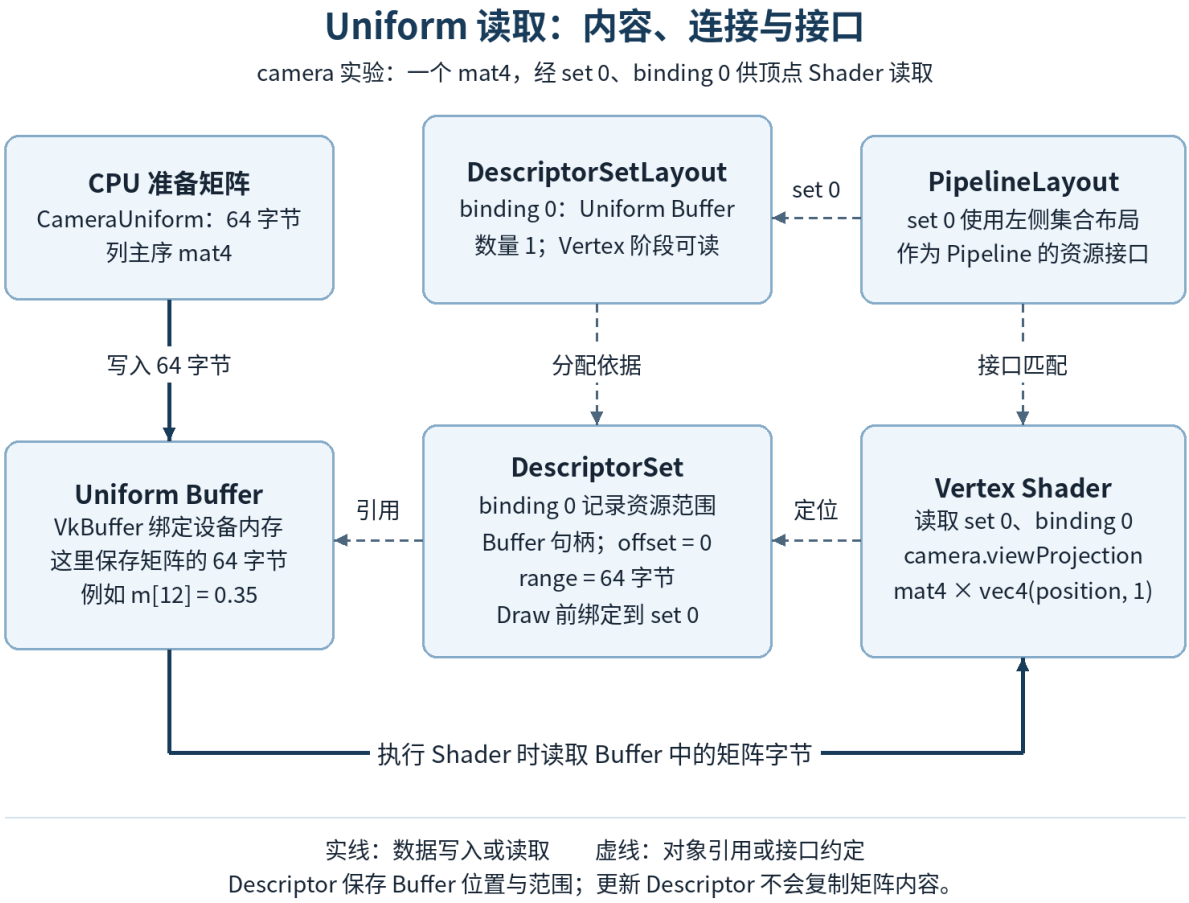


图 6 Uniform 内容与资源接口的连接

保持资源接口不变，将平移矩阵替换为透视矩阵

理解了矩阵和 Uniform Buffer 的连接后，就可以进行一次透视投影练习。在 `src/math.hpp` 的 `lab` 命名空间内加入下面函数，并在文件开头加入 `<cmath>`。本段的 n=0.1、f=10 是正的近远距离，s 对应 60 度竖直视角；方形输出使宽高比为 1。P 与 V 都按列存储，V 把 z=0 的平面移到相机前方 2 个单位。

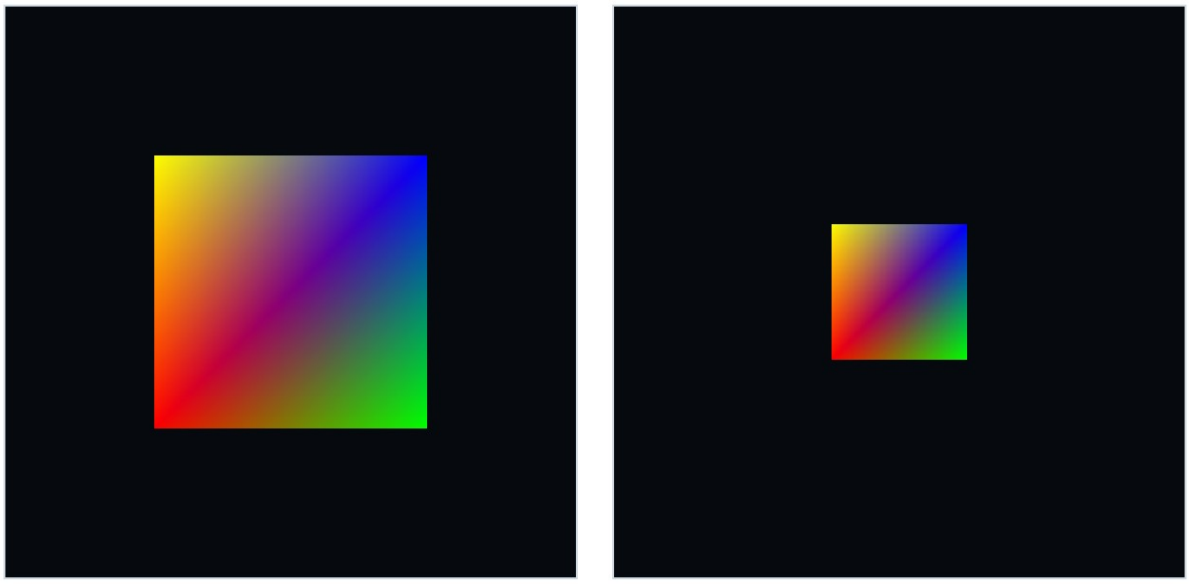
```
inline CameraUniform make_perspective_demo_matrix() {
    const float nearPlane = 0.1f;
    const float farPlane = 10.0f;
    const float distance = 2.0f;
    const float s = 1.0f / std::tan(3.1415926535f / 6.0f);
    const float A = farPlane / (nearPlane - farPlane);
    const float B = farPlane * nearPlane / (nearPlane - farPlane);

    const std::array<float, 16> P = {
        s, 0, 0, 0,
        0, -s, 0, 0,
```

```
        0, 0, A, -1,
        0, 0, B, 0
    };
    const std::array<float, 16> V = {
        1, 0, 0, 0,
        0, 1, 0, 0,
        0, 0, 1, 0,
        0, 0, -distance, 1
    };
    CameraUniform result{};
    for (int column = 0; column < 4; ++column)
        for (int row = 0; row < 4; ++row)
            for (int k = 0; k < 4; ++k)
                result.view_projection[column * 4 + row] +=
                    P[k * 4 + row] * V[column * 4 + k];
    return result;
}
```

循环实现矩阵乘法：结果的第 row 行、第 column 列，等于 P 的第 row 行与 V 的第 column 列逐项相乘再求和。不要交换 P 与 V；矩阵相乘的顺序会改变变换含义，不能把 P×V 随意交换为 V×P。

把 `run_camera` 中生成矩阵的那次 `make_camera_matrix(...)` 调用临时换成 `make_perspective_demo_matrix()`，重新构建，用 `--shift 0` 运行 camera。矩形仍居中，宽高约是原来的 0.866 倍；颜色的上下排列会随本段采用的正 Y 向上约定翻转。再只把 distance 从 2 改为 4，矩形的宽高都应再缩成一半。这才是由距离影响投影大小的证据。此扩展函数不接收 shift，练习完恢复原调用，后续纹理实验继续使用固定基线。



前景宽高 244×244 与 122×122，均保持居中

图 7 透视练习中距离增倍后的投影变化

也可以使用已经写好的 extras/perspective_demo.hpp：在 lessons.cpp 顶部加入 `#include "../extras/perspective_demo.hpp"`，再按上面的步骤替换一次矩阵调用。直接包含这个头文

件与在 math.hpp 手工加入函数，选择一种方式即可，避免重复定义。单独构建并验证的结果是：distance=2 时前景宽高为 244×244，distance 为 4 时为 122×122；两者均居中，验证层与同步验证均未报告错误。可在 evidence/perspective_summary.json 对照记录。

画面没有移动时，先查实际上传的 `m[12]`，再查 descriptor 指向的 Buffer 与 range，最后查 Shader 是否使用这个 binding。移动时出现奇怪缩放，常是把平移误写到 `m[3]` 等位置，改变了 w；完全看不见时，检查矩阵是否全零、是否按列排列，以及透视情况下模型是否确实处于相机前方。按照数据内容、资源连接、接口和算法逐项追查，能够避免盲目重建所有对象。

练习一：矩阵保持本章形式，把 `--shift` 从 0 改为 −0.25，图片会向哪里移动，大约移动多少像素？如果这个变化来自无旋转相机的横向移动，相机应向哪里移动？

练习二：`vkUpdateDescriptorSets` 已调用，随后在提交前把同一 Buffer 中的平移量改成 0.1，并按需要 flush。GPU 应读取旧平移还是新平移？为什么？

答案与推理：第一题场景向左移动约 64 像素，因为 NDC 横向长度 2 对应 512 像素，0.25 对应 64。与它等价的相机移动方向是向右。第二题应读取满足同步与缓存要求的新内容，因为 descriptor 保留的是资源位置与范围，并未拍下旧矩阵的一份快照。前提是更新发生在安全时机，且没有 GPU 与 CPU 同时读写这份内容的冲突。[42][43]

第 9 章 纹理上传与采样

前面的三角形通过顶点颜色插值得到内部颜色，适合表达渐变，却难以表现木纹、文字或棋盘图案。现在需要把一张图片用作颜色来源，并建立物体表面位置到图片位置的映射。纹理中的元素称为纹素（texel），最终输出图片中的元素称为像素（pixel）。一个像素可能由多个纹素共同决定；放大纹理时，多个像素也可能读取同一个纹素。

为避免把图像文件解码和 Vulkan 的资源流程混在一起，本章不读取外部图片。程序先在 CPU 上生成一张 64×64 的红白棋盘，再通过 Staging Buffer 上传，让 Shader 对它进行采样。这样可以直接观察：数据怎样从 CPU 进入 Image，再怎样影响输出画面。

```
cmake --build build --config Debug --parallel
./build/vulkan_lessons --lesson texture --output out/texture.ppm --validation
```

预期看到深色背景中央的棋盘矩形。完整入口是 `src/lessons.cpp` 的 `run_texture`；`src/resources.cpp` 中的 `make_texture_pixels` 生成像素，`upload_texture` 完成上传；`src/render.cpp` 的 `create_bindings` 建立 Shader 资源连接。对应的两个 Shader 是 `shaders/texture.vert`、`shaders/texture.frag`。

先把图案放到表面上

纹理坐标通常写作 `(u,v)`，所以顶点结构里的二维字段叫 UV。本例规定矩形左上角的 UV 为 (0,0)、右上角为 `(1,0)`、右下角为 `(1,1)`、左下角为 `(0,1)`。两个三角形组成矩形，Shader 把顶点 UV 输出给光栅化阶段，内部片元由插值得到自己的 UV。

当片元 UV 接近 `(0.5,0.5)` 时，就在贴图中心附近取颜色。对于 64×64、使用归一化坐标的图，纹素 `(i,j)` 的中心对应 `((i+0.5)/64,(j+0.5)/64)`。归一化坐标 0 和 1 分别位于纹理坐标的两端，具体边界外怎样取值还由采样器规则决定。这里的 v 方向遵从我们生成像素与分配 UV 的约定；若加载的图像采用另一种行顺序，应选择一个地方调整，避免 CPU 翻转与 Shader 翻转叠加。[44]

UV 的作用是建立表面到图片的映射，不要求它等于物体的 x、y。把右边两角的 u 改成 2，就让横向 UV 范围跨过两次完整贴图宽度。图案是否重复，取决于 Sampler 的寻址模式。

采样一张纹理，需要哪些对象？

Image 定义纹理的尺寸、格式、Mip 数量、Array Layer 数量及允许用途，并通过内存绑定获得存储空间。本章使用 RGBA8，至少需要 TRANSFER_DST（允许复制写入）和 SAMPLED（允许 Shader 采样）两种用途。采用 OPTIMAL Tiling 时，不应假设绑定内存能按 CPU 的逐行数组格式直接写入。[13]

ImageView 选择这个 Image 的哪部分以哪种视图访问。本章是二维颜色视图，覆盖 mip 0、array layer 0。View 不复制像素；同一 Image 的不同合法 View 可以让不同使用者看到不同范围。[20]

Sampler 规定坐标到颜色的取值规则。放大时使用 `magFilter`，缩小时使用 `minFilter`；NEAREST 选择一个纹素值，LINEAR 会在同一级相邻纹素之间插值。`addressModeU/V` 规定跨越边界后重复、钳制或采用其他允许方式。Sampler 也会控制下一章的 mip 选择范围与级间过滤。它不拥有贴图像素。[45]

本书使用 Combined Image Sampler Descriptor，把 ImageView 和 Sampler 一起连接到 Shader 的 sampler2D 接口。片元着色器先计算 UV，再通过绑定的 Descriptor 找到图像范围和采样规则，最后取得颜色。Image、ImageView、Sampler 和 Descriptor 共同完成这次采样，任何一个对象都不能单独替代整条通路。

输入纹理与最终颜色附件是两份不同的 Image。片元 Shader 从 64×64 棋盘中取样，为输出矩形计算颜色，再写入 512×512 的颜色附件。改变纹理 Sampler 不会改变输出附件尺寸；清除输出附件也不会改写输入纹理。先区分“谁被读取、谁被写入”，再检查 Image 句柄，就不容易混淆两条数据路径。

像素上传由复制命令完成

64×64 的 RGBA8 像素数据共 64×64×4＝16384 字节。CPU 先把这些字节写入带有 TRANSFER_SRC 用途的、可映射的 Staging Buffer，并按实际内存属性执行必要的 Flush；随后录制复制命令，将 Buffer 中的数据写入 Image。

下面代码中，cmd 是正在录制的命令缓冲区，stagingBuffer 已经含有像素，textureImage 是已经绑定内存的 64×64 Image。执行复制前，目标 Image 子资源必须已转换为 TRANSFER_DST_OPTIMAL。

```
VkBufferImageCopy copy{};
copy.bufferOffset = 0;
copy.bufferRowLength = 0;
copy.bufferImageHeight = 0;
copy.imageSubresource.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
copy.imageSubresource.mipLevel = 0;
copy.imageSubresource.baseArrayLayer = 0;
copy.imageSubresource.layerCount = 1;
copy.imageOffset = {0, 0, 0};
copy.imageExtent = {64, 64, 1};

vkCmdCopyBufferToImage(cmd, stagingBuffer, textureImage,
                       VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
                       1, &copy);
```

bufferRowLength 和 bufferImageHeight 为 0 表示按复制区域的宽高紧密排列；非零时，它们描述的是按纹素计算的行跨度和图像切片高度，而不是字节数。imageSubresource 选择目标的 mip 与 layer，imageOffset 和 imageExtent 再选该子资源内部的位置与尺寸。复制调用中的布局参数声明使用时的布局，并不替我们执行转换。[46][28]

这里发生了三种不同的操作：复制命令改变图像内容；布局转换改变 Image 子资源可用于何种访问的布局状态；同步依赖确保前面的写入能被后面的读取正确观察。Image Barrier 可以同时表达后两者，但不会生成棋盘像素，也不会自动复制 Staging Buffer 的内容。

首次使用新 Image 且无需保留旧内容时，先从 UNDEFINED 转成 TRANSFER_DST_OPTIMAL，目标范围是本章唯一的颜色子资源，目标阶段/访问是 COPY 与 TRANSFER_WRITE。复制结束后，下一位使用者是片元 Shader 的采样读取，因此交接如下。textureImage 与 cmd 的含义同上。[39]

```
VkImageMemoryBarrier2 ready{};
ready.sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2;
```

```
ready.srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT;
ready.srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT;
ready.dstStageMask = VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT;
ready.dstAccessMask = VK_ACCESS_2_SHADER_SAMPLED_READ_BIT;
ready.oldLayout = VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL;
ready.newLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
ready.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
ready.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
ready.image = textureImage;
ready.subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1};

VkDependencyInfo dependency{};
dependency.sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO;
dependency.imageMemoryBarrierCount = 1;
dependency.pImageMemoryBarriers = &ready;
vkCmdPipelineBarrier2(cmd, &dependency);
```

这些 Stage/Access 参数根据本例的前后访问者确定，不能不加判断地用于顶点 Shader 采样或 compute 写入的情况。staging Buffer 至少保持到复制完成；纹理 Image、View、Sampler 与相关 descriptor 则保持到所有引用它们的设备工作完成。

完整 `upload_texture` 会录制这一段上传命令，提交后等待完成，再返回给 `run_texture` 准备绘制。因此第一次跟做时可以在两个清晰的节点观察：函数返回前，GPU 已经完成棋盘像素的上传及相应布局转换；之后的 Draw 才消费这张图。以后为了并行而把上传和绘制分开提交时，需要保留同样的先后依赖，不能把现在的等待删掉以后期待对象创建顺序替代它。

将上传后的纹理连接到片元 Shader

纹理实验仍使用 set 0、binding 0 读取相机 Uniform Buffer，新增的 combined image sampler 放在 set 0、binding 1，集合布局中这个 binding 的可见阶段包含 FRAGMENT。下面 `textureView` 和 `sampler` 已创建，`textureSet` 已按包含这两个 binding 的布局分配。描述中的 `imageLayout` 应与采样时实际子资源布局一致；填这个字段仍然不是执行布局转换。[42][20]

```
VkDescriptorImageInfo imageInfo{};
imageInfo.sampler = sampler;
imageInfo.imageView = textureView;
imageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;

VkWriteDescriptorSet write{};
write.sType = VK_STRUCTURE_TYPE_WRITE_DESCRIPTOR_SET;
write.dstSet = textureSet;
write.dstBinding = 1;
write.descriptorCount = 1;
write.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
write.pImageInfo = &imageInfo;
vkUpdateDescriptorSets(device, 1, &write, 0, nullptr);
```

Shader 的核心读取如下。`texture.vert` 读取 location 0 的位置与 location 2 的 UV，把 UV 输出到 location 1；本阶段不读取顶点颜色，因此 Pipeline 只声明实际需要的属性。

```
layout(location = 1) in vec2 vertexUV;
layout(set = 0, binding = 1) uniform sampler2D colorTexture;
layout(location = 0) out vec4 outColor;

void main() {
    outColor = texture(colorTexture, vertexUV);
}
```

先把 run_texture 中的 const auto mesh = quad_vertices(0.8f); 临时换成下面两行，使横向 UV 范围从 0–1 扩大到 0–2。修改在 write_buffer 上传以前发生。

```
auto mesh = quad_vertices(0.8f);
for (auto& vertex : mesh) vertex.uv[0] *= 2.0f;
```

再到 create_bindings 把 Sampler 的 U 寻址方式设为 REPEAT。重新运行，横向应出现两份图案。保持 UV，再把 U 改成 CLAMP_TO_EDGE，超出范围的部分会延续边缘颜色。下一次只把 magFilter 在 NEAREST 与 LINEAR 间切换，比较被放大的棋盘边界。每次只观察一个因素：UV 如何变化、边界怎样取样，或相邻纹素怎样插值。结束后恢复网格与采样器，下一章从相同棋盘起步。

若矩形全为一种颜色，临时让片元输出 vec4(vertexUV,0.0,1.0)：若出现二维渐变，UV 通路大致成立，继续检查上传、View 与 descriptor；若 UV 渐变也没有出现，先检查顶点 location 2 与阶段输出 location 1。若棋盘被错行切开，核对每行字节数、复制区域和 CPU 数据顺序。若只有运行几次后才出问题，检查上传是否完成就释放 staging，以及同步范围是否真的覆盖采样子资源。

练习一：descriptor 已指向正确 ImageView，但去掉 vkCmdCopyBufferToImage，只保留两次布局转换，应该期待棋盘仍然存在吗？

练习二：只把 LINEAR 改为 NEAREST，需要重新上传像素吗？若换一张内容不同、同样大小格式的图片，是否必须改变 Shader 接口？

答案与推理：第一题不应期待棋盘存在，布局转换没有提供像素内容；从 UNDEFINED 开始也不承诺保留可用图案。第二题采样规则改变不要求改写原纹素，可创建相应 Sampler 并更新资源连接。换同类图片时可以沿用 sampler2D 接口，但要真正上传新内容、安排正确的读写交接；若换成另一张 Image，还要让 descriptor 选中相应 View。几何 UV、像素内容与资源连接由此可以分别变化。

第 10 章 mipLevels 与 arrayLayers

假设一张 64×64 的棋盘被显示在仅有 8×8 像素的远处平面上。一个输出像素平均要代表约 8×8 个纹素，如果每次只读取其中一个，结果就容易在红白之间跳动，随着相机移动产生闪烁。同一级的 LINEAR 过滤只混合附近少量纹素，无法直接代表覆盖区域内的全部细节。为缩小显示预先准备低分辨率纹理，正是 Mipmapping 的用途。

同一张纹理按分辨率逐级减半得到的一组图像，称为 Mip 链。原图是 Mip 0，后续级别的宽高通常各减半，最低保持为 1。完整的 64×64 链包含七级：64、32、16、8、4、2、1。创建 Image 时将 mipLevels 设为 7，只表示资源包含七个级别，不代表 Vulkan 已自动填充后六级的内容。[13][47]

观察不同 Mip Level 中的实际内容

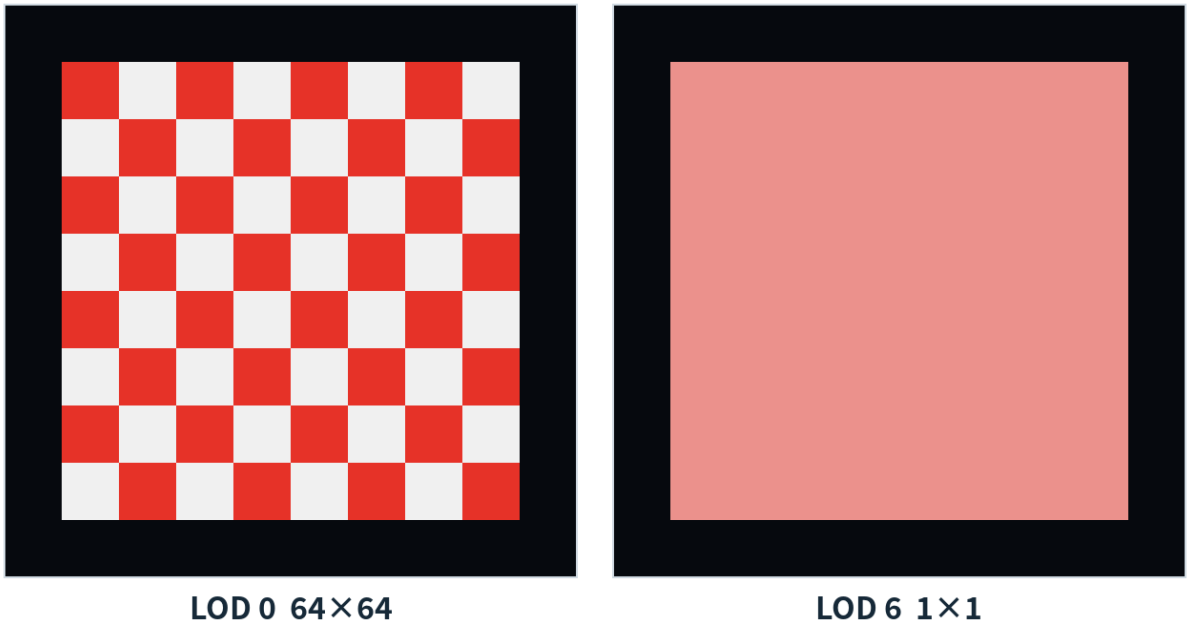
`src/lessons.cpp` 的 `run_mips` 使用上一章的棋盘，通过 `src/resources.cpp` 的 `upload_texture` 逐级执行线性 blit，实际生成所有 mip；对应片元 Shader 为 `shaders/mips.frag`。下面两次运行会采样同一 Image 中不同的 Mip Level。

```
cmake --build build --config Debug --parallel
./build/vulkan_lessons --lesson mips --lod 0 --output out/mip0.ppm --validation
./build/vulkan_lessons --lesson mips --lod 6 --output out/mip6.ppm --validation
```

第 0 级应显示清楚的棋盘；第 6 级只有一个纹素，把它放大到整块矩形后应得到近似均匀的淡红色。淡红色来自红格和浅色格在缩小过滤过程中的合成，不是程序给某一级预先涂上的演示颜色。终端还会打印每一级实际 blit 的目标尺寸。

注意，并非 Mip Level 每增加一级，图片就一定肉眼可见地变模糊。本实验的棋盘格与二倍缩小恰好对齐，而且使用 NEAREST 显示指定的整数 LOD，所以前几级仍可能呈现相同图案。配套实测中，LOD 0～3 保持棋盘，LOD 4～6 变为平均颜色 (235,145,140)。对比 LOD 0 与 6，可以直接观察 Shader 选中了不同分辨率的内容；其他级别是否正确生成，还应结合命令日志与目标子资源检查。

本实验通过 textureLod 显式给定细节等级（LOD），用来检查不同 Mip 的实际内容。通常，片元 Shader 的 texture 采样会根据屏幕相邻片元的 UV 变化，估算一个输出像素覆盖多少纹素，再选择合适的细节等级。例如，约 8×8 个纹素对应一个输出像素时，LOD≈$\log_2 8 = 3$，可把 Mip 3 当作直观参考。实际采样还受到 ImageView 范围、Sampler 的 LOD 限制与过滤规则影响。[44]



同一个 Image 的不同 mip，显式 LOD 与最近点采样

图 8 显式采样 mip 0 和 mip 6 的对照

生成 Mip 链时，每一级都有自己的读写关系

创建 mip 图像时，除了 SAMPLED 和 TRANSFER_DST，还需要 TRANSFER_SRC：刚生成的一级要作为下一次缩小的源。`upload_texture` 先检查格式在 OPTIMAL tiling 下是否支持 blit source、blit destination 及所需线性过滤；不支持时报告当前方法无法使用，而不是继续执行未经支持的 blit。实际项目也可以在 CPU 预生成后上传，或选择其他已支持的生成路径。[48]

整个过程可以按一级追踪。先把 CPU 像素复制到 mip 0，它此刻是 TRANSFER_DST；要生成 mip 1，就让 mip 0 的写入对 blit 读取可见，并把 mip 0 转为 TRANSFER_SRC。mip 1 在 TRANSFER_DST 中接收缩小结果。下一轮又让 mip 1 从“刚被写完”变成“用于读出”，生成 mip 2。最后一级也要转换为 Shader 可读取布局。

下面抽取 `upload_texture` 中一次 blit 的主体。`image` 是同时包含源级与目标级的同一个 Image，`level` 为目标级，`sourceWidth/Height` 与 `destinationWidth/Height` 分别表示源级和目标级的尺寸；目标尺寸按 `max(1,源尺寸/2)` 计算。此时源、目标子资源已各自处于代码声明的布局。

```
VkImageBlit region{};
region.srcSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, level - 1, 0, 1};
region.srcOffsets[0] = {0, 0, 0};
region.srcOffsets[1] = {sourceWidth, sourceHeight, 1};
region.dstSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, level, 0, 1};
region.dstOffsets[0] = {0, 0, 0};
region.dstOffsets[1] = {destinationWidth, destinationHeight, 1};

vkCmdBlitImage(cmd, image, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
               image, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
               1, &region, VK_FILTER_LINEAR);
```

和直接 copy 相比，blit 的源、目标尺寸可以不同，并按指定过滤方式重新取样。本例在同一 Image 的不同 mip 间操作，源与目标区域并不重叠。同步范围必须随一级一级的用途变化：COPY 或 BLIT 的写入交给下一次 BLIT 的读取；完成的级数再交给 FRAGMENT_SHADER 的采样读取。不同 Mip 可能处于不同 Layout，不能在这种情况下把整个 Image 都当作同一种旧布局来转换。[48]

资源、ImageView、Sampler：三个条件缺一不可

即使七个 Mip 都已生成，如果 ImageView 只包含一个级别，Shader 仍无法通过这个 View 访问其余级别。反过来，即使 View 覆盖七级，Sampler 的 maxLod 仍为 0，也无法采样更高的 Mip。因此，至少要同时检查 Image 是否有内容、View 覆盖哪些级别，以及 Sampler 允许哪些 LOD。[20][45]

本实验的 View 从 mip 0 开始包含七级，Sampler 允许 0 到 6。`minFilter` 控制某一级内部怎样组合相邻纹素，`mipmapMode` 控制不同级之间怎样选择或混合。把这两个参数都简单理解为“是否模糊”的开关，就无法解释为什么高 mip 明明存在却一直看不见。[45]

完整二维、每次减半的 mip 链增加的纹素量约为 1+1/4+1/16+…≈4/3 倍基础级；这是面积随宽高同时减半得到的几何级数。它不是每增加一级就再分配一张同样大的图，也不是省略原图。对于本实验，后面各级仍保存同一棋盘逐渐丢失细节后的结果。

Array Layer 表示另一张同规格的图像

现在有另一个需求：使用同一个采样接口，在红白棋盘、绿色横条和蓝黄竖条之间切换。三张图都需要完整的 64×64 分辨率，不是同一图案的不同细节。我们可以创建一张二维数组 Image，将它们分别存入三个 Array Layer。所有层共享基础尺寸、格式及 Mip 级数，但每层都有独立内容。[13][47]

`run_layers` 实际上传了三个数组层的像素，并使用 `VK_IMAGE_VIEW_TYPE_2D_ARRAY` View 和 `shaders/layers.frag` 中的 `sampler2DArray`。运行下面的命令，只改变选层参数：

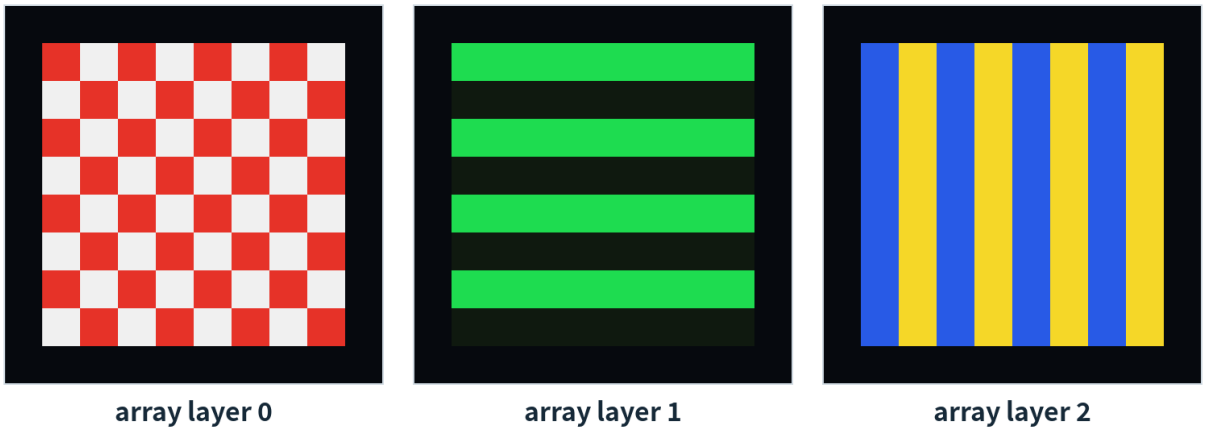
```
./build/vulkan_lessons --lesson layers --layer 0 --output out/layer0.ppm --validation
./build/vulkan_lessons --lesson layers --layer 1 --output out/layer1.ppm --validation
./build/vulkan_lessons --lesson layers --layer 2 --output out/layer2.ppm --validation
```

三次运行应分别显示红白棋盘、绿与深色交替的横条、蓝黄竖条，矩形形状和位置不变。采样器的数组层坐标用来选择其中一层，本书分别使用整数 0、1、2。二维数组纹理不会像三维纹理那样自动在相邻层之间进行线性插值；需要混合两层时，应分别采样再自行组合。[44]

```
#version 450
layout(location = 1) in vec2 vertexUV;
layout(set = 0, binding = 1) uniform sampler2DArray colorTexture;
layout(push_constant) uniform Options {
    layout(offset = 0) float lod;
    layout(offset = 4) int layer;
} options;
layout(location = 0) out vec4 outColor;

void main() {
    outColor = texture(colorTexture, vec3(vertexUV, float(options.layer)));
}
```

为了让命令行里的 LOD 和 layer 很容易送入 Shader，两个实验还各使用了 8 字节的 Push Constants。它是一小段由 `vkCmdPushConstants` 写进命令状态、按 PipelineLayout 声明的阶段与字节范围供 Shader 读取的数据；这里 float 的 offset 为 0，int 的 offset 为 4。它不代替纹理 descriptor，也没有上传图像。完整 `run_mips`、`run_layers` 在 Draw 前写入这两个值，并声明 FRAGMENT 阶段的相应范围；mips Shader 使用 lod，layers Shader 使用 layer，未使用的另一个字段只是让两者复用同一结构。[42]



一个 Image 中三份 64×64 内容，均使用 mip 0

图 9 同一个 Image 的三个数组层

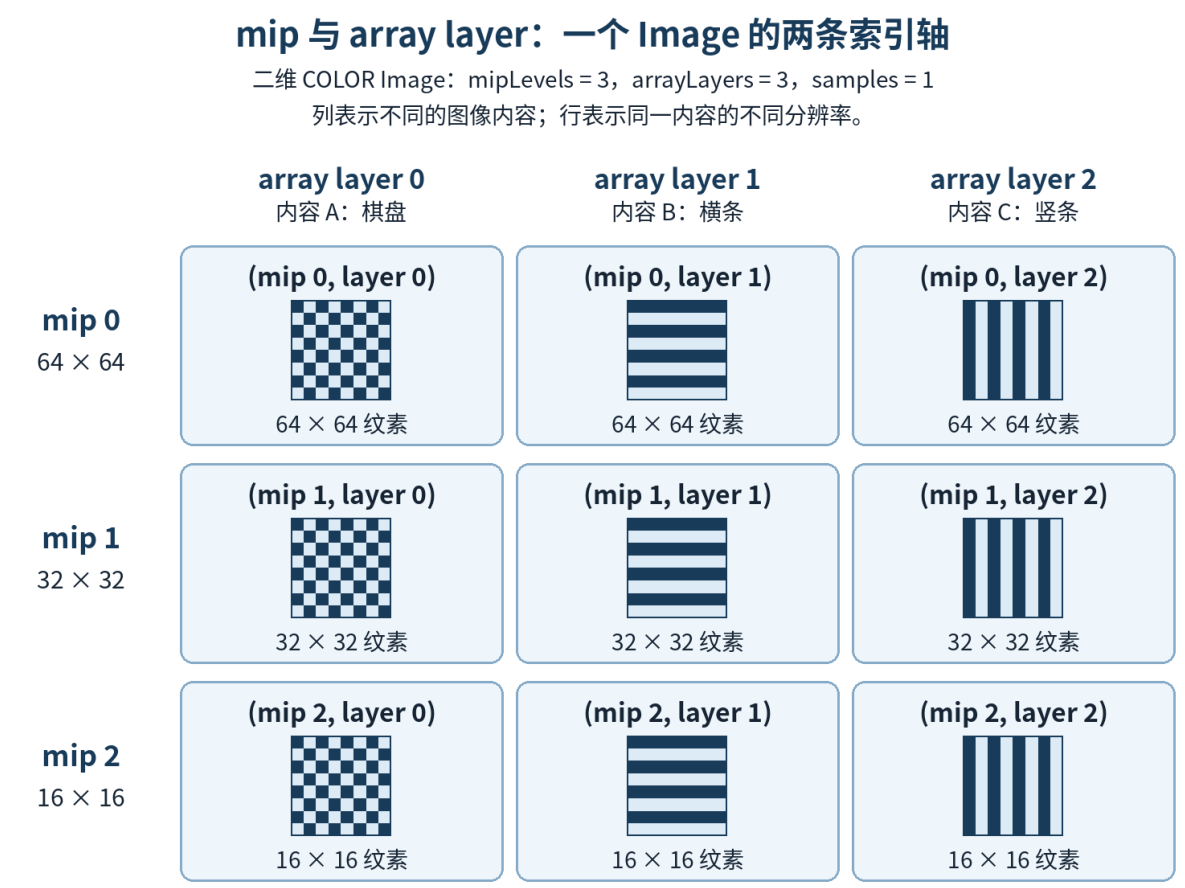


图 10 三种分辨率与三份内容组成九个子资源

用 3×3 网格同时理解 Mip 与 Array Layer

为了区分 Mip 与 Array Layer，假设创建一张基础尺寸 64×64、mipLevels=3、arrayLayers=3 的 Image。它的颜色子资源可排成下面的网格：每个格子都由一个 (Mip, Layer) 组合确定。注意，这只是解释范围的示意；实际运行的 Mips 实验是七级一层，Layers 实验是一级三层。

颜色子资源	array layer 0	array layer 1	array layer 2
mip 0	64×64，(0,0)	64×64，(0,1)	64×64，(0,2)
mip 1	32×32，(1,0)	32×32，(1,1)	32×32，(1,2)
mip 2	16×16，(2,0)	16×16，(2,1)	16×16，(2,2)

`VkImageSubresourceRange` 在这张网格上选择一个范围。例如下面选中后两行与后两列相交的四格：

```
VkImageSubresourceRange range{};
range.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
range.baseMipLevel = 1;
range.levelCount = 2;
range.baseArrayLayer = 1;
range.layerCount = 2;
```

两个 Count 字段表示范围包含多少级或多少层，而非结束索引。因此 mip 范围是 1、2，layer 范围也是 1、2。这样的范围可用于 View 或 barrier，但复制区域用的 `VkImageSubresourceLayers` 一次指定一个 mip、若干 array layer；不要把两个结构仅因名字相近就互换。[28][49]

如果一个 View 的 `baseMipLevel=1`，Shader 相对该 View 的 LOD 0 从原 Image 的 mip 1 开始；同理，View 从 array layer 1 开始时，Shader 选择视图层 0 对应原 Image 层 1。区分“Image 的绝对编号”和“通过 View 看到的相对编号”，才能解释同一张底层图像在不同 View 下的采样结果。[20][44]

还有第三个独立参数 `samples`。它描述每个像素或纹素位置的采样数，用于多重采样等机制；`samples=4`（仅作概念示意）不代表四个数组层，也不代表四级 `Mip`。本书所有实验都使用 `VK_SAMPLE_COUNT_1_BIT`。在本书的普通 Vulkan 1.3 图像创建路径中，多采样图像的 mipLevels 必须是 1；改变 samples 还需要匹配渲染附件与 Pipeline 的采样配置，不能只改一个 Image 字段就期待整个程序自动完成抗锯齿。[13]

练习一：`mipLevels=3,arrayLayers=3` 的二维颜色 Image 有几个 `(mip,layer)` 子资源？上面的 range 覆盖几个？每层向下缩小后，layer 数是否也跟着减半？

练习二：`--lod 6` 仍然显示清晰棋盘，已经确认参数到达 Shader。接下来依次检查哪些事实？`--layer 1` 却仍显示棋盘时，检查方向又有什么不同？

答案与推理：第一题共有 9 个颜色子资源，range 覆盖 4 个；缩小的是每层内部的二维尺寸，层数仍然是 3。第二题先核对 Image 是否真的创建并填充了七级，再核对 View 是否暴露它们、Sampler 是否允许 LOD 6。层选择异常则检查 2D_ARRAY View、View 的 baseArrayLayer/layerCount、实际上传的层内容以及采样第三坐标；它不靠调整 minFilter 解决。这里每一步都在回答“所选数据到底位于哪个子资源”。

第 11 章 多物体与深度

前面的矩形由两个三角形拼成，画面没有发生互相遮挡。现在加入两个有重叠区域的三角形：红色在近处，蓝色在远处。如果只写颜色附件且关闭深度测试，那么较晚绘制的片元可能覆盖较早的颜色。这样，后绘制的远处蓝色就可能盖住近处红色，结果取决于绘制顺序，而非空间中的远近关系。

要恢复正确的前后关系，可以为每个采样位置保存当前通过测试的深度。新片元到来时，将自身深度与保存的值比较，决定是否保留。用于保存这些深度值的 Image 就是深度附件。深度测试使不透明物体的遮挡可以按空间深度判断，而不必由 CPU 事先精确排好每一个片元的绘制次序。

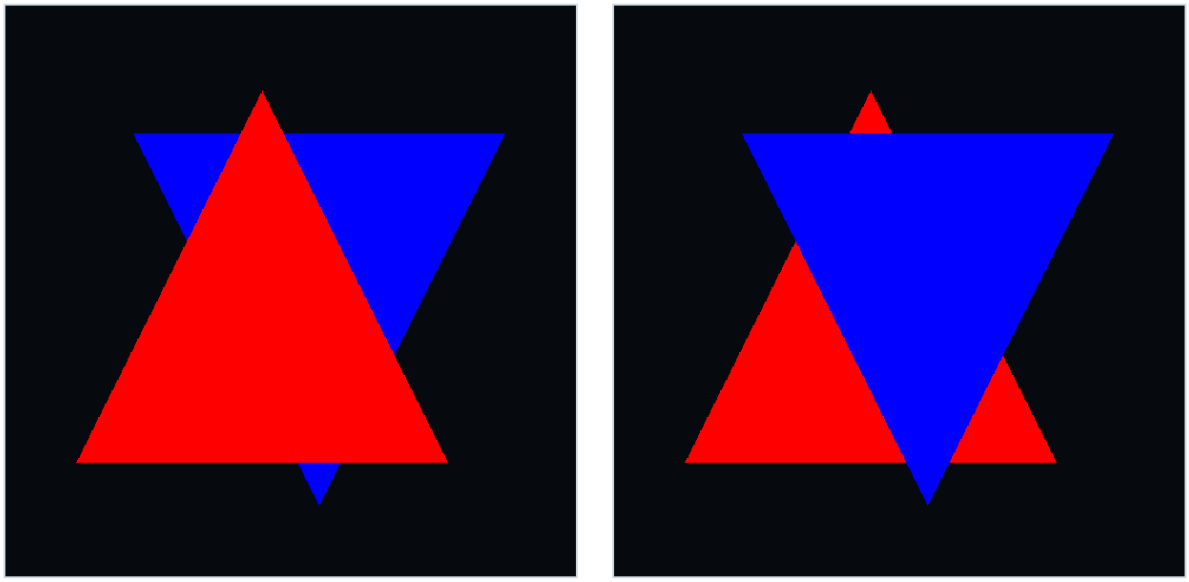
用两张结果图观察遮挡关系

`src/lessons.cpp` 的 `run_depth` 故意先画近处红色，再画远处蓝色。两者有一块明显交叠区，红色顶点的 z 为 0.2，蓝色为 0.7，w 都是 1。命令顺序让关闭测试时的错误容易被看见。

```
cmake --build build --config Debug --parallel
./build/vulkan_lessons --lesson depth --output out/depth_on.ppm --validation
./build/vulkan_lessons --lesson depth --no-depth --output out/depth_off.ppm --
  validation
```

启用深度测试时，两个三角形交叠处应显示近处的红色；关闭测试时，那里会显示后绘制的蓝色。红色未覆盖的蓝色区域仍会出现在两张图中。比较这两张图片，比单纯看到两个三角形更能说明深度测试真正改变了结果。

在完整代码中，这两个物体的六个顶点放在同一 Buffer，前三个是红色，后三个是蓝色。绑定一次 Buffer 与 Pipeline 后，先调用 `vkCmdDraw(cmd,3,1,0,0)`，再调用 `vkCmdDraw(cmd,3,1,3,0)`。第二次的 firstVertex=3 从后半段开始取顶点。Vulkan 不需要因为我们把它们称为“两个物体”就创建两条 Pipeline；只要输入接口与绘制规则相同，多个 Draw 就可以复用它们。



几何体与提交顺序不变，只改变深度测试与写入开关

图 11 打开与关闭深度测试的实际输出

深度值从裁剪坐标的 z 和 w 得到

回到第 6 章：顶点 Shader 输出裁剪坐标，透视除法给出 `z_ndc=z_clip/w_clip`，然后视口把它映射到 `[minDepth,maxDepth]`。本书将 minDepth 设为 0、maxDepth 设为 1，因此视口变换后的深度值与 z_ndc 相同。这里的 0.2 和 0.7 因为 w=1，恰好保持不变。[37][50]

在真实透视场景中，深度值通常不是“距相机几米”的原始距离。投影矩阵把相机空间的位置变换为裁剪坐标，除以 w 后得到用于比较的深度；它与距离之间通常不是线性关系。因此，不能直接拿世界坐标中的 z 去和另一个片元的投影深度比较。本章统一使用投影后的深度，并让较小的数表示更近。

三角形内部也要得到每个采样位置的深度。默认情况下，流水线对投影后的深度按屏幕空间位置进行插值；它并不是直接插值原始的相机距离。片元 Shader 可以在需要时写 `gl_FragDepth` 改变测试用深度，但本书不写它，因此深度来自固定功能的光栅化路径。[38][50]

当比较方式为 LESS，新的深度必须小于附件中的已有深度才通过。在 LESS 比较下，将深度清为 1，就为后续更近片元的通过预留了空间。红色的 0.2 先和 1 比较，通过并写入 0.2；蓝色的 0.7 再来，`0.7<0.2` 为假，交叠片元被拒绝。蓝色在红色之外的位置仍与初始的 1 比较，因此能显示出来。[51]

深度测试和深度写入必须分别决定

测试开关决定是否比较，写入开关决定通过测试后是否更新深度附件。两者服务于同一个流程，却影响不同结果。在绘制不透明物体的本例中，两者都开启，并关闭 stencil 与 depth bounds 测试。

```
VkPipelineDepthStencilStateCreateInfo depthState{};
depthState.sType = VK_STRUCTURE_TYPE_PIPELINE_DEPTH_STENCIL_STATE_CREATE_INFO;
depthState.depthTestEnable = VK_TRUE;
depthState.depthWriteEnable = VK_TRUE;
depthState.depthCompareOp = VK_COMPARE_OP_LESS;
depthState.depthBoundsTestEnable = VK_FALSE;
depthState.stencilTestEnable = VK_FALSE;
```

如果开启深度测试却关闭深度写入，而且整个 Render Pass 都没有向深度附件写值，那么每个片元看到的仍是清除值 1。红色和蓝色都能通过 LESS 测试，后绘制的蓝色仍会覆盖红色。这说明，仅开启测试并不能让后续片元自动知道前面出现过的深度。

另一种常见用途是先画不透明物体、已经建立深度，再画某些半透明表面时保留测试而关闭深度写入。这样可以利用已有不透明深度判断遮挡，而不把当前透明层当成完全挡住后面内容的实体。透明混合仍有自己的绘制顺序问题，本章的 LESS 加写入解决的是当前不透明实验的可见性。若把 depthTestEnable 关闭，depthWriteEnable 并不会独立开启正常的深度写入。[51]

为深度测试准备真正的存储空间

深度附件仍是一张 Image，创建流程沿用第 3 章，但格式、用途与 View aspect 改变。`src/resources.cpp` 的 `choose_depth_format` 依次检查 `D32_SFLOAT`、`D16_UNORM`，寻找在 OPTIMAL tiling 下支持 `DEPTH_STENCIL_ATTACHMENT` 功能的格式。程序使用查询选出的结果创建深度图，也把同一格式交给 Pipeline 的动态渲染格式声明。[50]

深度图尺寸与本次颜色输出一致，为 512×512，samples 同为 1；用途包含 `VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT`。本实验不读回深度内容，也不在 Shader 中把它当纹理读取，所以不必因“它是一张图”而添加尚未使用的 transfer 或 sampled 用途。View 选择 `VK_IMAGE_ASPECT_DEPTH_BIT`。这里所选两种格式都没有 stencil 分量，深度与 stencil 也不应被误当成颜色的几个通道。

Dynamic Rendering 开始时，需要把实际深度 View 交给 `pDepthAttachment`。下面 `depthView` 是已经创建的 View，`rendering` 是前面已设置好颜色附件、renderArea 与 layerCount 的 `VkRenderingInfo`；其余字段仍沿用原代码。

```
VkRenderingAttachmentInfo depthAttachment{};
depthAttachment.sType = VK_STRUCTURE_TYPE_RENDERING_ATTACHMENT_INFO;
depthAttachment.imageView = depthView;
depthAttachment.imageLayout = VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;
depthAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR;
depthAttachment.storeOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
depthAttachment.clearValue.depthStencil = {1.0f, 0};
rendering.pDepthAttachment = &depthAttachment;
```

storeOp=DONT_CARE 表示渲染结束后，本实验不再要求保留深度内容；它不影响渲染期间的正常深度写入与后续片元比较。如果下一次渲染还要使用已有深度，就需要保存旧内容，并安排后续的加载和同步。

另外，`create_graphics_pipeline` 中挂在 Pipeline 创建信息上的 `VkPipelineRenderingCreateInfo::depthAttachmentFormat` 必须按实际深度附件格式声

明。仅仅在录制阶段添上 pDepthAttachment，并不能补上 Pipeline 对格式的约定；仅仅开启 depthTestEnable，也不会自动创建深度 Image。

为什么深度写入不属于片元着色器写入？

深度比较位于片元操作的一部分，实际实现中，深度测试可能出现在片元着色器运行之前或之后。相应同步阶段是 EARLY_FRAGMENT_TESTS 与 LATE_FRAGMENT_TESTS；它们与 FRAGMENT_SHADER 阶段不是同一个访问类别。对当前入门路径，同时覆盖 early 与 late 能清楚表达我们依赖的是深度附件操作，不要求读者先判断每种 Shader 行为会把测试安排在哪里。[50]

第一次使用深度图时，不需要保留旧内容。可以让 transition_image 从 UNDEFINED 转到 DEPTH_STENCIL_ATTACHMENT_OPTIMAL，源阶段取 NONE、源访问为 0，目标阶段为下面两个位的组合，目标访问包含深度附件读与写。

```
const VkPipelineStageFlags2 depthStages =
    VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT |
    VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT;
const VkAccessFlags2 depthAccess =
    VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT |
    VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT;
```

读访问负责深度比较，写访问则对应清除和通过测试后的更新。使用深度专属的 aspect 选择范围，两个 queue family index 仍取 IGNORED，因为实验只有一条图形队列。稍后如果要复用上一次渲染的深度图，必须从上次真实的使用状态推导源阶段与访问；把 oldLayout 写成 UNDEFINED 只允许不保留内容，不能替应用等待尚未结束的访问。[50][39]

用一次修改验证一种故障原因

如果开启深度测试后只剩背景，先检查深度清除值是否误设为 0：在 LESS 规则下，深度 0.2 和 0.7 都无法通过。再检查投影后的深度范围、是否确实绑定了深度附件，以及深度测试与写入是否启用。如果结果仍由后绘制的颜色覆盖，还要检查是否在两次 Draw 之间错误清除了深度。

若两个表面几乎共面，局部出现不稳定斑纹，可能是投影后深度值过于接近，导致深度精度不足。先拉开距离验证，再检查近远裁剪面设置。盲目把所有问题归为缺少 barrier，会忽略这种来自几何与数值精度的现象。

现在在 run_depth 中交换两个三角形的 Draw 顺序，重新运行两个模式。启用深度的交叠区仍应为红色；关闭深度时，最后绘制的颜色会随顺序改变。恢复顺序后，再只将 depthWriteEnable 改成 VK_FALSE，观察它为何回到后绘制覆盖的表现。改完恢复基线，以便后面的同步与帧复用实验共享同一预期。

练习一：把深度清除值从 1 改为 0，仍用 LESS，本例两个三角形会怎样？如果把比较改为 GREATER，却继续用清除值 1，是否能自动得到正确的“反向深度”？

练习二：保持清除值 1、LESS 与写入，先收到深度 0.7，再收到 0.2，最后收到 0.5。这个位置最终保存哪个深度，为什么？

答案与推理：第一题清 0 后两者都无法通过 LESS；改成 GREATER 却清 1，也会让当前 0.2、0.7 都失败。反向深度需要让投影、比较方式与清除值共同遵守“更近对应更大值”的约定，不能只改一个比较枚举。第二题 0.7 先通过并写入，0.2 更小又通过并覆盖，0.5 大于已保存的 0.2 而失败，最终保存 0.2。此时绘制顺序可以改变中间过程，但对于本例不同深度的不透明表面，最后留下的仍是最近者。

第 12 章 从前后读写推导同步

前几章已经把三角形、纹理和深度数据变成了可见画面。现在暂时不看颜色，而是追踪一段数据的读写时间线：谁先写入，谁随后读取？即使全程只用一条图形队列，也必须解释：颜色附件刚完成写入时，后面的复制命令怎样才能读到正确结果？

录制顺序只说明命令如何排列，不能自动满足所有数据依赖。一次绘制包含顶点输入、Shader 执行和附件读写等阶段，前后命令的某些阶段可能重叠。因此，同步不是简单地给整幅图“加锁”，而是针对具体资源和可能冲突的访问，建立所需的执行顺序与内存依赖。[52]

先确定读写关系，再选择同步参数

打开 `src/render.cpp`，找到 `end_rendering_and_copy`。它先结束 dynamic rendering，随后将颜色 Image 复制到读回 Buffer，供 CPU 保存为 PPM。这里有两组不同的前后访问：第一组是 GPU 写颜色 image，随后 GPU 从它读取；第二组是 GPU 写读回 buffer，随后 CPU 从它读取。虽然它们组成同一条截图路径，但作用对象、消费者和同步方法都不相同。

执行依赖首先回答“前一个范围内的操作何时必须先于后一个范围内的操作完成”。但操作完成，不等于前次写入的数据就一定对后续访问可见。Vulkan 用“可用性”描述写入结果进入适当的内存域，再用“可见性”描述指定的后续访问可以观察这些数据。内存依赖负责把这两部分联系起来。读者无需猜测 GPU 有几级缓存，只要准确指出前后使用者，让实现完成必要的处理。[52]

常见冲突可分为三类。RAW（先写后读）要求消费者看到前一次写入；WAW（先写后写）要求后一次写入不会与前一次冲突；这两类通常需要正确的内存依赖。WAR（先读后写）只考虑这两次访问时没有旧写入要交给后续消费者，建立执行依赖即可避免旧读取尚未结束就发生覆盖。如果期间还做 Layout 转换，也必须分析转换本身的读写影响。[52][39]

将“颜色写完再复制”落实为 Image Barrier

下面是 `src/resources.cpp` 中 `transition_image` 最终构造的 image barrier 的展开阅读片段。它假定 `cmd` 正在记录、dynamic rendering 已结束，`colorImage` 就是刚绘制的颜色 image。完整调用位置在 `src/render.cpp:end_rendering_and_copy`，不需要读者另外拼接一个程序。

```
VkImageMemoryBarrier2 barrier{};
barrier.sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2;
barrier.srcStageMask =
    VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT;
barrier.srcAccessMask = VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT;
barrier.dstStageMask = VK_PIPELINE_STAGE_2_COPY_BIT;
barrier.dstAccessMask = VK_ACCESS_2_TRANSFER_READ_BIT;
barrier.oldLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
barrier.newLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL;
barrier.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
barrier.image = colorImage;
barrier.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
barrier.subresourceRange.baseMipLevel = 0;
barrier.subresourceRange.levelCount = 1;
```

```
barrier.subresourceRange.baseArrayLayer = 0;
barrier.subresourceRange.layerCount = 1;

VkDependencyInfo dependency{};
dependency.sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO;
dependency.imageMemoryBarrierCount = 1;
dependency.pImageMemoryBarriers = &barrier;
vkCmdPipelineBarrier2(cmd, &dependency);
```

这里的源 stage 来自颜色附件写入所处的阶段，因此是 COLOR_ATTACHMENT_OUTPUT。片元着色器虽然计算颜色，但附件写入的同步访问不是 FRAGMENT_SHADER/SHADER_WRITE。目标 Stage 根据后续的 vkCmdCopyImageToBuffer 确定，因此选择 COPY；这次操作从 image 读取，所以目标 access 是 TRANSFER_READ。stage 选择执行范围，access 选择该范围内需要处理的访问类型，它们必须相互匹配。[26]

oldLayout 和 newLayout 描述同一组子资源从旧布局转向新布局，但它们不能代替 Stage 与 Access。image 和 subresourceRange 指出这条 Barrier 针对哪个 Image、哪些 Mip 和 Array Layer。本实验的颜色图像仅有 Mip 0 和 Layer 0，因此各选一个；若以后使用数组纹理，照搬这个范围就只会处理 Layer 0。[26]

新建 Image 首次使用时，没有需要等待的早期 GPU 访问，可以用 UNDEFINED 作为旧布局，并将源 Stage/Access 设为 NONE。但同一 Image 第二次使用时，不能假装它又成为新对象。即使允许丢弃旧内容，也必须处理可能仍在进行的访问；UNDEFINED 只表示旧内容可被丢弃，并不能使访问自动完成。连续帧实验会记录每个 Image 的真实布局状态。[39][26]

将 GPU 写入的结果安全交给 CPU

颜色 image 上的 barrier 只完成了第一段交接。复制命令写的是另一个对象：读回 buffer。end_rendering_and_copy 在复制之后还记录一条 VkBufferMemoryBarrier2，源为 COPY/TRANSFER_WRITE，目标为 HOST/HOST_READ。它的 buffer 指向读回缓冲区，offset 为零，size 覆盖此次复制的字节；它不是颜色 Image 的第二次 Layout 转换，buffer 根本没有 image layout 字段。[30]

接下来提交这段命令，并等与这次提交关联的 fence。只有 vkWaitForFences 返回 VK_SUCCESS，CPU 才可以进入 readback_ppm 读取结果。如果内存不是 host coherent，还要在读取之前调用 vkInvalidateMappedMemoryRanges。完整顺序是：GPU 执行写入和读回依赖；提交完成后通知 Fence；CPU 等待成功；如有需要，执行 Invalidate；最后读取。invalidate 不等待 GPU；仅有 fence 也没有自动替你补上主机读取所需的全部内存依赖。[33][32]

HOST_COHERENT 只使主机侧的显式 flush、invalidate 不再必要。它不能让 CPU 提前读取正在复制的 buffer，也不能让 CPU 在 GPU 仍读取某个 UBO 时覆盖它。非 coherent 内存还要遵守 nonCoherentAtomSize 的范围规则；本工程映射完整 allocation，并由资源辅助函数处理所需范围。日后做子分配时，即使两个逻辑数据块不重叠，也应避免它们的并发访问落在同一个需要 flush 或 invalidate 的非一致性原子范围内。[19][53]

用同一种方法推导其他 Barrier

若把第 7 章的主机可见顶点 buffer 改为经 staging 复制上传的路径，复制的生产者是 COPY/TRANSFER_WRITE，消费者则是固定功能的顶点属性读取，目标应包含 VERTEX_ATTRIBUTE_INPUT/VERTEX_ATTRIBUTE_READ。不要因为名字里有“顶点”，就把目标写成顶点着色器。若 buffer 用作索引，消费者是 INDEX_INPUT/INDEX_READ。这里是在推导一种可迁移的上传方式，配套 Vertex 实验仍采用直接写入 HOST_VISIBLE Buffer 的方式。资源用途改变时，应重新找出真正执行访问的阶段。[39]

纹理上传的目标通常是 FRAGMENT_SHADER/SHADER_SAMPLED_READ。逐级生成 mip 时，刚生成的那一级下一步成为 blit 源，交接变为 BLIT/TRANSFER_WRITE 到 BLIT/TRANSFER_READ。最后转为采样布局时，要覆盖 shader 实际可能访问的 mip 和 layer。深度附件写入又在早期或晚期片元测试阶段；用二者的并集配合 DEPTH_STENCIL_ATTACHMENT_WRITE，可以覆盖本教材深度路径中的相关写入。[39]

把所有 Barrier 一律改成 ALL_COMMANDS 并不是可靠的修复方法。ALL_COMMANDS 可以扩大执行阶段范围，但不能自动修正 Buffer 句柄、补齐漏掉的 Array Layer，也不能让错误的 Access Mask 变成正确。排查初期可以在满足有效性要求的前提下暂用较宽范围；最终仍应能说明：哪个命令访问了哪块资源，下一次访问又需要什么保证。

还要区分三个层次。数据同步针对重叠的内存范围；对象生命周期要求销毁 buffer 前，所有已经提交且引用它的命令都完成，不能只因为其中一个范围空闲就销毁整个对象；主机线程同步则约束多个 CPU 线程如何访问 queue、command pool 等对象。GPU barrier 不能代替 C++ 互斥锁，而 C++ 互斥锁也不会等待 GPU 完成。相应 API 参考页的 Valid Usage 与 Host Synchronization 分别给出这些约束。[54][55][31]

动手验证与推理练习

在实验包根目录运行以下命令，然后沿上述两个函数逐行标出生产者、消费者和子资源范围。

```
mkdir -p out
./build/vulkan_lessons --lesson triangle --output out/sync_triangle.ppm --validation
./build/vulkan_lessons --lesson texture --output out/sync_texture.ppm --validation
```

预期第一张仍是有颜色插值的三角形，第二张能看见纹理图案；开启验证不会改变它们的设计结果。若输出图像只剩预期背景色，先确认绘制是否写入颜色目标；若 validation 报复制读取先前附件写入的 RAW 冲突，再检查颜色写到复制读这一段。二者的证据不同，不应只凭画面异常就随意删除或叠加 Barrier。没有报错也不是同步的数学证明，第 15 章会说明验证范围。

练习：假设把纹理生产者改为计算着色器写 storage image，片元着色器仍要采样结果。在同一条支持计算的队列上，原上传到采样的屏障应怎样改？

推理答案：先找写入，生产者已经不是 copy，而是计算着色器，所以源改为 COMPUTE_SHADER/SHADER_STORAGE_WRITE。消费者没有变，仍为 FRAGMENT_SHADER/SHADER_SAMPLED_READ。如果生产阶段使用 GENERAL，则从这个真实旧布局转到采样用的只读布局，范围覆盖实际写入并将被采样的子资源。创建 image 时还需要支持并声明

storage 与 sampled 两种用途，descriptor 中的布局也要与使用时的布局一致。只改 stage，不补这些资源条件，不能完成迁移；本题是设计练习，主工程尚未加入计算管线。[26]

第 13 章 CPU 与 GPU 并行及连续帧

单次绘制程序往往在完成一次提交后就退出，很容易忽略资源复用。连续帧中，CPU 可能已经开始写下一帧的相机矩阵，而 GPU 仍在读取上一帧的那一份数据。矩阵 Buffer 只是内存，并不知道“上一帧已经结束”。因此，首先要确定何时可以安全复用这段内存，再考虑怎样让 CPU 与 GPU 重叠工作。

本章使用 src/lessons.cpp 的 run_frames：它在同一进程中复用设备、Pipeline 和部分资源，持续更新 UBO、录制和提交绘制。--inflight 1 只使用一组帧资源；--inflight 2 使用两组独立的帧资源。这里的“帧槽”指一组需要整体管理并按完成状态复用的资源，和窗口 Swapchain 的 Image 编号不是同一回事。

先用单帧槽理解资源的安全复用

运行下面的命令。9 帧实验中，相机矩阵使用的水平位移从 −0.45 线性变化到 +0.45，`--shift` 可以在整条轨迹上附加偏移。矩阵构造在 `src/math.hpp:make_camera_matrix`。

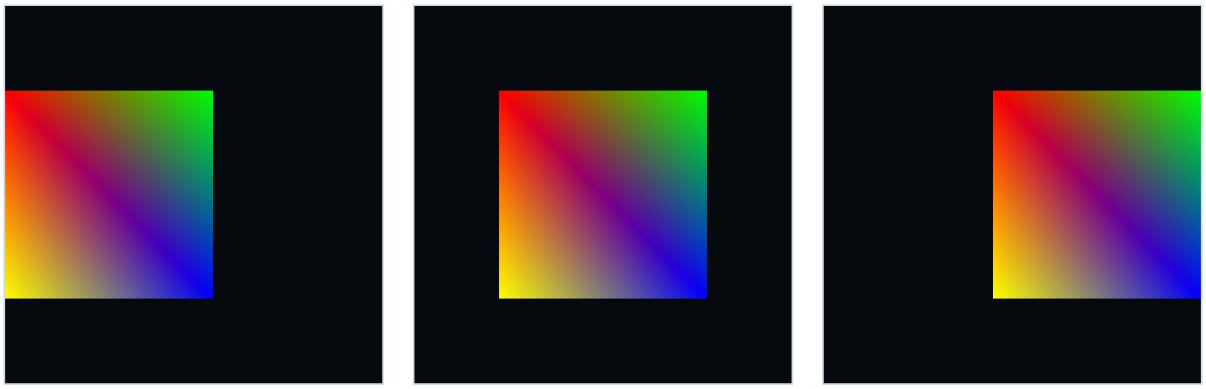
```
mkdir -p out
./build/vulkan_lessons --lesson frames --frames 9 --inflight 1 --output out/one.ppm --
  validation
```

检查 `out/one_000.ppm`、`out/one_004.ppm`、`out/one_008.ppm`，物体应从左侧经过中间到达右侧；`out/one.ppm` 与最后一帧对应。三张图采用同一套顶点与管线，变化来自每次更新的矩阵，不是重新生成三种几何体。这为随后诊断“CPU 的确在变、GPU 为什么没变”提供了最小案例。

单槽需要一个 UBO、引用它的 descriptor set、command buffer、输出 image、读回 buffer 和 fence。相关结构是 `src/lessons.hpp` 的 `FrameSlot`，各槽的 Descriptor Set 句柄保存在 bindings.sets 中。Fence 初始处于 signaled 状态，使第一次进入循环时不会等待一份从未提交过的工作。到下一次使用这个槽时，程序先等待它对应的 Fence；等待成功，才能处理上次提交产生的读回结果、改写 UBO、重置 CommandBuffer。`vkResetCommandBuffer` 不能作用于 Pending 状态的命令缓冲区，而且逐个重置要求其 CommandPool 创建时具有允许单独重置的标志。[55]

CPU 把本帧矩阵写入映射内存后，非一致性内存需要执行 Flush，再提交消费它的 GPU 工作。主机写入发生在提交之前，队列提交提供这里所需的主机到设备内存域交接；这与“提交以后继续修改同一段内存”有根本不同。后者会与仍未完成的设备读取冲突，flush 也不能使它变得安全。[53][52]

录制命令时绑定 Descriptor Set，并不会把当时的 UBO 内容复制到 CommandBuffer 里。Draw 执行时，Shader 仍要从被引用的 Buffer 读取数据。因此，同一份 Set 可以长期保持指向某个 UBO，而每帧只改写安全可用的内容；前提是上一轮 GPU 读取已经结束。反复更新描述符，也不会替你等待 GPU 或保存旧矩阵。



两槽实测输出；与单槽对应帧的 RGB 完全一致

图 12 连续帧的首帧 中间帧与末帧

为什么应在提交前才 Reset Fence?

Fence 复用可以理解为建立一次新的“提交完成”承诺：等待成功代表上次提交已完成；Reset 让 Fence 回到未触发；接下来的提交完成后才会重新触发它。如果 Reset 后由于错误或窗口重建而没有提交任何工作，下一次等待就可能一直无法完成。[32][56]

因此，`run_frames` 先完成矩阵准备和命令记录，在确认要调用 `vkQueueSubmit2` 的位置才 reset fence。下面摘出提交结构的关键写法。`cmd` 和 `frameFence` 都来自当前槽；结果检查在完整函数中执行。

```
VkCommandBufferSubmitInfo commandInfo{};
commandInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_SUBMIT_INFO;
commandInfo.commandBuffer = cmd;

VkSubmitInfo2 submitInfo{};
submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO_2;
submitInfo.commandBufferInfoCount = 1;
submitInfo.pCommandBufferInfos = &commandInfo;

VkResult resetResult = vkResetFences(device, 1, &frameFence);
if (resetResult != VK_SUCCESS) {
    throw std::runtime_error("vkResetFences failed");
}
VkResult submitResult =
    vkQueueSubmit2(graphicsQueue, 1, &submitInfo, frameFence);
if (submitResult != VK_SUCCESS) {
    throw std::runtime_error("vkQueueSubmit2 failed");
}
```

离屏单队列的这次提交没有需要等待的 acquire semaphore，也没有需要交给 present 的 semaphore，所以两个 semaphore 数量保持零。fence 作为 `vkQueueSubmit2` 的最后一个参数，让 CPU 以后能确认工作完成。提交要求 fence 未触发，并且没有关联尚未完成的工作；设备创建时也必须启用 `synchronization2`，仅仅具有 1.3 版函数声明还不够。[31]

若提交失败，不要把已 reset 的 fence 当作“正在正常处理下一帧”，继续进入下一轮等待。实验程序报告失败并退出正常渲染路径。VK_ERROR_DEVICE_LOST 同样需要进入错误处理，不能无限重试等待并把卡住归咎于慢显卡。

两组帧资源怎样使 CPU 提前准备下一帧?

先保留单槽输出，再运行双槽版本：

```
./build/vulkan_lessons --lesson frames --frames 9 --inflight 2 --output out/two.ppm --
  validation
```

第 n 帧通过 n % 2 选择 Frame Index。两组帧槽各自具有 UBO、Descriptor Set、CommandBuffer、Fence、颜色图像与读回 Buffer。如果还要同时运行两帧的深度渲染，也必须准备独立的深度目标，或者为共享深度目标建立额外同步。只把 Fence 扩成两个，却让 CPU 继续改写同一个 UBO，并不能实现安全的双缓冲。

到达的工作点	CPU 正在做什么	允许 GPU 同时做什么
提交第 0 帧后	准备槽 1 的矩阵并记录第 1 帧	读取槽 0 矩阵并绘制第 0 帧
提交第 1 帧后	请求复用槽 0，先等待槽 0 完成	执行已经提交的工作
槽 0 可复用后	读取需保存的第 0 帧结果，写入第 2 帧矩阵	使用槽 1 的数据执行第 1 帧

表中展示的是双槽程序允许发生的重叠，不保证每次实际运行都恰好按这个时间表执行。驱动调度、CPU/GPU 速度和磁盘写入都会影响时序。双槽的好处是 CPU 准备下一帧时，不必总因上一帧尚未完成而阻塞；它不会自动让帧率翻倍。

截图时机也会改变实验。若每次提交后立即等待，再读取并保存本帧图像，即使分配了两个槽，主循环仍会被自己串行化。工程在需要复用槽、已经等待完成时，由 save_completed_frame 读取该槽上一次需要保存的结果；循环结束后等待剩余提交，再保存尚未处理的结果。槽里的 pending_frame 记住结果属于哪一个实际帧号，否则第 0 帧的文件可能被误标成第 2 帧。

两组模式的第 0、4、8 帧应得到相同的物体位置。这检验的是：改变帧调度方式，并没有改变每一帧的输入和结果。Frame Index 用来选择资源槽，真正的帧号用来计算相机位置；如果误用槽编号计算位移，双槽只会在两个位置之间跳动。即使这类程序逻辑完全符合 Vulkan API 的使用规则，验证层也不会知道你想得到一段连续动画。

配套 evidence/summary.json 保存了本次运行结果，文件前缀使用 frames1 和 frames2。两种模式的第 0、4、8 帧 RGB 数据逐字节一致；把非背景像素的列索引取平均，三帧分别为 140.5、255.5、370.5，验证了位置向右推进。evidence/frames2.stdout.txt 还显示第 0、1 帧先后提交，随后在复用槽 0 时保存第 0 帧，符合延迟读回设计。这些记录来自 llvmpipe 软件 Vulkan 实现，验证的是代码路径、同步验证结果和输出，不是独立显卡上的吞吐测试。

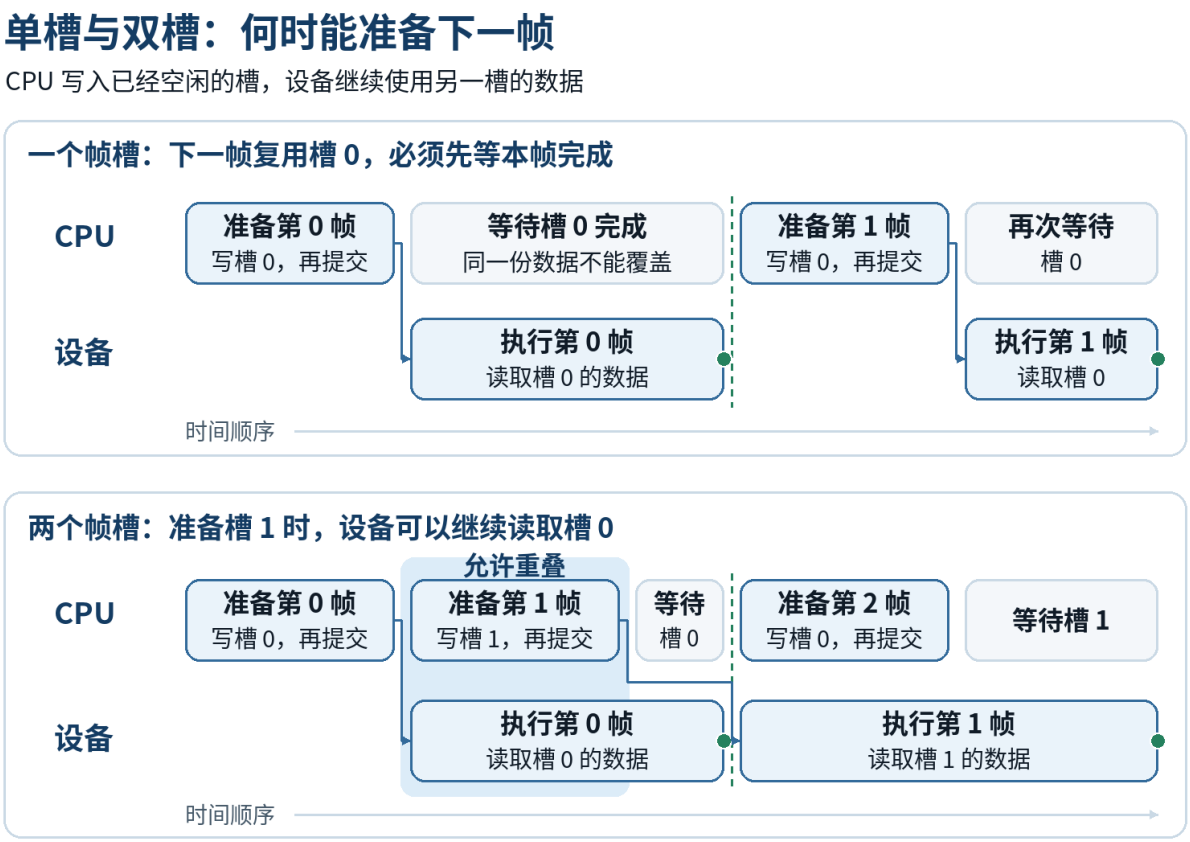


图 13 单槽与双槽的资源复用时序

哪些资源可以共享，哪些必须单独保护？

双槽复制的是会被并发修改或复用的状态，不是把整个 Vulkan 应用复制两遍。两帧在两帧中保持只读的顶点 Buffer、纹理和图形 Pipeline 可以共享，只要它们在使用期间保持有效。每帧要改写的矩阵和待重新录制的命令缓冲区则需要各自的完成证明。读回缓冲区与输出 image 若共享，还会额外引入帧间 RAW、WAR 或 WAW 关系；本实验使用独立目标，让你先看清 CPU 资源复用这一条主线。[54][55]

每个槽的输出 Image Layout 也应分别记录。槽 0 第二次投入使用时，要从槽 0 上次读回留下的布局继续；不能把“全局上一帧”的 image 状态套给它。若以后改为一个大 UBO 内的多个切片，应同时满足 uniform buffer 偏移对齐、非 coherent 原子范围要求，并保证对应 descriptor 真正指向当前切片。缓冲区分成两段，并不意味着访问范围已经自动正确。[30][53]

最后一帧提交成功，也不表示可以立即离开函数，让局部资源的析构函数开始销毁对象。循环结束后，应确认所有仍有提交的槽都完成，再读回余下快照，最后释放被这些提交使用的资源。单队列里的较后提交 fence 具有相应的前序同步范围，但初学实现逐槽收尾更容易与资源归属对上；不要因为刚刚等待过某个 Fence，就释放无法确认受其完成条件覆盖的资源。[54][31]

阅读日志时，要把 CPU 的动作与 GPU 的完成分开。“submitted frame 3”只说明 CPU 调用了提交；接着出现“prepare frame 4”说明程序没有在这两个日志点之间主动等待，却不能据此测量

GPU 耗时。若要测 GPU 执行时间，应使用 GPU 时间戳查询并换算单位。本章只检验输出一致性和资源安全复用，不用文件保存耗时推断渲染性能。

练习：双槽版本的两张快照交替重复，但没有 validation error。把 `--inflight 2` 改回 1 以后，物体甚至完全不动。最应先查看哪里？

推理答案：先查看 `run_frames` 中用于构造矩阵的参数是否错误地来自 `frameIndex`。双槽索引只有 0、1，因此会反复得到两份矩阵；单槽索引始终为 0，因此会一直得到同一份矩阵。这组现象直接对应“资源槽编号被误当成动画帧号”，比随机改 barrier 更能解释证据。修复应使用从 0 递增的实际帧号计算位移，仍用槽编号选择 UBO、command buffer 和 fence，然后重跑两种在途数量，对照第 0、4、8 帧。

第 14 章 窗口接入设计（拓展）

前十三章的图片由 CPU 保存到文件；接入窗口后，最终结果需要交给呈现引擎，再由系统显示。呈现引擎对图像的使用不完全由图形命令缓冲区表示。因此，图形提交结束，不一定意味着它涉及的所有呈现资源已经可以复用。

本章只提供窗口接入的设计说明，不包含已实际运行验证的 GLFW 窗口程序，也没有 --lesson window 入口。下面会说明各个对象为什么需要，以及怎样管理获取、绘制、呈现和重建的关系；它不是一份可以直接复制粘贴的完整 Swapchain 实现。离屏工程中的 Shader、顶点与纹理资源、管线构建和绘制命令仍可复用，但需要另外创建 Surface、Swapchain 并管理其生命周期。

为什么选设备时还要检查 Surface?

`VkSurfaceKHR` 表示 `Vulkan` 与平台窗口系统之间的呈现接口；`Swapchain` 提供可获取、可呈现的 `Image`。应用通过 `vkGetSwapchainImagesKHR` 获取这些图像，再为它们创建 `ImageView`，而不像普通离屏目标那样自行给这些 `Image` 分配内存。应用负责销毁自己创建的 `View`；`Swapchain Image` 的生命周期由 `Swapchain` 管理。[57]

若选用 GLFW，应先初始化 GLFW，使用 GLFW_NO_API 创建不包含 OpenGL 上下文的窗口，通过 `glfwGetRequiredInstanceExtensions` 取得创建 surface 所需的实例扩展，再创建 instance，随后调用 `glfwCreateWindowSurface`。平台扩展列表不能写成只适用于某台 Linux 电脑的一组固定字符串。[58]

有了实际 surface，才对候选设备的队列家族调用 `vkGetPhysicalDeviceSurfaceSupportKHR`。本教材的接入设计要求同一个队列族中的一条 Queue 同时支持 Graphics 和针对该 Surface 的 Present，从而避免这里的跨队列族所有权转移。找不到这样的家族就报告本简化路线不支持；完整引擎可以另行实现不同家族的情况。设备还必须支持并启用 `VK_KHR_swapchain`，以及原有的 dynamic rendering 与 synchronization2 特性。支持 Vulkan1.3 并不等于所有窗口扩展已经启用。[57]

根据查询结果填写 Swapchain 的参数

首先查询 Surface 的能力、可用的 Format/ColorSpace 组合，以及 Present Mode。希望使用三张图像，并不意味着创建后必然恰好有三张：minImageCount 表示请求的最小图像数量，必须满足能力范围；创建后还应查询实际获得的数量。常见策略是请求至少比最小值多一张；仅当 maxImageCount 非零时才以它限制上界，零表示这里没有明确上限。[59]

`imageFormat` 与 `imageColorSpace` 应来自同一项支持结果。不能从两个不同候选项中随意拼接 Format 与 ColorSpace。`imageUsage` 描述以后对这些 image 的实际访问；若直接作为颜色附件绘制，只请求并核实所需的 `COLOR_ATTACHMENT` 用途。想沿用离屏截图，还需额外查询 `TRANSFER_SRC` 支持，不能因为自己创建的 image 支持复制，就假设所有 swapchain image 也支持。[60]

`imageExtent` 采用 surface 指定的固定 extent；当 currentExtent 返回表示未固定尺寸的特殊值时，才根据窗口的 framebuffer 像素尺寸，在允许范围内选择。窗口逻辑尺寸与实际像素尺寸可能

不同；最小化导致宽或高为零时，应继续处理窗口事件并等待恢复，而不是创建零尺寸 swapchain。普通单视图窗口的 `imageArrayLayers` 为 1，`preTransform` 可采用当前变换，`compositeAlpha` 必须选自支持位。入门可选标准要求支持的 FIFO 呈现模式，它按呈现队列与垂直消隐节奏处理请求。[59][60][61]

随后为每张 image 创建匹配的 view。dynamic rendering 的 `VkPipelineRenderingCreateInfo::pColorAttachmentFormats` 必须与实际颜色附件 format 兼容；离屏使用 RGBA，窗口附件采用 BGRA 时，不能直接把原管线当作自动适配。resize 后还应重新确定 viewport、scissor、renderArea，以及如有深度附件时的匹配尺寸。[62]

区分帧槽索引与交换链图像索引

Frame Index 由应用轮换，用来选当前可复用的帧槽；Image Index 由 vkAcquireNextImageKHR 返回，指定这一次获得的 Swapchain Image。两者分别指向“应用自己的帧资源”和“呈现系统提供的图像”，即使数字偶尔相等，也不能混用。

资源	由哪个索引选择	复用依据
UBO、descriptor set、command buffer、提交 fence	`frameIndex`	该槽的图形提交已经完成
acquire semaphore	`frameIndex`	先前等待它的图形提交已经完成
swapchain image、对应 view、记录的 layout	`imageIndex`	已获取该 image 并建立 acquire 等待
被 present 等待的 render-finished semaphore	`imageIndex`	同图像重新获取与等待，或相应 present fence

这里最容易错的是信号量的复用条件。图形提交的 fence 触发，说明提交完成，但不保证呈现引擎已经消费那次 render-finished semaphore。若只按两个帧槽分配两颗 present 等待信号量，再随着 `frameIndex` 重用，旧 present 可能仍在使用它。按实际 swapchain image 数量分配，并按 `imageIndex` 选择；当再次获取同一 image 且执行了它的 acquire 等待，就能建立复用先前对应 present semaphore 所需的顺序。仅拿到返回索引、还未等 acquire 信号，不是这个完成证明。[63]

一帧怎样从获取图像走到呈现？

先等待当前帧槽对应的提交 Fence，使该槽的命令与 UBO 可复用，然后获取 image。`vkAcquireNextImageKHR` 使用的是 binary semaphore；给它的 semaphore 必须未触发，且没有尚未完成的 signal 或 wait 操作。有限 timeout 还允许程序在暂时没有图像时继续处理事件；不能在没有保证 surface 取得进展的条件下无条件使用无限 timeout。[64]

返回 `VK_SUCCESS` 或 `VK_SUBOPTIMAL_KHR` 都意味着已成功获取一张 Swapchain Image。后者表示可以继续使用、但交换链已不再最合适：应记下重建请求，继续本帧的等待、绘制与 present，使获取信号被消费。若遇到 SUBOPTIMAL 就立即中断后续处理，下一次 acquire 可能重用仍为

signaled 或有操作在途的 semaphore。VK_ERROR_OUT_OF_DATE_KHR 则应转入重建流程，不继续使用未获取的 image；这一分支尚未提交本帧，提交 fence 应保持原来已触发的状态。[57][64]

现在用当前帧的矩阵记录命令，把本次 imageIndex 对应的 image 转换为颜色附件布局。对本设计直接绘制的路径，提交中的 acquire 等待 stage 选 COLOR_ATTACHMENT_OUTPUT；前置 image barrier 的源 stage 也取这一阶段、源 access 取 NONE，目标为 COLOR_ATTACHMENT_OUTPUT/COLOR_ATTACHMENT_WRITE，使布局转换与后续附件访问在 acquire 等待之后发生。若第一项实际操作改为 copy 或 blit，就要据此重新选择等待及屏障范围。[39]

绘制结束后，从颜色附件布局转换为 PRESENT_SRC_KHR：源是附件写入，目标 stage/access 可用 sync2 的 NONE/NONE，因为下一个消费者是呈现引擎。提交的 render-finished semaphore 采用 ALL_COMMANDS signal stage，覆盖本提交的绘制及末尾转换；present 等待该 semaphore 后，规范提供呈现读取所需的可见性。等待 stage 与信号 stage 是两个不同方向的范围，不能不加思考地复制同一个值。[65][66]

确认命令录制成功且确实要提交后，才 Reset 当前提交 Fence，调用 vkQueueSubmit2；提交成功再填写 VkPresentInfoKHR，其 pWaitSemaphores 指向这张 image 对应的 render-finished semaphore，pSwapchains 指向当前 swapchain，pImageIndices 指向刚取得的 imageIndex。对于单个 swapchain，swapchainCount 为 1；这一次 present 应呈现实际绘制的那张 image，而不是帧槽编号。[66]

重建与关闭还需要额外的完成证明

窗口大小变化回调最好只记录“需要重建”，实际销毁应在主循环中可证明安全的时刻执行。vkQueueWaitIdle、vkDeviceWaitIdle 能等待相应图形工作，但在基础 WSI 路线中，不能据此无条件断定呈现引擎已释放所有资源。呈现请求有独立的生命周期，不能仅凭前一帧图形提交的 Fence 触发，就假定可以销毁所有相关对象。[63]

若要把本设计实现成具有明确回收依据的窗口程序，可要求支持 VK_KHR_swapchain_maintenance1，或与之对应的 EXT 版本，并查询、启用 swapchainMaintenance1 特性。在 VkPresentInfoKHR::pNext 挂入 VkSwapchainPresentFenceInfoKHR，给每次 present 请求关联一个尚未触发、未被其他在途工作占用的 fence。对应 EXT 结构是等价别名，但实际启用的扩展与所用头文件必须一致。[67][68]

扩展依赖也应在创建 instance 以前规划。KHR 路径依赖实例扩展 VK_KHR_surface_maintenance1，EXT 路径对应 VK_EXT_surface_maintenance1；二者还要满足各自扩展页列出的依赖链，不能只向 Device 扩展列表加入一个名称。1.3 路线已满足所需的 1.1 核心能力，但不会自动启用这些 surface 扩展。[69][70]

Present Fence 可以提供规范定义的呈现资源安全回收条件，但不等于一定等到了画面真正显示出来。等待相应请求的 Fence 后，才可按其保证范围回收该请求使用的等待 Semaphore；旧 Swapchain 的历史呈现请求也都满足回收条件后，才能销毁其相关资源。与此同时，还需确认旧 View、深度 Image 等对象不再被图形提交引用。[68]

重建时重新查询能力，创建新 swapchain 与 view。将旧句柄传入 `oldSwapchain` 会使旧 Swapchain 进入 retired 状态，即使新建失败，也不能再从旧 Swapchain 获取新的图像。应保留待回收资源及其未完成请求记录，完成上述等待后再销毁。`vkQueuePresentKHR` 返回 out-of-date 一类呈现拒绝结果时，相关队列等待操作仍被视为已经入队；内存不足导致根本未入队的错误则不同，不要登记一个永远不会被触发的 present fence 后再无限等待它。[60][71]

如果设备不支持 Maintenance1，本书的离屏实验仍可正常学习。真正实现窗口程序时，则需要明确提示这种回收方案不可用，或者另行实现并验证一条合法的呈现生命周期策略。不能绕过扩展检查却继续使用 Present Fence 结构，也不能无条件用“已经调用 Queue Idle”代替呈现完成证明。

根据窗口故障现象检查设计

窗口初次集成时，先只清除为可识别的固定颜色，再接回三角形。清除也看不见时，记录 acquire 结果、`imageIndex`、被绑定的 view 和 present 结果，先证明获取、绘制和呈现的是同一张 Swapchain Image。只有 resize 后卡在 fence 等待，优先找“reset 之后提前返回”的分支；只有运行若干帧后出现 semaphore 复用错误，则沿表格检查 present 等待信号量的归属。这样的日志应包含实际帧号、帧槽索引和 image 索引，让三条不同的生命周期能对齐。

练习：两个帧槽，实际取得三张 swapchain image。某帧 `frameIndex=0`，acquire 返回 `imageIndex=2`。本帧应使用哪些数组元素？若 acquire 同时返回 `SUBOPTIMAL`，是否可以直接开始重建？

推理答案：UBO、command buffer、提交 fence 和 acquire semaphore 来自槽 0；颜色 ImageView 与 Render-Finished Semaphore 对应 Image Index 2。`SUBOPTIMAL` 已经成功获取图像，所以先完成消费 acquire 信号的提交，并把该图像交给 present，再根据重建标志处理后续流程。重建与资源释放还要等待相应图形提交及 present 资源回收条件。把 image2 传给 present、却渲染槽 0 对应的另一张 image，会显示错误内容；把 present semaphore 也按槽 0 轮换，则是另一种独立的生命周期错误。[63][64]

第 15 章 从报错到定位问题

学习图形 API 时，最难排查的情形往往不是崩溃，而是调用返回成功，画面却不符合预期。VK_SUCCESS 只说明对应 API 调用成功，并不保证顶点位于可见范围、纹理已有正确内容，或矩阵计算符合你的意图。排查时应依次问：API 使用是否合法？命令有没有正确执行？得到的数据是否符合设计？每个判断都需要可以观察的证据。

本章先保留正确的基线输出，再通过明确标注的故障推演学习如何阅读 VUID。以下 VUID 来自参考规范，并不表示本次实验实际产生了这些错误；验证层版本变化后，消息的完整文字也可能不同。

先保留一份已知正确的输出

从实验包根目录执行以下步骤，并保留日志。`--validation` 会要求存在 VK_LAYER_KHRONOS_validation，同时显式启用 synchronization validation；若所需层或扩展不可用，程序报告缺失，而不是悄悄忽略这个选项。

```
mkdir -p out
./build/vulkan_lessons --lesson probe --validation > out/probe.log 2>&1
./build/vulkan_lessons --lesson clear --output out/clear.ppm --validation >
  out/clear.log 2>&1
./build/vulkan_lessons --lesson triangle --output out/triangle.ppm --validation >
  out/triangle.log 2>&1
python3 scripts/ppm_tools.py out/clear.ppm --expect clear --png out/clear.png
python3 scripts/ppm_tools.py out/triangle.ppm --expect triangle --png out/triangle.png
```

clear 的目标是 512×512，每个 RGB 像素为 `(6,9,14)`，它来自 `src/render.cpp` 中明确设置的清除颜色。PPM 只保存 RGB，不做额外的 sRGB 转换。triangle 应在同样的背景上出现红、绿、蓝角之间连续插值的三角形。先确认 clear，再确认 triangle，意味着先把设备、附件与读回路径跑通，再把问题缩小到绘制产生像素的过程。

如果不想依赖图像查看器，可以直接用 Python 标准库检查工程生成的 P6 文件。把下面代码保存到 lab 目录中的 check_pixels.py，再在同一目录运行 `python3 check_pixels.py`；Windows 使用 `py -3 check_pixels.py`。下列代码只针对本工程固定的、无注释的三行 PPM 头，不声称是完整的通用图片解码器。

```
from pathlib import Path
from collections import Counter

for name in ("clear", "triangle"):
    raw = Path("out", name + ".ppm").read_bytes()
    magic, dimensions, maximum, pixels = raw.split(b"\n", 3)
    width, height = map(int, dimensions.split())
    assert magic == b"P6" and maximum == b"255"
    assert (width, height) == (512, 512)
    assert len(pixels) == width * height * 3
    colors = Counter(zip(pixels[0::3], pixels[1::3], pixels[2::3]))
    center = 3 * ((height // 2) * width + width // 2)
    print(name, "RGB bytes:", len(pixels),
          "unique colors:", len(colors),
          "center:", tuple(pixels[center:center + 3]))
```

```
    if name == "clear":
        assert colors == {(6, 9, 14): width * height}
```

两张 512×512 的 PPM 图像，其 RGB 像素区都应占 786432 字节；clear 只有一种颜色，triangle 则包含多种颜色。文件长度不对时，先查复制范围、行布局和 PPM 输出。即使数据长度正确，也不能直接证明绘制正确，还需检查实际图案。

配套 `evidence/clear.ppm` 中 262144 个像素全部为上述背景色；`evidence/triangle.ppm` 中有 68658 个像素与背景色不同，中心像素为 `(118,69,68)`。这是本次实现与环境的实际记录，可与 `evidence/summary.json` 对照；跨实现检查三角形时仍应结合覆盖范围和插值颜色的合理性，不能仅凭某个边缘像素的细微差异就断定同步错误。

让日志说明这次实际执行了什么

`probe.log` 应列出 loader 版本、设备、队列能力、所需特性和格式能力。每次正式实验也会报告所选设备，并出现 `Enabled: dynamicRendering=1, synchronization2=1` 这样的启用信息。设备名称有助于对比不同机器的结果，但不要把“同型号显卡”当作相同环境；驱动、loader、验证层版本都应随问题记录一起保留。

工程在 `src/context.cpp` 创建 instance 时，把 `VkValidationFeaturesEXT` 加到创建链中，其启用数组包含 `VK_VALIDATION_FEATURE_ENABLE_SYNCHRONIZATION_VALIDATION_EXT`。该特性专门帮助发现绘制、复制等访问间缺失或错误的同步关系；只是加载验证层，并不能假定所有可选验证功能都已启用。[72][73]

debug callback 输出消息严重程度、`pMessageIdName` 和 `pMessage`。前者通常给出可搜索的 VUID 或其他消息标识，后者包含具体对象与违反的条件。回调数据还提供相关对象、队列标签与命令标签信息；随着程序扩大，给关键资源命名会比记住一串十六进制句柄更容易追踪。[74]

排查时先读第一条有意义的 Error，不要只看末尾的一串后续错误。创建失败后出现的空句柄错误，可能只是同一问题的连锁反应。先找到报错对应的 Vulkan 调用，再追溯它使用的资源和参数。许多 vkCmd* 录制函数不返回 VkResult，不能给 vkCmdDraw “补一个返回值检查”来替代验证回调。日志中的 errors=0、warnings=0 只是当前验证结果，不保证程序的全部逻辑都正确。

沿着具体 VUID 追溯错误来源

下面用格式不匹配来推演一次报错：颜色目标仍是 `VK_FORMAT_R8G8B8A8_UNORM`，却把 `create_graphics_pipeline` 中 dynamic rendering 的颜色格式声明改成 `VK_FORMAT_B8G8R8A8_UNORM`。在本教材未启用额外 unused-attachment 特性的条件下，绘制会违反 `VUID-vkCmdDraw-dynamicRenderingUnusedAttachments-08910`。这条规则比较的是正在使用的颜色附件 view 格式与管线创建时声明的对应格式。[62]

定位时先看 `src/render.cpp:begin_rendering` 传给 `pColorAttachments` 的 view，再找该 view 的 format，最后对照 `create_graphics_pipeline` 的 `VkPipelineRenderingCreateInfo`。错误虽然在 Draw 时暴露，修复位置却是创建与连接这两端格式的地方。补一个 barrier 不会把 RGBA 变成 BGRA；修改片元输出顺序也不能消除管线与附件格式不匹配。修好后重新构建，重跑 triangle，并用保留的背景及三角形输出验证结果。

第二个推演是 `VUID-vkResetCommandBuffer-commandBuffer-00045`。它要求 command buffer 不处于 pending 状态。先找报错句柄属于哪个帧槽，再找该槽最近一次成功提交所用的 fence，最后看 reset 之前是否真正等待到了这个 fence。若等的是另一槽，或等待返回 timeout 却继续执行，代码表面上虽然调用了 Wait，实际等待的却不是正确的提交。修复对象是资源归属和返回分支。[55]

第三个推演是 `VUID-vkQueueSubmit2-fence-04894`，它指出提交使用的 fence 仍为 signaled。先确认上一轮确实完成，再把 reset 放到本轮确定要提交的位置。不要为了消除这个 VUID 而把 reset 机械地提到帧循环顶端；第 14 章的 out-of-date 早退分支会因此制造永远没有后续 signal 的 fence。这三例都要求把一条规则还原为具体对象的时间线，而不是记忆“遇到错误就加等待”。[31][56]

没有 VUID 时，沿着绘制路径排查

假设 clear 正确、triangle 只有背景。先核实运行参数确实选择 triangle，并检查新 shader 是否已经被构建为当前加载的 SPIR-V。然后确认 `run_triangle` 记录了 Draw、绑定了预期管线，viewport 与 scissor 不是零尺寸，三个裁剪坐标也没有全部落到不可见范围。此时只改片元输出为固定亮色，是一个很小的定位实验：若仍完全看不见，需要继续追查几何覆盖和绘制状态；若亮色三角形出现，则位置路径成立，应回头看颜色接口与计算。每次实验后恢复基线，才知道下一次变化来自哪里。

vertex 模式中，首个顶点正常、后两个越来越偏离，常与 stride 错误吻合：越向后读取，累计偏移越大。利用“绑定偏移 + 顶点编号 × Stride + Attribute Offset”的公式算出第二个顶点位置实际读的是哪几个字节，再对照 C++数组。这个计算比反复试不同的 shader 更直接。画面正确也仍要保留断言，防止以后修改结构体导致 `sizeof(Vertex)` 和字段偏移发生变化。

如果 Camera 实验正常，而 Texture 实验只显示单一颜色，先临时输出 UV 渐变，确认顶点 UV 和阶段接口是否抵达片元着色器。UV 正确后，再沿 `run_texture` 查 CPU 像素、staging 上传、目标子资源、最终 layout、image view、sampler 和 descriptor。若 mip0 正确而高 LOD 错误，把 `--lod 0` 与较大 LOD 的输出并排观察，再检查那些级别是否真正生成、view 是否覆盖、sampler 是否允许访问，以及最后转换是否包含相应范围。不要仅凭“纹理是黑色”就判定是哪一个对象出错。

出现同步报错时，先找出前后两次访问

synchronization validation 的消息可能包含 RAW、WAR 或 WAW。先从消息中找出资源和范围，再分别写下“前一个命令、stage/access、子资源”与“后一个命令、stage/access、子资源”。例如先向附件写入颜色、再通过 Copy 读取，是本章前面已经复现的截图路径；如果 barrier 只覆盖 layer0，而实际 copy 读取 layer2，扩大 stage 不会解决范围缺失。若报的是 layout 转换与旧写入冲突，则把转换本身当作一次可能访问 image 的操作分析。[52][39]

同步验证也有边界：它无法跟踪任意 CPU 映射内存的读写，因此 CPU 提前覆盖 GPU 正在使用的 UBO，未必会有相应报错；Shader Descriptor 精确访问和内存别名等方面也存在已知限制。没有 Error 时仍应检查资源的使用时间线；遇到 Warning 与 Error，则分别阅读其含义，判断是性能、可移植性提示，还是有效使用约束被违反。[75]

练习：程序在 GPU 提交后立刻 invalidate 并保存 PPM，图像偶尔像上一帧。开发者添加了 HOST_COHERENT 内存要求，认为这样就不必等 fence。这个修复成立吗？

推理答案：不成立。观测提示读取与设备写入的完成顺序需要核查，但不能仅凭“像上一帧”确定唯一原因。先确认读回 buffer 对应哪次提交，再确认 copy 后的 COPY/TRANSFER_WRITE 到 HOST/HOST_READ 依赖，随后等待覆盖该 copy 的 fence 成功；非 coherent 路径在其后 invalidate。coherent 内存只能省略最后一项显式缓存维护，不能让尚未完成的 copy 提前完成。最后再检查延后保存时的帧号标签，排除数据正确但文件命名错误这一种解释。[33][19]

第 16 章 综合练习与后续学习

到这里，你已经能够沿着同一条路径解释 GPU 绘制：C++ 准备数据，Buffer 与 Image 提供存储，Shader 进行计算，Pipeline 规定绘制规则，Descriptor 连接资源，CommandBuffer 记录命令，同步机制约束前后访问，最后由 CPU 检查输出。以下练习要求你从需求或故障现象推导应当修改哪些环节，而不是机械记忆创建结构体的字段。

每题先独立写出预测，再查看答案。如果需要运行，请先保存原始输出作为对照，每轮只改动与当前假设直接相关的部分，随后执行 cmake --build build --config Debug --parallel。标注为“迁移设计”的题目并没有对应的现成 CLI 模式，需在已经跑通的工程上自行扩展。

从画面区分视口变换与裁剪

题目：运行 triangle 实验。甲将 Viewport 的宽度从 512 改为 256，保持 Scissor 为 512×512；乙保持 Viewport 为 512×512，只将 Scissor 的宽度改为 256。两种修改会分别怎样影响三角形？如果画面中的三角形变窄，能否断定顶点坐标发生了变化?

推理答案：甲改变了 NDC 到帧缓冲坐标的映射，三角形被映射到左半幅，水平方向随之缩窄。乙没有改变坐标映射，只是禁止右半幅区域写入，因此三角形在右侧被裁掉，左半部分不会因此压缩。看到三角形变窄，不能直接推断顶点数据改变；检查 Shader 输出和 Viewport 设置，才能确定变化发生在哪一步。

用字节算式定位顶点输入

题目：假设每条 Vertex 记录占 32 字节，颜色字段在记录中的偏移是 12 字节。绑定顶点 Buffer 时，起始偏移为 64 字节；现在要读取相对于这一起点编号为 2 的顶点（从 0 开始计数）。它的颜色数据从整个 Buffer 的第几个字节开始？如果误将 Stride 设为 24 字节，又会读到哪里?

推理答案：正确偏移是 64 + 2 × 32 + 12 = 140 字节。若把 Stride 错设为 24，则读取偏移变为 64 + 2 × 24 + 12 = 124 字节，比正确位置提前 16 字节。GPU 不会根据 C++ 成员名自动寻找 color 字段；应使用 sizeof(Vertex) 设置绑定的 Stride，用 offsetof(Vertex, color) 设置属性偏移，并确认 Format 是三个 float。修改 Shader 的颜色计算，并不能纠正已经读错的输入字节。

为什么只改深度清除值就让物体消失

题目：depth 实验使用普通深度方向、LESS 比较，并开启深度测试和写入。现在仅将深度清除值从 1 改为 0，其他片元深度仍在合法的 0～1 范围内，结果可能只剩背景。为什么这种现象不一定触发 Validation Error?

推理答案：LESS 要求新片元的深度严格小于深度附件中已有的值。清除为 1 时，较小的深度可以通过；清除为 0 后，合法范围内的片元深度都不小于 0，连等于 0 也无法通过比较。这里的参数本身可以合法，只是组合后的深度测试行为不符合预期。因此应根据深度方向恢复合适的清除值和比较操作，而不是仅依据是否有验证错误来判断。

创建七级 Mip 是否等于生成七级内容?

题目：一个 64×64 的纹理创建了 7 个 Mip Level，并把这 7 级全部转换为 Shader 只读布局，却只向 Mip 0 上传了棋盘图案。现在修改采样器和 Shader，明确要求读取 Mip 6。能否据此得到正确的棋盘平均色？如果不能，还缺少哪些步骤？

推理答案：创建 Mip Level 只建立存储空间，布局转换只改变子资源的使用状态并建立相应依赖，都不会自动生成缩小后的纹素。还必须实际生成或上传 Mip 1～6，并检查图像格式是否支持所选操作。若采用逐级 Blit，需要设置每一级的正确尺寸，并在前一级写入与后续读取之间建立依赖；最后还要保证 ImageView 和 Sampler 允许访问相应级别。没有被正确写入的数据，即使显示得像平均色，也不能证明结果有效。[39][26]

为数组纹理写出精确的子资源范围

题目：一张 Image 含有 4 个 Mip Level、3 个 Array Layer。程序刚刚更新了 Layer 2 的 Mip 1、2、3，接下来要让 Shader 采样这些数据，但现有 Barrier 的范围为 baseMipLevel=0、levelCount=1、baseArrayLayer=0、layerCount=1。仅将 Stage 改成 ALL_COMMANDS 能修复问题吗？请写出正确的四个范围字段。

推理答案：不能。原来的范围仅覆盖 Layer 0 的 Mip 0，与本次更新的子资源没有交集。正确范围应为 baseMipLevel=1、levelCount=3、baseArrayLayer=2、layerCount=1；对于普通颜色纹理，Aspect 为 COLOR。还必须按实际写入操作选择源 Stage/Access，按采样所在的 Shader 阶段选择目标 Stage/Access，并使用这些子资源真正的旧 Layout。范围正确只是建立完整依赖的必要条件之一。[26]

两个 Fence 能否保护同一个 UBO?

题目：开发者准备了两个帧槽，每个槽都有自己的 CommandBuffer 和 Fence，却让两份 Descriptor Set 都指向同一个 64 字节的相机 UBO。CPU 提交第 0 帧后，立即将第 1 帧矩阵写进同一段 UBO 内存，再提交第 1 帧。即使内存具有 HOST_COHERENT 属性，而且验证层没有报错，这套设计是否安全?

推理答案：不成立。第 0 帧 GPU 读取可能尚未结束，CPU 就覆盖了同一范围；有两颗 fence 只意味着可以分别观察两个提交，不意味着它们保护的数据已经分成两份。修复可以为每槽创建自己的 UBO，并令其 descriptor 指向正确对象；也可以用满足对齐与非一致性范围约束的大 buffer 切片。每次只写当前已安全回收的槽。HOST_COHERENT 不提供 GPU 完成等待，验证也不跟踪任意主机写入，因而两项观察都不能替代这个推理。[19][75]

结果正确，为什么文件的帧号仍然会错?

题目：单帧槽和双帧槽的最后一张图片看起来相同，但双帧槽输出中，标为“第 2 帧”的文件实际上保存着第 0 帧的画面。程序在准备第 2 帧时等待槽 0 的旧 Fence，然后读取槽 0 的读回 Buffer，却用当前循环变量 2 作为文件名。应该先检查矩阵、Barrier，还是保存逻辑？如何验证你的判断?

推理答案：槽 0 的旧提交对应第 0 帧，因此等待它完成后，读回 Buffer 中保存第 0 帧的结果是正常的。错误发生在命名阶段：程序使用了“正在准备的帧号”，而不是“这个结果实际所属的帧号”。每个槽都应记录上一次提交的实际帧号，完成读回后用该编号保存文件，然后再更新这个槽的提交信息。可以对照单帧槽输出和各槽的提交日志验证。

把下面代码保存到 lab 目录中的 compare_frames.py，再在同一目录运行 `python3 compare_frames.py`；Windows 使用 `py -3 compare_frames.py`。

```
from pathlib import Path

for frame in (0, 4, 8):
    one = Path(f"out/one_{frame:03d}.ppm").read_bytes()
    two = Path(f"out/two_{frame:03d}.ppm").read_bytes()
    print(f"frame {frame}:", "identical" if one == two else "different")
```

这个比较服务于本实验的确定输入与同设备对照，不把不同 GPU 在所有渲染场景中的逐字节一致作为 Vulkan 的普遍承诺。若发现差异，应继续保留两张文件与帧槽日志，区分输入计算、同步和标号问题。

从三项日志辨认呈现对象是否匹配

题目：窗口接入设计中的日志显示 frame=100、frameIndex=0、imageIndex=2，绘制使用了 view2，但随后传给 VkPresentInfoKHR::pImageIndices 的却是 0。开发者认为图形提交的 Fence 已触发，Present 就会自动选择刚绘制完成的图像。这个判断错在哪里?

推理答案：Present 使用应用明确传入的 Image Index，没有“挑最新画面”的隐含规则。应传入本次 acquire 得到并实际绘制的 2；0 是否仍是一个已获取图像、其中有什么内容，是另一件事。提交 fence 只说明对应图形工作完成，不会修正 present 参数。进一步核对 render-finished semaphore 也按 image2 选择，而不是随帧槽 0 轮换，才能同时修正对象选择与信号量复用两条独立关系。[63][66]

将离屏输出尺寸改为 1024×768

题目：这是一道工程修改题。假设只把颜色 Image 的尺寸从 512×512 改为 1024×768，而 Viewport、Copy 范围和读回 Buffer 仍使用原来的 512×512 参数，能否得到完整的新尺寸图片？请列出需要一起修改的配置，并给出一个可计算的验收指标。

推理答案：不能。改变颜色 Image 的尺寸，不会自动更新依赖它的其他配置。应把输出尺寸集中为一组参数，并据此更新颜色 Image、可选的深度 Image、renderArea、Viewport、Scissor、复制范围、读回 Buffer 和文件头。本工程使用 RGBA8 时，1024×768 的 GPU 像素数据为 3145728 字

节，最终 RGB 数据为 2359296 字节。若涉及透视投影，还要同步更新宽高比。验收时既检查文件尺寸，也检查画面比例是否正确。

添加一遍“将绘制结果作为纹理”的处理

题目：这是一道两遍绘制的迁移设计题，没有对应的现成 CLI 模式。第一遍绘制原有场景；第二遍绘制覆盖整个输出区域的图元，并在 Fragment Shader 中采样第一遍的颜色，计算 1.0 - rgb。请说明：需要新增哪些资源绑定？两次绘制之间需要什么同步与布局转换？最终应该从哪个目标读回?

推理答案：第一遍颜色 image 必须同时支持并声明颜色附件与采样用途，为它建立可采样的 view、sampler 和 descriptor。第二遍使用另一个颜色目标，避免在这条基础路线中一边写同一附件一边把它作为普通纹理读取。第一遍结束后，把源 image 从附件布局转为采样只读布局，源为 `COLOR_ATTACHMENT_OUTPUT/COLOR_ATTACHMENT_WRITE`，目标为第二遍的 `FRAGMENT_SHADER/SHADER_SAMPLED_READ`。第二遍的 descriptor 指向第一遍结果，最终 Copy 从第二遍绘制的颜色目标读取，并沿第 12 章的主机读回链保存。[39][26]

可以先检查背景，因为它的颜色容易预测：第一遍背景为 (6,9,14)，沿本书没有额外 sRGB 转换的 UNORM8 路径做反色，预期约为 (249,246,241)。之后再检查三角形内部。如果整张输出只有单一颜色，先确认第二遍的 UV 和 Descriptor 确实读到了第一遍 Image，再检查反色计算。不能把“第二遍存在 Draw 命令”当成“读写依赖一定正确”的证据。

怎样判断自己已经掌握了这条主线?

回到原始工程，完整跑一遍 probe、clear、triangle、vertex、camera、texture、depth、mips、layers 与 frames。你应能为每课指出输入在哪里准备、哪条命令真正消费输入、哪一段资源发生写入、谁负责下一次读取、哪个输出能证明预期。如果某一步只能解释为“教程就是这样写的”，就沿对应函数向前追一层，直到能说出一个具体对象和访问。

继续学习时，先选择能接在现有链条上的一个问题。想画真实模型，可以从索引 buffer、更多顶点属性和法线开始；想让场景更丰富，可以从材质参数、灯光计算和多物体 descriptor 组织开始；想减少 CPU 等待，可以沿帧槽继续学习子分配、动态偏移、时间戳与批量提交；想接入窗口，则先把第 14 章的获取、呈现、resize 与回收状态真正实现并验证。

官方资料也可以按问题类型分工查阅。Khronos Vulkan Guide 的同步示例适合检查一条已经写出的生产者到消费者关系；API 参考页适合核对某个字段的有效条件和 VUID；规范章节适合在多个对象、多个队列或主机与设备交接时确认保证范围。Khronos Vulkan Samples 提供更大的完整实现，但先找到与你当前问题相同的那条资源路径，再扩展到示例的框架层，阅读会更有目标。该项目有自己的构建要求，当前文档要求至少 C++20；它是后续参考工程，不能把它的编译要求与本书 C++17 实验混为一谈。[39][76][77]

你不必等到能默写所有 Vulkan 结构体才开始自己的项目。更重要的是保留一个持续可运行的最小版本：增加资源用途时能说清数据来源和访问关系；复用资源时能说明前次工作何时完成；最后用可观察的结果检查预测。即使系统逐渐复杂，也仍然可以拆回这些具体问题。

实验复现与验证记录

配套代码位于实验包的 lab 目录。各章的可运行主线共享同一个 vulkan_lessons 程序，完整入口位于 src/lessons.cpp。教材为了突出概念，只截取相关代码；配套工程仍包含创建、提交、错误处理与资源释放的完整路径。第一次运行前请先阅读 README.md。

一次运行全部主线对照

配置并构建后，在 lab 目录运行下面命令。脚本将要求验证层和同步验证，运行各个案例、解析 PPM、保存 PNG 和独立日志，并写出汇总结果。

```
python3 scripts/run_all.py --exe build/vulkan_lessons --out out/all
```

Windows 使用 `py -3` 和 `build/vulkan_lessons.exe`。你自己运行产生的结果保存在 out/all；实验包中的 evidence 是本次编写教材时保留的原始验证结果。不要用 evidence 中已有图片的存在来代替一次新的运行成功。

本次实际运行结果

2026 年 10 月 19 日，在 Linux x86_64 环境中完成了 21 个 Vulkan 主线案例和一个 CPU 图片参考程序。Vulkan 使用 llvmpipe 软件实现，报告 API 1.4.318；应用只请求本书的 Vulkan 1.3 核心路线。Mesa 包为 25.2.8，LLVM 为 20.1.2；使用 GCC 13.3、CMake 3.28.3、glslangValidator 15.1.0、Vulkan 头文件和验证层 1.3.275。八个 Shader 均针对 Vulkan 1.3 构建。完整版本信息与构建日志保存在 evidence 中。

21 个主线案例均正常退出，各自 stderr 为空，验证层统计为 0 个错误、0 个警告，同步验证已显式启用。CPU 参考程序的像素与 Vulkan clear 输出一致。下表只列最能说明每项改动效果的检查；完整命令和统计在 evidence/summary.json 中。

对照	本次观察	说明什么
CPU 与 clear	262144 个像素均为 RGB 6 9 14	图像生成和读回回到同一文件布局
triangle 与 vertex	RGB 逐字节一致，68658 个非背景像素	顶点改由 Buffer 输入后形状与颜色保持
camera shift 0 与 0.35	前景质心右移 89.5 像素	与几何预测 89.6 像素一致到栅格量化
texture 与 mip 0 与 layer 0	RGB 逐字节一致	不同接口路径访问相同图案
mip 0 与 mip 6	棋盘变为均匀 RGB 235 145 140	实际生成并选择了 1×1 的最高 mip
depth 与关闭 depth	中心从红色变为蓝色	后绘制的远表面被深度测试拒绝
单槽与双槽	第 0 4 8 帧的 RGB 分别一致	在改变复用调度后仍保持同样输出

第 8 章的透视扩展另外在隔离副本中构建和运行了两次，距离 2 与 4 对应 244×244 和 122×122 的前景，两次也通过验证。它们是教材中的附加跟做练习，不计入主线的 21 个案例；记录在 perspective_summary.json，完整扩展函数在 extras/perspective_demo.hpp。

怎样理解本次验证的范围

本次检查证明配套源码、Shader、命令参数与记录的输出在测试环境中相互对应，但不能据此推断独立显卡性能或跨平台兼容性。Windows、实体 GPU、非一致性内存分支，以及第 14 章的窗口获取与呈现流程，均未在本次测试环境中实际运行；应在目标机器上分别验证。

不同设备在光栅化边缘和浮点数转换等细节上可能存在允许范围内的差异。因此，本书主要检查图片范围、颜色区域和同一环境中的对照结果，不把某张图片的固定哈希值当作所有设备都必须满足的标准。遇到失败，应保留验证信息并检查前后访问关系，而不是为了“看起来通过”直接关掉检查。

练习后的恢复方法

每次只修改与当前练习直接相关的参数或代码，并把输出另存为新文件。实验结束后恢复原值，再运行相关基线。例如改变 clear 颜色后恢复 6 9 14，改变深度比较后恢复 LESS 和清除值 1，透视练习后恢复 run_camera 的 make_camera_matrix 调用。书中后续对照与验证脚本依赖这份共同起点。

实验包不包含预编译的显卡驱动、SDK 或当前机器的 build 目录。请在自己的环境构建，以便 Shader 路径、编译配置和运行实现相互匹配。

一手资料索引

接口与有效使用条件以 Khronos 规范、API 参考页和官方指南为主；环境、构建和 PPM 文件格式使用各自维护方资料。核对日期为 2026 年 10 月 19 日。书中的数值推导和程序输出另由配套实验给出。

[1] Netpbm PPM Format Specification
https://netpbm.sourceforge.net/doc/ppm.html

[2] Khronos Vulkan Specification Fundamentals
https://docs.vulkan.org/spec/latest/chapters/fundamentals.html

[3] LunarG Getting Started with the Windows Vulkan SDK
https://vulkan.lunarg.com/doc/view/latest/windows/getting_started.html

[4] LunarG Getting Started with the Linux Tarball Vulkan SDK
https://vulkan.lunarg.com/doc/view/latest/linux/getting_started.html

[5] CMake FindVulkan
https://cmake.org/cmake/help/latest/module/FindVulkan.html

[6] Khronos Vulkan Guide Validation Overview
https://docs.vulkan.org/guide/latest/validation_overview.html

[7] Khronos vkCreateInstance
https://docs.vulkan.org/refpages/latest/refpages/source/vkCreateInstance.html

[8] Khronos vkCreateDevice
https://docs.vulkan.org/refpages/latest/refpages/source/vkCreateDevice.html

[9] Khronos VkDeviceQueueCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkDeviceQueueCreateInfo.html

[10] Khronos Vulkan Guide Enabling Features
https://docs.vulkan.org/guide/latest/enabling_features.html

[11] Khronos VkPhysicalDeviceVulkan13Features
https://docs.vulkan.org/refpages/latest/refpages/source/VkPhysicalDeviceVulkan13Features.html

[12] Khronos VkFormat
https://docs.vulkan.org/refpages/latest/refpages/source/VkFormat.html

[13] Khronos VkImageCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkImageCreateInfo.html

[14] Khronos VkBufferCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkBufferCreateInfo.html

[15] Khronos vkBindImageMemory
https://docs.vulkan.org/refpages/latest/refpages/source/vkBindImageMemory.html

[16] Khronos VkMemoryRequirements
https://docs.vulkan.org/refpages/latest/refpages/source/VkMemoryRequirements.html

[17] Khronos VkMemoryDedicatedAllocateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkMemoryDedicatedAllocateInfo.html

[18] Khronos Vulkan Guide Memory Allocation
https://docs.vulkan.org/guide/latest/memory_allocation.html

[19] Khronos VkMemoryPropertyFlagBits
https://docs.vulkan.org/refpages/latest/refpages/source/VkMemoryPropertyFlagBits.html

[20] Khronos VkImageViewCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkImageViewCreateInfo.html

[21] Khronos vkMapMemory
https://docs.vulkan.org/refpages/latest/refpages/source/vkMapMemory.html

[22] Khronos VkRenderingAttachmentInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkRenderingAttachmentInfo.html

[23] Khronos VkCommandBuffer
https://docs.vulkan.org/refpages/latest/refpages/source/VkCommandBuffer.html

[24] Khronos VkCommandPoolCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkCommandPoolCreateInfo.html

[25] Khronos Vulkan Specification Command Buffers
https://docs.vulkan.org/spec/latest/chapters/cmdbuffers.html

[26] Khronos VkImageMemoryBarrier2
https://docs.vulkan.org/refpages/latest/refpages/source/VkImageMemoryBarrier2.html

[27] Khronos VkRenderingInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkRenderingInfo.html

[28] Khronos VkBufferImageCopy
https://docs.vulkan.org/refpages/latest/refpages/source/VkBufferImageCopy.html

[29] Khronos vkCmdCopyImageToBuffer
https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdCopyImageToBuffer.html

[30] Khronos VkBufferMemoryBarrier2
https://docs.vulkan.org/refpages/latest/refpages/source/VkBufferMemoryBarrier2.html

[31] Khronos vkQueueSubmit2
https://docs.vulkan.org/refpages/latest/refpages/source/vkQueueSubmit2.html

[32] Khronos vkWaitForFences
https://docs.vulkan.org/refpages/latest/refpages/source/vkWaitForFences.html

[33] Khronos vkInvalidateMappedMemoryRanges
https://docs.vulkan.org/refpages/latest/refpages/source/vkInvalidateMappedMemoryRanges.html

[34] Khronos VkMappedMemoryRange
https://docs.vulkan.org/refpages/latest/refpages/source/VkMappedMemoryRange.html

[35] Khronos Vulkan Specification — Shader Interfaces
https://docs.vulkan.org/spec/latest/chapters/interfaces.html

[36] Khronos Vulkan Specification — Fixed-Function Vertex Processing
https://docs.vulkan.org/spec/latest/chapters/fxvertex.html

[37] Khronos Vulkan Specification — Fixed-Function Vertex Post-Processing
https://docs.vulkan.org/spec/latest/chapters/vertexpostproc.html

[38] Khronos Vulkan Specification — Rasterization
https://docs.vulkan.org/spec/latest/chapters/primsrast.html

[39] Khronos Vulkan Guide — Synchronization Examples
https://docs.vulkan.org/guide/latest/synchronization_examples.html

[40] Khronos GLSL Specification — Variables and Types
https://docs.vulkan.org/glsl/latest/chapters/variables.html

[41] Khronos Vulkan Guide — Shader Memory Layout
https://docs.vulkan.org/guide/latest/shader_memory_layout.html

[42] Khronos Vulkan Specification — Descriptor Sets
https://docs.vulkan.org/spec/latest/chapters/descriptorsets.html

[43] Khronos Vulkan API Reference — vkCmdBindDescriptorSets
https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdBindDescriptorSets.html

[44] Khronos Vulkan Specification — Sampling
https://docs.vulkan.org/spec/latest/chapters/textures.html

[45] Khronos Vulkan API Reference — VkSamplerCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkSamplerCreateInfo.html

[46] Khronos Vulkan API Reference — vkCmdCopyBufferToImage
https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdCopyBufferToImage.html

[47] Khronos Vulkan Specification — Images
https://docs.vulkan.org/spec/latest/chapters/images.html

[48] Khronos Vulkan API Reference — vkCmdBlitImage
https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdBlitImage.html

[49] Khronos Vulkan API Reference — VkImageSubresourceRange
https://docs.vulkan.org/refpages/latest/refpages/source/VkImageSubresourceRange.html

[50] Khronos Vulkan Guide — Depth
https://docs.vulkan.org/guide/latest/depth.html

[51] Khronos Vulkan API Reference — VkPipelineDepthStencilStateCreateInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkPipelineDepthStencilStateCreateInfo.html

[52] Khronos. Vulkan Specification — Synchronization and Cache Control
https://docs.vulkan.org/spec/latest/chapters/synchronization.html

[53] Khronos. vkFlushMappedMemoryRanges
https://docs.vulkan.org/refpages/latest/refpages/source/vkFlushMappedMemoryRanges.html

[54] Khronos. vkDestroyBuffer
https://docs.vulkan.org/refpages/latest/refpages/source/vkDestroyBuffer.html

[55] Khronos. vkResetCommandBuffer
https://docs.vulkan.org/refpages/latest/refpages/source/vkResetCommandBuffer.html

[56] Khronos. vkResetFences
https://docs.vulkan.org/refpages/latest/refpages/source/vkResetFences.html

[57] Khronos. Vulkan Specification — Window System Integration
https://docs.vulkan.org/spec/latest/chapters/VK_KHR_surface/wsi.html

[58] GLFW. Vulkan Guide
https://www.glfw.org/docs/latest/vulkan_guide.html

[59] Khronos. VkSurfaceCapabilitiesKHR
https://docs.vulkan.org/refpages/latest/refpages/source/VkSurfaceCapabilitiesKHR.html

[60] Khronos. VkSwapchainCreateInfoKHR
https://docs.vulkan.org/refpages/latest/refpages/source/VkSwapchainCreateInfoKHR.html

[61] Khronos. VkPresentModeKHR
https://docs.vulkan.org/refpages/latest/refpages/source/VkPresentModeKHR.html

[62] Khronos. vkCmdDraw — Dynamic rendering valid usage
https://docs.vulkan.org/refpages/latest/refpages/source/vkCmdDraw.html

[63] Khronos. Vulkan Guide — Swapchain Semaphore Reuse
https://docs.vulkan.org/guide/latest/swapchain_semaphore_reuse.html

[64] Khronos. vkAcquireNextImageKHR
https://docs.vulkan.org/refpages/latest/refpages/source/vkAcquireNextImageKHR.html

[65] Khronos. VkSemaphoreSubmitInfo
https://docs.vulkan.org/refpages/latest/refpages/source/VkSemaphoreSubmitInfo.html

[66] Khronos. VkPresentInfoKHR
https://docs.vulkan.org/refpages/latest/refpages/source/VkPresentInfoKHR.html

[67] Khronos. VkPhysicalDeviceSwapchainMaintenance1FeaturesKHR

https://docs.vulkan.org/refpages/latest/refpages/source/VkPhysicalDeviceSwapchainMaintenance1FeaturesKHR.html

[68] Khronos. VkSwapchainPresentFenceInfoKHR

https://docs.vulkan.org/refpages/latest/refpages/source/VkSwapchainPresentFenceInfoKHR.html

[69] Khronos. VK_KHR_swapchain_maintenance1 — Extension and Version Dependencies

https://docs.vulkan.org/refpages/latest/refpages/source/VK_KHR_swapchain_maintenance1.html

[70] Khronos. VK_EXT_swapchain_maintenance1 — Extension and Version Dependencies

https://docs.vulkan.org/refpages/latest/refpages/source/VK_EXT_swapchain_maintenance1.html

[71] Khronos. vkQueuePresentKHR

https://docs.vulkan.org/refpages/latest/refpages/source/vkQueuePresentKHR.html

[72] Khronos. VkValidationFeaturesEXT

https://docs.vulkan.org/refpages/latest/refpages/source/VkValidationFeaturesEXT.html

[73] Khronos. VkValidationFeatureEnableEXT

https://docs.vulkan.org/refpages/latest/refpages/source/VkValidationFeatureEnableEXT.html

[74] Khronos. VkDebugUtilsMessengerCallbackDataEXT

https://docs.vulkan.org/refpages/latest/refpages/source/VkDebugUtilsMessengerCallbackDataEXT.html

[75] LunarG, Vulkan Validation Layers documentation. Synchronization Validation Usage — Known Limitations

https://vulkan.lunarg.com/doc/view/latest/linux/synchronization_usage.html

[76] Khronos. Vulkan Guide — Development Tools

https://docs.vulkan.org/guide/latest/development_tools.html

[77] Khronos. Vulkan Samples — Introduction and Requirements

https://docs.vulkan.org/samples/latest/README.html